

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÀI GIẢNG MÔN HỌC

KIẾN TRÚC MÁY TÍNH

CHƯƠNG 2 KHỐI XỬ LÝ TRUNG TÂM

*Tài liệu lưu hành nội bộ dùng cho sinh viên hệ đại học chính quy
ngành Công nghệ Kỹ thuật Điện tử – Viễn thông*

HÀ NỘI

MỤC LỤC

2 Khối xử lý trung tâm	1
Chuẩn bị: sáu điều cần biết trước khi đọc chương	2
2.1 Sơ đồ khối tổng quát và các thành phần chức năng của CPU	5
2.1.1 CPU là gì và nó làm gì	5
2.1.2 Ba thành phần bên trong CPU	6
2.1.3 Bốn việc CPU làm đi làm lại	7
2.1.4 Chu trình lệnh	8
2.2 Khối logic và số học (ALU)	11
2.2.1 ALU làm những gì	12
2.2.2 Bộ cộng	13
2.2.3 Phép trừ bằng mã bù hai	15
2.2.4 Phép dịch và phép quay bit	16
2.2.5 Phép nhân và phép chia	16
2.2.6 Đơn vị dấu phẩy động	17
2.2.7 Các cờ trạng thái	18
2.3 Khối điều khiển (CU)	19
2.3.1 Khối điều khiển làm gì	20
2.3.2 Các tín hiệu vào và ra của khối điều khiển	20
2.3.3 Vi thao tác	21
2.3.4 Hai cách chế tạo khối điều khiển	21
2.4 Các thanh ghi	24
2.4.1 Thanh ghi là gì và vì sao cần nó	24
2.4.2 Thanh ghi địa chỉ	24
2.4.3 Ngăn xếp và con trỏ ngăn xếp	25
2.4.4 Thanh ghi dữ liệu và thanh ghi đa năng	27
2.4.5 Thanh ghi lệnh	27
2.4.6 Thanh ghi trạng thái	28
2.4.7 Tập thanh ghi của một số bộ xử lý thật	28
2.5 Hệ thống bus trong CPU	29
2.5.1 Bus dữ liệu	30
2.5.2 Bus địa chỉ	30
2.5.3 Bus điều khiển	31
2.5.4 Đường dữ liệu bên trong CPU	31

2.6	Hiệu năng CPU	32
2.6.1	Hiệu năng nghĩa là gì	32
2.6.2	Tốc độ xung nhịp CPU	33
2.6.3	Thời gian chạy tính theo số chu kỳ	34
2.6.4	Thời gian chạy tính theo số lệnh	35
2.6.5	MIPS – và vì sao phải cẩn thận với nó	37
2.6.6	Những yếu tố ảnh hưởng đến hiệu năng	39
	Phụ lục A: Kỹ thuật ống dẫn lệnh	40
	Phụ lục B: Nhập môn hợp ngữ	41
	Phụ lục C: Thực hành trên web	50
	Câu hỏi ôn tập	53
	Bài tập	55
	Đáp án và hướng dẫn giải	61
	Bảng thuật ngữ Việt – Anh	71

CHƯƠNG 2

KHỐI XỬ LÝ TRUNG TÂM

MỤC TIÊU CỦA CHƯƠNG

Sau khi học xong chương này, sinh viên có khả năng:

- 1) Vẽ được sơ đồ khối của CPU và nói được ba thành phần bên trong nó làm gì.
- 2) Kể lại được sáu bước mà CPU làm với mỗi lệnh, và cho biết thanh ghi nào tham gia ở bước nào.
- 3) Giải thích được ALU cộng hai số bằng cách nào, xây dựng được bộ cộng toàn phần, cho biết vì sao phép trừ không cần mạch riêng, và phép dịch bit tương đương với nhân chia thế nào.
- 4) Kể được các tín hiệu đi vào và đi ra khỏi điều khiển, và so sánh được hai cách chế tạo khối điều khiển.
- 5) Phân loại được các thanh ghi, mô tả được ngăn xếp hoạt động ra sao, và xác định được bốn cờ ZF, SF, CF, OF sau một phép toán cho trước.
- 6) Nêu được ba loại bus làm gì, độ rộng của chúng ảnh hưởng tới cái gì, và mô tả được đường dữ liệu bên trong CPU.
- 7) Tính được thời gian chạy chương trình, CPI trung bình và MIPS; so sánh được hiệu năng của hai bộ xử lý.
- 8) Đọc và viết được chương trình hợp ngữ đơn giản có rẽ nhánh, vòng lặp và chương trình con, rồi chạy thử được trên trình mô phỏng trực tuyến.

Chương này nói về cái gì

Chương 1 cho thấy máy tính gồm ba khối lớn: bộ xử lý trung tâm, bộ nhớ và hệ thống vào-ra. Chương này mở khối thứ nhất ra xem bên trong: **bộ xử lý trung tâm**, viết tắt là CPU.

Nội dung chia thành sáu phần, đúng theo thứ tự bài giảng trên lớp:

- Phần **2.1** dựng bức tranh tổng thể: CPU có những bộ phận nào, và một lệnh được thực hiện qua những bước nào.
- Bốn phần tiếp theo mô tả từng bộ phận đã kể ở phần một – **2.2** đơn vị số học và logic, **2.3** khối điều khiển, **2.4** tập thanh ghi, **2.5** hệ thống bus nối chúng lại với nhau.
- Phần **2.6** chuyển sang một câu hỏi khác hẳn: sau khi đã biết CPU làm việc thế nào, ta *đo* xem nó nhanh hay chậm ra sao?

Cuối chương có ba phụ lục. **Phụ lục A** nói về ống dẫn lệnh. **Phụ lục B** dạy hợp ngữ cơ bản – đây là lúc bạn tự tay viết lệnh cho CPU chạy, thay vì chỉ đọc về nó. **Phụ lục C** giới thiệu ba trang web miễn phí chạy thẳng trên trình duyệt, không phải cài đặt gì – ở đó bạn nhìn thấy tận mắt mọi thứ chương này mô tả, và chạy thử được mọi đoạn mã in trong Phụ lục B.

Nên đọc chương này thế nào

Trước hết, đừng bỏ qua phần *Chuẩn bị* ngay sau đây. Nó chỉ dài vài trang và nhắc lại sáu thứ cơ bản (bit, số nhị phân, số âm, hệ mười sáu, cổng logic, đơn vị thời gian). Thiếu chúng thì đọc tới giữa chương sẽ tắc.

Bốn phần giữa nên đọc *đúng thứ tự*, vì các khái niệm xây nối tiếp nhau: hiểu chu trình lệnh rồi mới hiểu vì sao cần thanh ghi PC và IR; hiểu thanh ghi rồi mới hiểu tín hiệu vào của khối điều khiển.

Riêng phần 2.6 về hiệu năng, kinh nghiệm cho thấy đọc công thức thôi là chưa đủ. Phải tự cầm bút làm lại tất cả các ví dụ, rồi làm tiếp bài tập cuối chương. Chỉ khi tự làm bạn mới phát hiện ra mình đang nhầm chỗ nào.

Trong bài có bốn loại hộp, mỗi loại một mục đích:

- hộp *Định nghĩa* (viền xanh lá) – khái niệm cần thuộc;
- hộp *Ví dụ* (nền xanh nhạt) – bài mẫu có lời giải từng bước, hãy tự làm trước khi xem lời giải;
- hộp *Ghi chú* (vạch cam bên trái) – lời giải thích thêm, chỗ hay nhầm, hoặc liên hệ thực tế;
- hộp *Ghi nhớ* (nền xanh lá nhạt) – tóm tắt cuối mỗi phần, dùng để ôn nhanh trước khi thi.

Riêng từ Phụ lục B trở đi còn có các *khung mã hợp ngữ* nền xám. Đừng chỉ đọc chúng: hãy gõ lại vào trang mô phỏng ở Phụ lục C và chạy thử. Học hợp ngữ mà không tự chạy thì gần như không đọng lại gì.

Chuẩn bị: sáu điều cần biết trước khi đọc chương

Chương này nói về phần cứng bên trong máy tính. Để đọc được, bạn cần sáu thứ rất cơ bản dưới đây. Tất cả đều đã có ở chương 1, nhưng ta nhắc lại thật ngắn để bạn không phải lật ngược lại. Nếu thấy quen rồi thì đọc lướt qua cũng được.

1. Bit và byte

Máy tính chỉ biết hai trạng thái: có điện và không có điện. Ta quy ước gọi hai trạng thái đó là **1** và **0**. Mỗi ô nhớ giữ được một trong hai giá trị ấy gọi là một **bit**.

Một bit thì ít thông tin quá, nên người ta gộp 8 bit lại thành một **byte**. Một byte biểu diễn được $2^8 = 256$ giá trị khác nhau, đủ để chứa một chữ cái hoặc một số từ 0 đến 255.

Khi viết dãy bit, ta hay tách thành từng nhóm bốn cho dễ đọc, ví dụ 0110 1101 thay vì 01101101.

2. Số nhị phân

Trong hệ mười quen thuộc, số 205 có nghĩa là $2 \times 100 + 0 \times 10 + 5 \times 1$. Mỗi vị trí ứng với một lũy thừa của 10.

Hệ nhị phân làm y hệt như vậy, chỉ đổi 10 thành 2. Số 1101 có nghĩa là

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13.$$

Cách tính nhanh: viết các trọng số phía trên mỗi bit rồi cộng những chỗ có bit 1.

Trọng số	128	64	32	16	8	4	2	1
Dãy bit	0	0	0	0	1	1	0	1

Cộng các trọng số ở chỗ có bit 1: $8 + 4 + 1 = 13$.

3. Số âm: mã bù hai

Máy tính không có dấu trừ. Muốn biểu diễn số âm, người ta dùng một mẹo gọi là **mã bù hai**. Cách làm gồm hai bước:

1. Đảo toàn bộ các bit (0 thành 1, 1 thành 0). Kết quả gọi là *bù một*.
2. Cộng thêm 1.

Ví dụ tìm -13 trên 8 bit: $13 = 0000\ 1101$; đảo bit được $1111\ 0010$; cộng 1 được $-13 = 1111\ 0011$.

Khi dùng mã bù hai, **bit cao nhất chính là bit dấu**: bằng 0 thì số dương, bằng 1 thì số âm. Với 8 bit, dải biểu diễn được là từ -128 đến $+127$.

Muốn đọc giá trị của một số bù hai 8 bit, chỉ cần coi trọng số của bit cao nhất là -128 thay vì $+128$. Ví dụ $1111\ 0011 = -128 + 64 + 32 + 16 + 2 + 1 = -13$. Đúng.

Vì sao lại chọn cách rắc rối này? Vì nó khiến *phép trừ trở thành phép cộng*. Máy tính chỉ cần một mạch cộng duy nhất là làm được cả cộng lẫn trừ. Ta sẽ thấy điều đó ở mục 2.2.3.

4. Hệ mười sáu

Dãy bit dài rất khó đọc và dễ chép nhầm. Vì vậy người ta viết gọn lại bằng **hệ mười sáu** (hexadecimal, thường gọi tắt là hexa). Hệ này có 16 chữ số: từ 0 đến 9, rồi A, B, C, D, E, F thay cho 10 đến 15.

Quy tắc đổi rất dễ: **mỗi bốn bit đổi thành đúng một chữ số hexa**.

0000=0	0001=1	0010=2	0011=3	0100=4	0101=5	0110=6	0111=7
1000=8	1001=9	1010=A	1011=B	1100=C	1101=D	1110=E	1111=F

Ví dụ: $1111\ 0011 = (F3)_{16}$. Trong tài liệu này, số hệ mười sáu luôn được viết kèm chỉ số 16 ở dưới, như $(F3)_{16}$, để khỏi nhầm với số hệ mười.

5. Bốn phép logic cơ bản

Ngoài cộng trừ, máy tính còn làm bốn phép trên từng bit. Bạn chỉ cần nhớ bảng nhỏ này:

Phép	Cho kết quả 1 khi	Ví dụ trên 1 bit
NOT (đảo)	đầu vào bằng 0	NOT 0 = 1; NOT 1 = 0
AND (và)	cả hai đầu vào đều bằng 1	1 AND 1 = 1; còn lại đều bằng 0
OR (hoặc)	ít nhất một đầu vào bằng 1	0 OR 0 = 0; còn lại đều bằng 1
XOR (hoặc loại trừ)	hai đầu vào khác nhau	$0 \oplus 1 = 1$; $1 \oplus 1 = 0$

Ký hiệu thường dùng: $A \cdot B$ hoặc AB là AND, $A + B$ là OR, $A \oplus B$ là XOR, $\bar{A}$ là NOT. Khi làm trên nhiều bit thì phép được thực hiện riêng rẽ ở từng cột, không có nhớ sang cột bên cạnh.

Mạch điện thực hiện các phép này gọi là **cổng logic**. Cổng logic là viên gạch nhỏ nhất xây nên mọi mạch số, kể cả CPU.

6. Đơn vị thời gian và tần số

Phần cuối chương có nhiều phép tính về thời gian. Bạn cần thuộc các tiền tố sau:

Thời gian	Tần số
1 ms (mili giây) = 10^{-3} s	1 kHz = 10^3 Hz
1 μ s (micro giây) = 10^{-6} s	1 MHz = 10^6 Hz
1 ns (nano giây) = 10^{-9} s	1 GHz = 10^9 Hz
1 ps (pico giây) = 10^{-12} s	

Quan hệ giữa tần số f và chu kỳ T luôn là $T = 1/f$. Ví dụ máy chạy ở 2 GHz thì mỗi chu kỳ dài $T = 1/(2 \times 10^9) = 0,5$ ns.

Ghi nhớ trước khi vào bài

Bit là 0 hoặc 1. Byte là 8 bit. Số nhị phân đọc bằng cách cộng trọng số. Số âm dùng mã bù hai: đảo bit rồi cộng 1, bit cao nhất là bit dấu. Bốn bit gộp thành một chữ số hexa. Bốn phép logic là NOT, AND, OR, XOR. Và $T = 1/f$.

2.1 Sơ đồ khối tổng quát và các thành phần chức năng của CPU

2.1.1 CPU là gì và nó làm gì

Định nghĩa 2.1: Bộ xử lý trung tâm

Bộ xử lý trung tâm, còn gọi là khối xử lý trung tâm (CPU – Central Processing Unit), là thành phần của máy tính làm hai việc: *điều khiển* hoạt động của toàn bộ hệ thống và *xử lý* dữ liệu, theo đúng dãy lệnh của chương trình đang nằm trong bộ nhớ chính.

Định nghĩa trên hơi khô. Ta hãy hình dung bằng một ví dụ đời thường.

Bạn vào bếp nấu ăn theo một công thức viết sẵn. Quyển sổ công thức đặt trên bàn, mỗi dòng là một việc phải làm: “lấy 200 g bột”, “cho vào tô”, “khuấy đều”. Bạn đọc từng dòng, làm đúng dòng đó, rồi đọc dòng tiếp theo.

Máy tính hoạt động y hệt như vậy:

- **Quyển sổ công thức** là chương trình nằm trong bộ nhớ. Mỗi dòng gọi là một *lệnh*.
- **Người nấu** là CPU. Nó đọc lệnh, hiểu lệnh, rồi làm.
- **Kệ nguyên liệu** là bộ nhớ dữ liệu.
- **Cửa nhận nguyên liệu và trả món ăn** là hệ thống vào–ra.

Một điểm cần nói ngay để khỏi hiểu nhầm: trong máy tính, quyển sổ công thức và kệ nguyên liệu thực ra nằm chung *một* bộ nhớ, chỉ khác vùng. Lệnh và dữ liệu đều là những dãy bit như nhau, và ta sẽ thấy điều đó quan trọng thế nào ngay dưới đây.

Từ hình ảnh đó, ba điều quan trọng nhất của định nghĩa trở nên dễ hiểu.

CPU có hai việc chứ không phải một

Khi mới học, ai cũng nghĩ CPU chỉ để tính toán: cộng, trừ, so sánh. Đó mới là một nửa. Nửa còn lại là *điều khiển*: quyết định làm gì trước, làm gì sau, khi nào thì nhảy sang chỗ khác.

Trong quyển sổ công thức, không phải dòng nào cũng là phép tính. Có dòng “lấy bột ra khỏi kệ” – đó là chuyển dữ liệu. Có dòng “nếu bột đã hết thì bỏ qua bước sau” – đó là rẽ nhánh. Trong một chương trình thật, số lệnh chuyển dữ liệu và rẽ nhánh thường *nhiều hơn* số lệnh tính toán.

CPU chạy theo chương trình nằm trong bộ nhớ

Bản thân CPU không “biết” phải làm gì. Nó chỉ biết một việc duy nhất: lấy lệnh kế tiếp từ bộ nhớ ra, làm đúng lệnh đó, rồi lại lấy lệnh kế tiếp. Cứ thế cho tới khi tắt máy.

Đây chính là *nguyên lý chương trình lưu trữ* của von Neumann đã học ở mục 1.5.1. Hệ quả của nó rất lớn: muốn máy tính làm việc khác thì chỉ cần thay nội dung bộ nhớ, không phải thay

phần cứng. Nhờ vậy cùng một chiếc máy tính vừa soạn được văn bản, vừa nghe được nhạc, vừa chơi được game.

CPU là thành phần nhanh nhất trong hệ thống

Con số thật khó hình dung, nên ta phóng đại lên cho dễ thấy. Giả sử một chu kỳ làm việc của CPU kéo dài đúng *một giây*. Khi đó, theo cùng tỉ lệ:

Thao tác	Thời gian thật	Nếu 1 chu kỳ CPU = 1 giây
Một chu kỳ của CPU	$\approx 0,3 \text{ ns}$	1 giây
Lấy dữ liệu từ bộ nhớ đệm	$\approx 1 \text{ ns}$	3 giây
Lấy dữ liệu từ RAM	$\approx 60 \text{ ns}$	3 phút
Đọc một khối từ ổ SSD	$\approx 100 \mu\text{s}$	4 ngày
Đọc một khối từ ổ đĩa cứng	$\approx 10 \text{ ms}$	1 năm

Bảng này giải thích gần như toàn bộ những kỹ thuật bạn sẽ học ở các chương sau. CPU nhanh đến mức nó thường xuyên phải *ngồi chờ* các thành phần khác. Chênh lệch tốc độ này chính là nguồn gốc của nhiều kỹ thuật quan trọng: bộ nhớ đệm (một vùng nhớ nhỏ nhưng nhanh, đặt sát CPU), ống lệnh (xem Phụ lục A), truy nhập bộ nhớ trực tiếp.

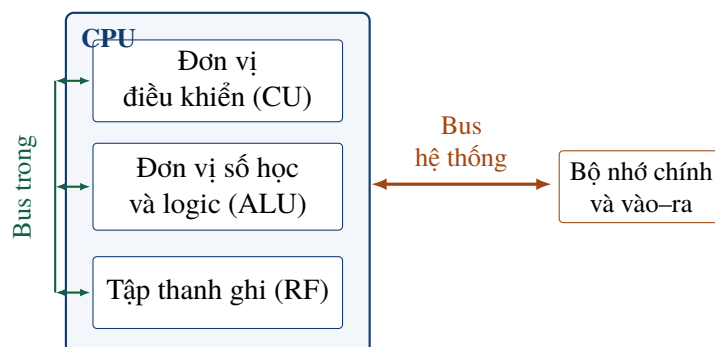
Cần phân biệt *bộ xử lý* với *bộ vi xử lý*. Bộ xử lý là tên gọi theo chức năng. Bộ vi xử lý (microprocessor) là bộ xử lý được làm gọn trên một con chip duy nhất. Ngày nay hai từ này dùng thay nhau được, nhưng ở các máy tính đời cũ, một bộ xử lý có thể to bằng cả một tủ máy gồm nhiều bảng mạch.

2.1.2 Ba thành phần bên trong CPU

Mở CPU ra xem, ta thấy ba khối chức năng (hình 2.1). Vẫn dùng hình ảnh nhà bếp để nhớ cho nhanh:

- **Đơn vị điều khiển** (CU – Control Unit) là *người quản đốc*. Nó đọc lệnh, hiểu lệnh, rồi ra lệnh cho các khối khác làm việc đúng thứ tự và đúng thời điểm. CU không tự tính toán gì cả; nó chỉ phát ra *tín hiệu điều khiển*, giống như tiếng hô của người chỉ huy.
- **Đơn vị số học và logic** (ALU – Arithmetic and Logic Unit) là *chiếc máy tính bỏ túi*. Đưa cho nó hai số và bảo cộng, nó cộng. Đưa hai dãy bit và bảo AND, nó AND. Nó không biết vì sao phải làm, chỉ biết làm.
- **Tập thanh ghi** (RF – Register File) là *mặt bàn làm việc*. Đây là chỗ để tạm những thứ đang dùng dở. Mặt bàn thì nhỏ nhưng với tay tới ngay, còn kệ đồ (bộ nhớ) thì rộng nhưng phải đi lấy, mất thời gian.

Ba khối này nói chuyện với nhau qua *bus trong* (internal bus), và nói chuyện với thế giới bên ngoài qua *bus hệ thống* (system bus). Bus chỉ là tên gọi của một bó dây dẫn dùng chung, giống một hành lang mà mọi phòng đều mở cửa ra đó.



Hình 2.1. Sơ đồ khối tổng quát của CPU và ba thành phần chức năng cơ bản

Cách chia ba khối này là chia theo *chức năng*, không phải chia theo vị trí vật lý. Nếu soi kính hiển vi vào một con chip thật, bạn sẽ không thấy ba ô vuông tách bạch như hình vẽ: mạch của CU nằm xen kẽ với mạch của ALU, còn tập thanh ghi thì được xây bằng chính các ô nhớ SRAM đặt sát ALU để dây dẫn ngắn nhất có thể.

Dù vậy, cách chia này vẫn có ích nhất khi học, vì nó tách bạch ba câu hỏi hoàn toàn khác nhau: *làm gì* (CU quyết định), *làm thế nào* (ALU thực hiện) và *làm trên cái gì* (dữ liệu nằm ở RF).

Càng nhiều thanh ghi thì CPU càng nhanh, vì càng ít phải chạy ra bộ nhớ lấy dữ liệu. Nhưng không thể tăng số thanh ghi tùy ý. Lý do: trong mỗi lệnh máy phải có chỗ ghi rõ dùng thanh ghi nào, mà chỗ đó chỉ có vài bit. Muốn tăng gấp đôi số thanh ghi thì phải thêm một bit vào mỗi lệnh, làm lệnh dài ra và chương trình phình to.

2.1.3 Bốn việc CPU làm đi làm lại

Nhìn từ bên ngoài, mọi thứ CPU làm đều quy về bốn thao tác sau:

- (1) **Nhận lệnh** (fetch instruction): đọc lệnh từ bộ nhớ vào CPU. Giống như đọc dòng tiếp theo trong quyển công thức.
- (2) **Giải mã lệnh** (decode instruction): xem lệnh đó bảo làm gì.
- (3) **Nhận dữ liệu** (fetch data): lấy dữ liệu cần thiết từ bộ nhớ hoặc từ cổng vào-ra, rồi **xử lý dữ liệu** (process data) bằng phép toán số học hoặc logic.
- (4) **Ghi dữ liệu** (write data): cất kết quả ra bộ nhớ hoặc gửi ra cổng vào-ra.

Bốn thao tác này được sắp lại thành một vòng khép kín, gọi là *chu trình lệnh*.

2.1.4 Chu trình lệnh

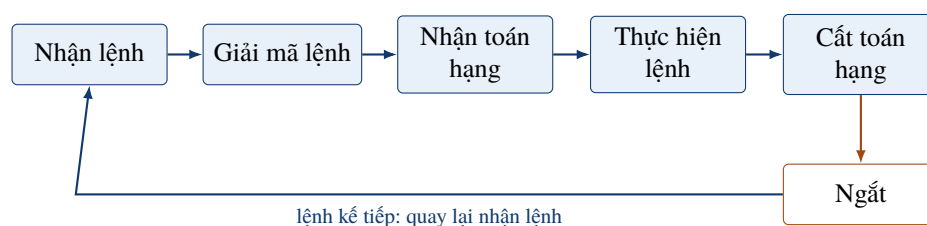
Định nghĩa 2.2: Chu trình lệnh

Chu trình lệnh (instruction cycle) là dãy các công đoạn mà CPU phải thực hiện để hoàn tất một lệnh máy. CPU lặp lại chu trình này liên tục từ lúc bật máy cho tới lúc tắt máy.

Chu trình lệnh gồm sáu công đoạn: nhận lệnh, giải mã lệnh, nhận toán hạng, thực hiện lệnh, cất toán hạng và ngắt.

Không phải lệnh nào cũng đi qua đủ sáu công đoạn. Lệnh `NOP` (không làm gì cả) chỉ có hai công đoạn đầu. Lệnh cộng hai thanh ghi thì không cần đi lấy hay cất dữ liệu ở bộ nhớ. Còn công đoạn ngắt chỉ xảy ra khi có ai đó đang “gọi” CPU.

Toán hạng (operand) chỉ đơn giản là dữ liệu mà lệnh làm việc trên đó. Trong phép tính $5 + 3$ thì 5 và 3 là hai toán hạng, còn dấu $+$ là phép toán. Từ này sẽ gặp rất nhiều, nên hãy nhớ luôn.



Hình 2.2. Sáu công đoạn của một chu trình lệnh

Công đoạn 1 – Nhận lệnh

Bắt đầu mỗi chu trình, CPU phải đi lấy lệnh từ bộ nhớ về. Nhưng lấy ở đâu? CPU cần một chỗ để nhớ “mình đang đọc tới dòng nào”. Chỗ đó là một thanh ghi đặc biệt tên là **PC** (Program Counter – bộ đếm chương trình).

Diễn biến như sau:

- PC đang giữ địa chỉ của lệnh sắp lấy. CPU đưa địa chỉ đó ra *bus địa chỉ*, tức là báo cho bộ nhớ biết cần lấy ở ô nào.
- CPU phát tín hiệu điều khiển đọc bộ nhớ.
- Bộ nhớ trả lệnh về qua *bus dữ liệu*. Lệnh này được đưa vào một thanh ghi khác tên là **IR** (Instruction Register – thanh ghi lệnh), nơi giữ lệnh trong suốt thời gian CPU xử lý nó.
- Ngay sau đó, nội dung PC tự động tăng lên để trở sang lệnh kế tiếp.

Vì sao PC được tăng ngay ở bước này, chứ không đợi làm xong lệnh mới tăng?

Vì như thế thì địa chỉ trở về đã sẵn sàng. Giả sử lệnh đang thực hiện là lệnh “gọi chương

trình con”, tức nhảy sang chỗ khác làm một việc rồi quay lại. Muốn quay lại đúng chỗ, CPU phải nhớ chỗ cũ. Nhờ PC đã tăng từ trước, chỗ cũ đang nằm sẵn trong PC, chỉ việc cất đi là xong.

Tăng bao nhiêu? Bằng đúng độ dài một lệnh. Nếu mỗi lệnh dài 4 byte và bộ nhớ đánh địa chỉ theo từng byte thì $PC \leftarrow PC + 4$. Ký hiệu $\leftarrow$ nghĩa là “gán giá trị bên phải vào bên trái”.

Công đoạn 2 – Giải mã lệnh

Lệnh trong IR chỉ là một dãy bit. CPU phải hiểu dãy bit đó nghĩa là gì. Việc này gọi là *giải mã* và do CU đảm nhiệm.

Giải mã xong, CU biết được: phải làm phép toán nào, dữ liệu nằm ở đâu, có bao nhiêu toán hạng, lệnh này dài bao nhiêu byte, toán hạng lấy theo kiểu nào (tức *chế độ địa chỉ*), và kết quả cất đi đâu.

Toàn bộ việc này diễn ra bên trong CPU, không phải chạy ra bộ nhớ, nên rất nhanh.

Công đoạn 3 – Nhận toán hạng

Bây giờ CPU cần lấy dữ liệu về để tính. Dữ liệu có thể nằm ngay trong lệnh, nằm trong thanh ghi, hoặc nằm trong bộ nhớ. Trường hợp nằm trong bộ nhớ có hai kiểu.

Hai kiểu này còn được gọi là hai *chế độ địa chỉ*.

Kiểu trực tiếp. Trong lệnh ghi thẳng địa chỉ của dữ liệu. Ví dụ lệnh nói “lấy dữ liệu ở ô số 250”. CPU làm ba bước:

1. đưa địa chỉ 250 ra bus địa chỉ;
2. phát tín hiệu điều khiển đọc;
3. nhận dữ liệu về.

Kiểu gián tiếp. Trong lệnh không ghi địa chỉ của dữ liệu, mà ghi địa chỉ của một ô *chứa* địa chỉ đó. Giống như tra sổ: “địa chỉ nhà An ghi ở trang 12” – phải mở trang 12 ra xem rồi mới biết đường mà đi. CPU làm sáu bước:

1. đưa địa chỉ ra bus địa chỉ;
2. phát tín hiệu điều khiển đọc;
3. nhận về nội dung ô nhớ – *đó mới chính là địa chỉ của dữ liệu*;
4. đưa địa chỉ vừa nhận được ra bus địa chỉ;
5. phát tín hiệu điều khiển đọc lần thứ hai;
6. lúc này mới nhận được dữ liệu thật.

Ta gọi mỗi lần CPU phát địa chỉ ra bus, phát tín hiệu điều khiển rồi nhận dữ liệu về là một **chu kỳ bus**. Theo cách gọi đó: kiểu trực tiếp tốn một chu kỳ bus để lấy toán hạng, còn kiểu gián tiếp tốn hai – tức chậm hơn đúng một chu kỳ bus. Bù lại, nó cho phép đổi địa chỉ dữ liệu ngay trong lúc chương trình đang chạy mà không phải sửa lệnh. Đó chính là cách con trỏ và mảng hoạt động – những thứ bạn sẽ gặp khi học lập trình.

Công đoạn 4 – Thực hiện lệnh

Đây là lúc lệnh thực sự được làm. Tùy loại lệnh mà CPU huy động khối nào:

- đọc hoặc ghi bộ nhớ;
- trao đổi dữ liệu với cổng vào–ra;
- chuyển dữ liệu từ thanh ghi này sang thanh ghi khác;
- làm phép toán số học hoặc logic – lúc này ALU vào cuộc;
- chuyển điều khiển, tức rẽ nhánh: thay đổi nội dung PC để nhảy sang chỗ khác.

Công đoạn 5 – Cất toán hạng

Nếu kết quả cần đưa ra bộ nhớ hoặc ra cổng vào–ra, CPU làm bốn bước:

1. đưa địa chỉ ra bus địa chỉ;
2. đưa dữ liệu cần ghi ra bus dữ liệu;
3. phát tín hiệu điều khiển ghi;
4. bộ nhớ chép dữ liệu trên bus vào đúng ô có địa chỉ đó.

Nếu kết quả để luôn trong thanh ghi thì bỏ qua công đoạn này.

Công đoạn 6 – Ngắt

Ngắt (interrupt) là tình huống một thiết bị bên ngoài cần CPU chú ý gấp. Ví dụ bạn gõ một phím: bàn phím phải báo cho CPU biết ngay, không thể đợi CPU chạy hết chương trình rồi mới hỏi.

Cuối mỗi chu trình lệnh, CPU kiểm tra xem có yêu cầu ngắt nào đang chờ không. Nếu có, và nếu CPU đang cho phép nhận ngắt, nó làm như sau:

- lấy nội dung PC – chính là địa chỉ sẽ quay về sau khi xử lý xong – đưa ra bus dữ liệu;
- đưa một địa chỉ ra bus địa chỉ, thường lấy từ thanh ghi **SP** (Stack Pointer – con trỏ ngăn xếp; sẽ học ở mục 2.4.3);
- phát tín hiệu điều khiển ghi bộ nhớ;
- địa chỉ trở về được ghi vào chỗ đó, gọi là *cất vào ngăn xếp*;

- nạp vào PC địa chỉ lệnh đầu tiên của đoạn chương trình chuyên xử lý ngắt.

Từ chu trình lệnh kế tiếp, CPU bắt đầu chạy đoạn chương trình xử lý ngắt. Xong việc, nó gặp lệnh “trở về từ ngắt”: địa chỉ cũ được lấy ngược từ ngăn xếp nạp lại vào PC, và chương trình chính chạy tiếp đúng chỗ đã dừng, như chưa từng bị gián đoạn.

Ví dụ 2.1: Theo dõi một chu trình lệnh từ đầu tới cuối

Cho biết: bộ nhớ đánh địa chỉ theo byte, mỗi lệnh dài 4 byte. Tại thời điểm đang xét $PC = (0100)_{16}$. Ô nhớ $(0100)_{16}$ chứa lệnh $\text{ADD } R1, [0250]$, nghĩa là “lấy nội dung ô nhớ $(0250)_{16}$ cộng vào thanh ghi R1”. Biết $R1 = (0025)_{16}$ và ô nhớ $(0250)_{16}$ chứa $(0011)_{16}$.

Hỏi: sau khi lệnh chạy xong, PC, IR và R1 bằng bao nhiêu?

Giải. Ta đi lần lượt từng công đoạn. Trong đó MAR là thanh ghi giữ địa chỉ gửi ra bus, còn MBR là thanh ghi giữ dữ liệu nhận về từ bus.

1. *Nhận lệnh.* $MAR \leftarrow PC = (0100)_{16}$. CPU đọc bộ nhớ, mã lệnh về tới IR. Ngay lập tức $PC \leftarrow PC + 4 = (0104)_{16}$.
2. *Giải mã.* CU đọc mã lệnh trong IR và nhận ra: đây là phép cộng, kết quả cất vào R1, toán hạng thứ hai nằm ở bộ nhớ theo kiểu trực tiếp.
3. *Nhận toán hạng.* $MAR \leftarrow (0250)_{16}$. CPU đọc bộ nhớ, $MBR \leftarrow (0011)_{16}$.
4. *Thực hiện.* ALU tính $(0025)_{16} + (0011)_{16}$. Đổi sang hệ mười cho dễ kiểm: $37 + 17 = 54$, mà $54 = (0036)_{16}$. Kết quả ghi vào R1.
5. *Cất toán hạng.* Không có, vì kết quả nằm luôn trong thanh ghi. Các cờ được cập nhật: kết quả khác 0 và không có nhớ ra, nên $ZF = 0$ và $CF = 0$.

Đáp số: $PC = (0104)_{16}$, $R1 = (0036)_{16}$, IR vẫn giữ mã lệnh ADD , còn ô nhớ $(0250)_{16}$ không đổi.

Điều dễ nhầm nhất ở bài này: PC đã trở sang lệnh kế tiếp *ngay từ bước 1*, chứ không phải ở bước cuối.

Ghi nhớ

CPU có ba khối: CU (điều khiển), ALU (tính toán), RF (thanh ghi). Nó lặp mãi một vòng sáu bước: nhận lệnh $\rightarrow$ giải mã $\rightarrow$ nhận toán hạng $\rightarrow$ thực hiện $\rightarrow$ cất toán hạng $\rightarrow$ kiểm tra ngắt. Thanh ghi PC nhớ chỗ đang đọc tới, thanh ghi IR giữ lệnh đang làm.

2.2 Khối logic và số học (ALU)

2.2.1 ALU làm những gì

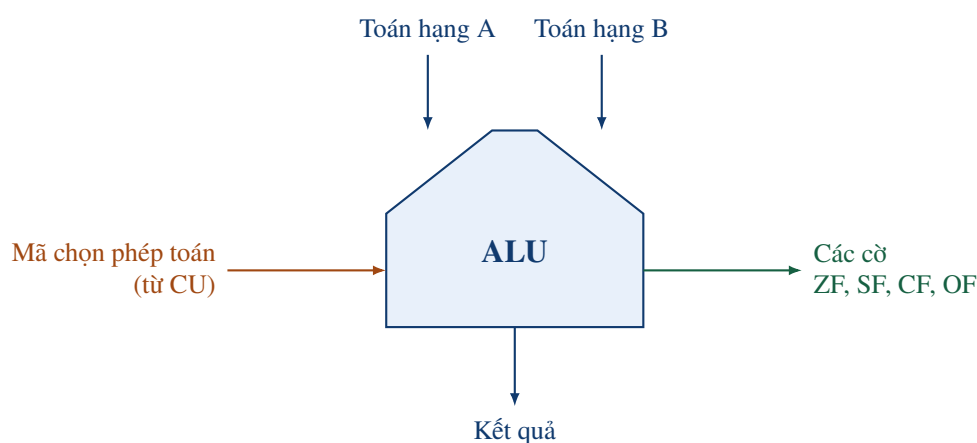
ALU là nơi mọi phép biến đổi dữ liệu được thực hiện. Nếu ví CPU là một văn phòng thì ALU là chiếc máy tính bỏ túi đặt trên bàn: nó không quyết định gì cả, chỉ nhận số vào, làm phép, trả kết quả ra.

Tập phép toán của ALU chia làm hai nhóm:

- **Phép toán số học:** cộng, trừ, nhân, chia, tăng thêm một (increment), bớt đi một (decrement), đảo dấu.
- **Phép toán logic:** AND, OR, XOR, NOT, dịch bit và quay bit.

Hình 2.3 cho thấy ALU nhìn từ bên ngoài. Nó có bốn nhóm đường vào–ra, cộng thêm một nhóm bit báo trạng thái:

- *hai đường vào* cho hai toán hạng *A* và *B*;
- *một đường điều khiển* từ CU tới, nói cho ALU biết lần này phải làm phép gì (cộng? AND? dịch trái?);
- *một đường ra* cho kết quả;
- và một nhóm bit phụ gọi là *các cờ*, báo lại xem kết quả có gì đặc biệt không – ta sẽ nói kỹ ở mục 2.2.7.



Hình 2.3. Sơ đồ khối của đơn vị số học và logic

Trong thực tế, ALU không phải một mạch duy nhất làm được mọi thứ. Nó là nhiều mạch nhỏ đặt cạnh nhau, mỗi mạch chuyên một việc: một mạch cộng, một mạch dịch, một mạch làm phép logic, một mạch nhân. Cả bốn mạch cùng chạy một lúc trên cùng dữ liệu, rồi mã chọn phép toán từ CU quyết định lấy kết quả của mạch nào đưa ra ngoài.

Cách làm này nghe có vẻ lãng phí – ba mạch kia tính xong rồi vứt đi, lại tốn thêm rất nhiều tranzito. Nhưng nó đổi được thứ quý hơn nhiều: *tốc độ*. Không phải chờ chọn xong mới tính, nên mọi phép đơn giản đều xong trong một chu kỳ.

2.2.2 Bộ cộng

Bộ cộng là mạch quan trọng nhất trong ALU. Lý do: hầu như phép nào cũng quy về phép cộng được. Trừ là cộng với số âm. Tăng một là cộng 1. So sánh là trừ rồi xem kết quả. Ngay cả nhân cũng là cộng nhiều lần.

Vậy nên ta xây bộ cộng từ những viên gạch nhỏ nhất.

Cộng hai bit: bộ cộng bán phần

Hãy bắt đầu từ bài toán bé nhất: cộng hai bit A và B . Có bốn trường hợp:

A	B	Bit tổng S	Bit nhớ C	Đọc là
0	0	0	0	không, viết 0
0	1	1	0	một, viết 1
1	0	1	0	một, viết 1
1	1	0	1	hai, viết 0 nhớ 1

Trường hợp cuối cho ta thấy: kết quả cần *hai* bit chứ không phải một. Bit thấp gọi là **bit tổng** S , bit cao gọi là **bit nhớ** C (carry) – đúng như khi ta cộng tay và nói “viết 0 nhớ 1”.

Bây giờ so cột S với bảng XOR, và cột C với bảng AND ở phần chuẩn bị. Chúng khớp hoàn toàn. Vậy:

$$S = A \oplus B, \quad C = A \cdot B.$$

Mạch làm việc này gọi là **bộ cộng bán phần** (half adder), chỉ cần một cổng XOR và một cổng AND. Sở dĩ gọi là “bán phần” vì nó còn thiếu một thứ: nó không nhận được bit nhớ từ cột bên phải chuyển sang. Cho nên nó chỉ dùng được cho bit thấp nhất.

Cộng ba bit: bộ cộng toàn phần

Khi cộng số nhiều bit, ở mỗi cột ta phải cộng *ba* thứ: bit của A , bit của B , và bit nhớ từ cột bên phải chuyển sang. Y hệt như khi cộng tay ở tiểu học.

Mạch cộng ba bit gọi là **bộ cộng toàn phần** (full adder). Đầu vào là A , B và C_{in} (bit nhớ vào); đầu ra là S (bit tổng) và C_{out} (bit nhớ ra).

Từ bảng 2.1 suy ra hai công thức:

$$S = A \oplus B \oplus C_{in}, \quad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B).$$

Không cần học thuộc hai công thức đó. Chỉ cần nhớ hai câu này, đọc thẳng từ bảng:

- bit tổng S bằng 1 khi *số bit 1 trong ba đầu vào là số lẻ*;
- bit nhớ ra C_{out} bằng 1 khi *có từ hai bit 1 trở lên*.

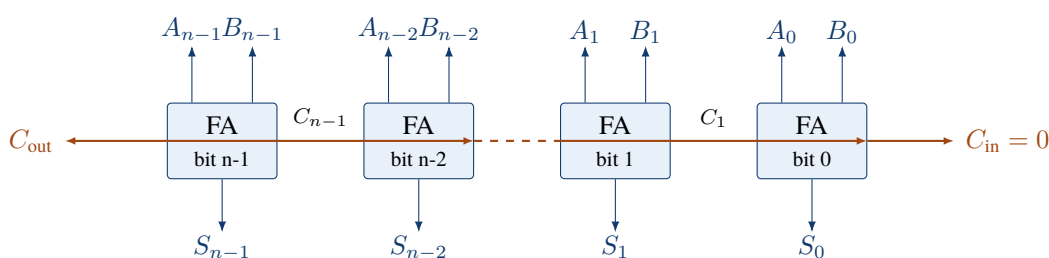
Bảng 2.1. Bảng chân lý của bộ cộng toàn phần

A	B	C_{in}	S	C_{out}	Giải thích
0	0	0	0	0	$0 + 0 + 0 = 0$
0	0	1	1	0	$0 + 0 + 1 = 1$
0	1	0	1	0	$0 + 1 + 0 = 1$
0	1	1	0	1	$0 + 1 + 1 = 2$, viết 0 nhớ 1
1	0	0	1	0	$1 + 0 + 0 = 1$
1	0	1	0	1	$1 + 0 + 1 = 2$, viết 0 nhớ 1
1	1	0	0	1	$1 + 1 + 0 = 2$, viết 0 nhớ 1
1	1	1	1	1	$1 + 1 + 1 = 3$, viết 1 nhớ 1

Cộng n bit: ghép nối tiếp

Muốn cộng hai số 8 bit, ta lấy 8 bộ cộng toàn phần xếp cạnh nhau. Bit nhớ ra của bộ bên phải nối thẳng vào bit nhớ vào của bộ bên trái – đúng như khi ta cộng tay, nhớ 1 sang cột kế tiếp. Bộ ở tận cùng bên phải có $C_{in} = 0$ vì chẳng có cột nào bên phải nó cả.

Mạch này gọi là **bộ cộng nối tiếp** (ripple-carry adder), hình 2.4. Chữ “ripple” nghĩa là gợn sóng, chỉ việc bit nhớ lan dần từ phải sang trái.



Hình 2.4. Bộ cộng nối tiếp n bit ghép từ n bộ cộng toàn phần

Cách ghép này có một nhược điểm lớn. Bộ cộng thứ hai không thể tính xong trước khi bộ thứ nhất trả bit nhớ ra. Bộ thứ ba lại phải đợi bộ thứ hai. Cứ thế, muốn có kết quả cuối cùng thì phải đợi bit nhớ chạy qua đủ n tầng.

Với $n = 8$ thì không sao. Với $n = 64$ – độ rộng của bộ xử lý hiện đại – thì độ trễ đó đủ lớn để làm chậm cả CPU.

Cách khắc phục gọi là **bộ cộng nhìn trước bit nhớ** (carry-lookahead adder). Ý tưởng rất hay: thay vì đợi bit nhớ truyền tới, ta *tính trước* nó bằng cách nhìn thẳng vào các bit đầu vào.

Cụ thể, ở mỗi cột ta xét hai câu hỏi:

- Cột này có *tự sinh* ra bit nhớ không? Có, khi cả A_i và B_i đều bằng 1. Đặt $G_i = A_i \cdot B_i$ (G là generate – sinh ra).
- Cột này có *cho bit nhớ đi qua* không? Có, khi A_i và B_i khác nhau. Đặt $P_i = A_i \oplus B_i$ (P là propagate – lan truyền).

Hai đại lượng G_i và P_i tính được ngay lập tức từ đầu vào, không phải đợi ai cả. Từ chúng, mọi bit nhớ được tính song song bằng một mạch riêng. Kết quả: thời gian cộng không còn tỉ lệ với n mà chỉ tỉ lệ với $\log n$ – nhanh hơn rất nhiều.

Bạn không cần tự thiết kế mạch nhìn trước bit nhớ. Chỉ cần nhớ *vấn đề* (bit nhớ lan chậm) và *hướng giải quyết* (tính trước bit nhớ bằng hai hàm sinh và lan truyền). Đó là mức hiểu đủ cho môn này.

2.2.3 Phép trừ bằng mã bù hai

Bây giờ tới điều thú vị nhất của mã bù hai: ALU *không cần mạch trừ*.

Nhớ lại: số âm của B chính là $\overline{B} + 1$ (đảo bit rồi cộng 1). Mà trừ đi một số thì bằng cộng với số âm của nó. Vậy:

$$A - B = A + (-B) = A + \overline{B} + 1.$$

Trên mạch, việc này rẽ đến bất ngờ. Chỉ cần thêm:

- một hàng cổng XOR ở đầu vào B – vì $B \oplus 1 = \overline{B}$ còn $B \oplus 0 = B$, nên một tín hiệu điều khiển là đủ để chọn đảo hay không đảo;
- nối chính tín hiệu điều khiển đó vào C_{in} của bộ cộng thấp nhất – thế là cộng thêm 1 xong.

Đặt tín hiệu điều khiển bằng 0 thì mạch cộng, bằng 1 thì mạch trừ. Một mạch, hai công dụng.

Ví dụ 2.2: Thực hiện phép trừ $58 - 91$ trên 8 bit

Hãy tính $58 - 91$ trên số bù hai 8 bit và cho biết giá trị các cờ.

Giải. Làm theo bốn bước.

Bước 1 – đổi hai số sang nhị phân. $58 = 32 + 16 + 8 + 2 = 0011\ 1010$ và $91 = 64 + 16 + 8 + 2 + 1 = 0101\ 1011$.

Bước 2 – tìm -91 . Đảo bit của $0101\ 1011$ được $1010\ 0100$, cộng thêm 1 được $-91 = 1010\ 0101$.

Bước 3 – cộng.

$$\begin{array}{r} 0011\ 1010 \\ + 1010\ 0101 \\ \hline 1101\ 1111 \end{array}$$

Bước 4 – đọc kết quả. Bit cao nhất bằng 1 nên đây là số âm. Giá trị: $-128 + 64 + 16 + 8 + 4 + 2 + 1 = -33$. Mà $58 - 91 = -33$. Đúng.

Các cờ. Phần này chỉ cần đọc lướt lúc này; mục 2.2.7 sẽ giải thích đầy đủ, sau đó hãy quay lại đọc kỹ bốn dòng dưới đây. $ZF = 0$ vì kết quả khác 0; $SF = 1$ vì bit cao nhất bằng 1; $CF = 1$ vì phép trừ này phải mượn; $OF = 0$ vì hai toán hạng khác dấu nên không thể

trần.

2.2.4 Phép dịch và phép quay bit

Dịch bit là phép rẻ nhất trong ALU. Với phép dịch cố định một chỗ, nó không cần cổng logic nào cả – chỉ là việc nối lại dây, lấy bit ở vị trí này nối sang vị trí bên cạnh. Nhưng nó lại rất hay dùng.

Có ba kiểu dịch và hai kiểu quay:

- **Dịch trái logic** (SHL): mọi bit đi sang trái một chỗ, bit 0 chèn vào từ bên phải. Kết quả bằng số cũ *nhân 2*. Dễ hiểu thôi: trong hệ mười, dịch trái một chỗ là nhân 10; trong hệ hai thì là nhân 2.
- **Dịch phải logic** (SHR): mọi bit đi sang phải, bit 0 chèn vào từ bên trái. Kết quả bằng số cũ *chia 2*, nhưng chỉ đúng với *số không dấu*.
- **Dịch phải số học** (SAR): giống SHR, nhưng bit chèn vào từ bên trái là bản sao của bit dấu cũ. Nhờ vậy số âm vẫn là số âm sau khi dịch. Đây mới là phép chia 2 đúng cho *số có dấu*.
- **Quay trái, quay phải** (ROL, ROR): bit bị đẩy ra ở một đầu được đưa vòng lại đầu kia. Không mất bit nào, giống như xoay một vòng tròn.

Dùng nhầm SHR cho số có dấu là lỗi kinh điển, ai học cũng mắc ít nhất một lần. Xét $A = 1111\ 0000$, hiểu là số bù hai thì $A = -16$.

- Dịch phải *số học*: $1111\ 1000 = -8$. Đúng bằng $-16 \div 2$.
- Dịch phải *logic*: $0111\ 1000 = +120$. Sai hoàn toàn.

Khi lập trình bằng C, bạn không phải chọn tay: kiểu `int` (có dấu) sẽ sinh ra lệnh SAR, còn `unsigned int` (không dấu) sinh ra lệnh SHR. Nhưng phải hiểu vì sao, để khỏi ngạc nhiên khi thấy kết quả lạ.

2.2.5 Phép nhân và phép chia

Nhân hai số n bit cho kết quả dài tới $2n$ bit – ví dụ nhân hai số 8 bit có thể ra số 16 bit. Đó là lý do phép nhân luôn phức tạp hơn phép cộng.

Thuật toán cơ bản gọi là *dịch và cộng*, và nó chính là cách bạn nhân tay ở tiểu học, chỉ khác là làm trong hệ hai. Ta duyệt lần lượt từng bit của số nhân, từ phải sang trái:

- nếu bit đó bằng 1 thì cộng số bị nhân (đã dịch trái đúng số chỗ) vào tổng;
- nếu bit đó bằng 0 thì bỏ qua;
- dịch trái số bị nhân thêm một chỗ, rồi xét bit tiếp theo.

Nhân hai số n bit như vậy tốn n bước. Với số có dấu, có một thuật toán khéo hơn tên là *thuật toán Booth*, xử lý thẳng số bù hai mà không phải tách dấu ra riêng.

Phép chia dùng thuật toán *dịch và trừ*, đối xứng với phép nhân, và tốn kém hơn hẳn. Trên bộ xử lý hiện đại, một phép chia số nguyên mất khoảng 20 đến 40 chu kỳ, trong khi một phép cộng chỉ mất một chu kỳ. Đó là lý do người lập trình giỏi tránh phép chia khi có thể.

Ví dụ 2.3: Nhân bằng phép dịch trên vi điều khiển

Nhiều vi điều khiển 8 bit rẻ tiền không có lệnh nhân. Hãy thực hiện phép nhân $A \times 10$ chỉ bằng phép dịch và phép cộng.

Giải. Ý tưởng: tách 10 thành tổng các lũy thừa của 2, vì nhân với lũy thừa của 2 chính là dịch trái.

$$10 = 8 + 2 = 2^3 + 2^1 \implies A \times 10 = (A \ll 3) + (A \ll 1),$$

trong đó $\ll$ là ký hiệu dịch trái.

Kiểm tra với $A = 13 = 00001101$:

- $A \ll 3 = 01101000 = 104$ (đúng bằng 13×8);
- $A \ll 1 = 00011010 = 26$ (đúng bằng 13×2);
- tổng: $104 + 26 = 130 = 13 \times 10$. Đúng.

Vậy một phép nhân được thay bằng hai phép dịch và một phép cộng – rẻ hơn nhiều. Chính trình biên dịch cũng làm mẹo này: gộp phép nhân với hằng số, nó thường tự đổi thành dãy dịch và cộng.

2.2.6 Đơn vị dấu phẩy động

Cho tới giờ ta chỉ nói về số nguyên. Còn số thực, kiểu 3,14 hay $-0,5$, thì sao?

Số thực trong máy tính được lưu theo chuẩn IEEE 754 mà bạn đã học ở mục 1.6.7: một bit dấu, một nhóm bit cho số mũ, một nhóm bit cho phần định trị. Cộng hai số kiểu này phức tạp hơn hẳn cộng hai số nguyên, vì phải:

1. tách riêng ba trường dấu, mũ, định trị;
2. cân bằng số mũ của hai số (dịch phần định trị cho tới khi hai số mũ bằng nhau);
3. cộng phần định trị;
4. chuẩn hoá lại kết quả và làm tròn.

Vì phức tạp như vậy nên người ta giao việc này cho một khối riêng, gọi là **đơn vị dấu phẩy động** (FPU – Floating Point Unit).

Ngày xưa FPU là một con chip *đồng xử lý* (coprocessor) rời bán kèm: 8087 đi với 8086,

80287 đi với 80286. Ai không mua thì máy vẫn chạy, chỉ chậm. Từ đời 80486 trở đi, FPU được tích hợp thẳng vào cùng chip với CPU và trở thành thứ mặc định phải có.

Các vi điều khiển nhỏ thì khác. ESP32 có FPU cho số thực đơn chính xác, nhưng nhiều dòng AVR và ARM Cortex-M0 thì không có. Trên những chip đó, phép cộng số thực được trình biên dịch thay bằng một chương trình con viết bằng phần mềm, chạy chậm hơn hàng chục lần. Đây là điều cần nhớ khi lập trình nhúng: một dòng lệnh trông vô hại như $x = a * b$; với float có thể tốn hàng trăm chu kỳ.

2.2.7 Các cờ trạng thái

Sau mỗi phép toán, ALU không chỉ trả kết quả. Nó còn bật hoặc tắt vài bit nhỏ để báo lại xem kết quả có gì đáng chú ý không. Những bit đó gọi là **cờ** (flag).

Cờ giống như đèn báo trên bảng điều khiển ô tô: kết quả chính là chiếc xe chạy được bao nhiêu km, còn đèn báo cho biết có gì bất thường.

Bốn cờ phép toán quan trọng nhất được tổng hợp ở bảng 2.2.

Bảng 2.2. Bốn cờ phép toán cơ bản

Cờ	Tên đầy đủ	Được đặt lên 1 khi
ZF	Zero Flag – cờ rỗng	kết quả của phép toán bằng 0
SF	Sign Flag – cờ dấu	kết quả phép toán nhỏ hơn 0, tức bit cao nhất bằng 1
CF	Carry Flag – cờ nhớ	phép cộng có nhớ ra ngoài bit cao nhất. Với phép trừ thì CF là cờ mượn: bằng 1 khi phải mượn, tức khi <i>không</i> có nhớ ra khỏi bit cao nhất. Đây là cờ báo tràn đối với <i>số không dấu</i>
OF	Overflow Flag – cờ tràn	cộng hai số nguyên cùng dấu mà kết quả lại ra dấu ngược lại. Đây là cờ báo tràn đối với <i>số có dấu</i>

Vì sao lại cần tới hai cờ báo tràn?

Đây là chỗ khó nhất của mục này, nên ta nói thật kỹ.

Hãy nhớ lại: một dãy 8 bit như 1111 1111 *tự nó* không mang dấu. Nếu ta bảo đó là số không dấu thì nó là 255. Nếu ta bảo đó là số bù hai thì nó là -1. Cùng một dãy bit, hai cách hiểu, hai giá trị khác nhau.

ALU thì *không biết* chương trình đang hiểu theo cách nào. Nó chỉ cộng các bit lại. Vậy làm sao báo tràn?

Câu trả lời: nó báo cả hai. Sau mỗi phép cộng, ALU đặt CF (tràn nếu hiểu là không dấu) và đặt OF (tràn nếu hiểu là có dấu). Việc nhìn cờ nào là do lệnh phía sau quyết định.

Đây chính là lý do tập lệnh x86 có *hai* bộ lệnh nhảy khác nhau cho việc so sánh: JA/JB (above/below) dùng cho số không dấu, còn JG/JL (greater/less) dùng cho số có dấu. Cùng một phép so sánh, nhưng dùng nhầm lệnh nhảy thì chương trình chạy sai.

Ví dụ 2.4: Xác định các cờ sau bốn phép cộng 8 bit

Xác định giá trị bốn cờ ZF, SF, CF, OF sau mỗi phép cộng số 8 bit sau đây: a) $0111\ 1111 + 0000\ 0001$; b) $1111\ 1111 + 0000\ 0001$; c) $1000\ 0000 + 1000\ 0000$; d) $0011\ 0000 + 0000\ 1111$.

Giải. Trước hết cộng bình thường, rồi mới xét từng cờ.

	Kết quả 8 bit	ZF	SF	CF	OF
a)	1000 0000	0	1	0	1
b)	0000 0000	1	0	1	0
c)	0000 0000	1	0	1	1
d)	0011 1111	0	0	0	0

Câu a) là $127 + 1$. Hiểu theo số có dấu: cộng hai số dương mà ra kết quả âm (-128), rõ ràng sai, nên $OF = 1$. Hiểu theo số không dấu: $127 + 1 = 128$, hoàn toàn nằm trong dải $0-255$, không có nhớ ra ngoài, nên $CF = 0$.

Câu b) Hiểu theo số không dấu là $255 + 1 = 256$, vượt quá 255 nên tràn, $CF = 1$. Hiểu theo số có dấu là $(-1) + 1 = 0$, hoàn toàn đúng, nên $OF = 0$. Kết quả bằng 0 nên $ZF = 1$.

Câu c) Hiểu theo số có dấu là $(-128) + (-128) = -256$, vượt xa dải cho phép, nên $OF = 1$. Hiểu theo số không dấu là $128 + 128 = 256$, cũng tràn, nên $CF = 1$. Lần này cả hai cờ cùng bật.

Câu d) $48 + 15 = 63$, đúng ở cả hai cách hiểu, không cờ nào bật (ngoài việc kết quả khác 0 và dương).

Bốn trường hợp trên cho thấy rõ: CF và OF hoàn toàn độc lập. Có thể chỉ một cờ bật, có thể cả hai, có thể không cờ nào.

Ghi nhớ

ALU làm phép số học và phép logic. Viên gạch của nó là bộ cộng toàn phần, ghép n cái lại thành bộ cộng n bit. Nhờ mã bù hai, phép trừ dùng chung mạch với phép cộng. Dịch trái là nhân 2, dịch phải là chia 2 – nhưng số có dấu phải dùng dịch phải *số học*. Sau mỗi phép toán, bốn cờ ZF, SF, CF, OF cho biết kết quả có gì đặc biệt; CF dành cho số không dấu, OF dành cho số có dấu.

2.3 Khối điều khiển (CU)

2.3.1 Khối điều khiển làm gì

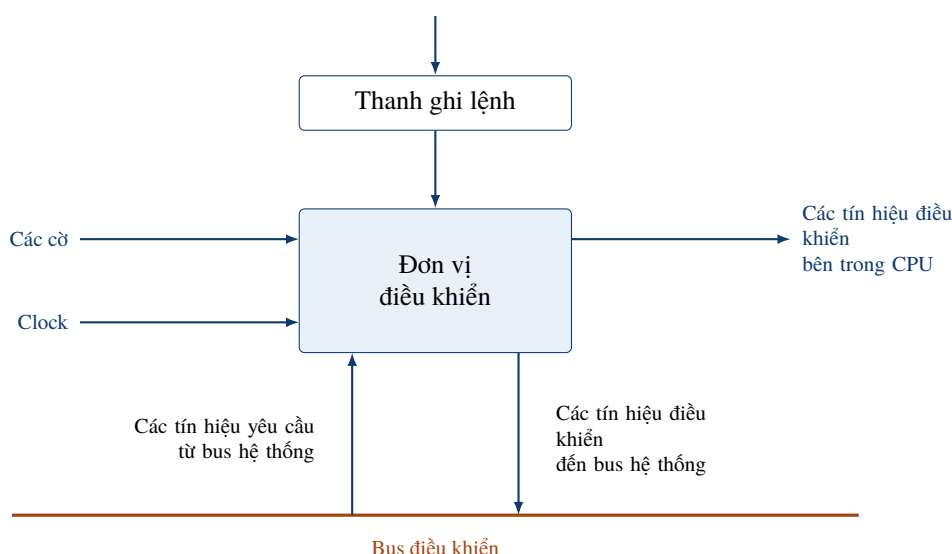
Khối điều khiển là “nhạc trưởng” của CPU. Một dàn nhạc có nhiều nhạc công giỏi nhưng nếu ai cũng chơi lúc mình thích thì chỉ ra tiếng ồn. Nhạc trưởng không chơi nhạc cụ nào cả; việc của ông là quyết định *ai chơi, chơi lúc nào*.

CU cũng vậy: nó không xử lý dữ liệu, mà quyết định ở mỗi thời điểm khối nào được phép làm gì. Cụ thể, CU có năm nhiệm vụ:

- (1) nhận lệnh từ bộ nhớ đưa vào thanh ghi lệnh;
- (2) tăng nội dung của PC để trở sang lệnh kế tiếp;
- (3) giải mã lệnh nhận được để biết lệnh yêu cầu làm gì;
- (4) phát ra các tín hiệu điều khiển để lệnh được thực hiện;
- (5) nhận tín hiệu yêu cầu từ bus hệ thống và đáp ứng các yêu cầu đó.

2.3.2 Các tín hiệu vào và ra của khối điều khiển

Hình 2.5 vẽ CU cùng những sợi dây nối vào nó. Có bốn nhóm dây đi vào (thông tin CU cần biết) và hai nhóm dây đi ra (mệnh lệnh CU phát đi).



Hình 2.5. Các tín hiệu vào và ra của khối điều khiển

Bốn nhóm tín hiệu vào:

- *Clock* – tín hiệu xung nhịp, lấy từ một mạch tạo dao động. Đây là nhịp đập chung của cả CPU, giống tiếng gõ nhịp của nhạc trưởng. Mỗi việc nhỏ mà CPU làm đều xảy ra trong một hoặc vài nhịp.
- *Instruction register* – mã lệnh lấy từ thanh ghi lệnh đưa sang để giải mã. Không có nó thì CU chẳng biết phải điều khiển cái gì.

- *Flags* – các cờ lấy từ thanh ghi cờ, cho biết kết quả phép toán vừa rồi ra sao. Nhóm tín hiệu này là thứ giúp máy tính *ra quyết định*: lệnh “nếu kết quả bằng 0 thì nhảy” chính là CU nhìn cờ ZF.
- *Control signals from control bus* – yêu cầu từ bên ngoài gửi vào qua bus điều khiển, chẳng hạn tín hiệu xin ngắt hoặc xin nhường bus.

Hai nhóm tín hiệu ra:

- *Control signals within the processor* – mệnh lệnh gửi tới các khối bên trong CPU: bảo thanh ghi này chuyển nội dung sang thanh ghi kia, bảo ALU chọn phép toán nào.
- *Control signals to control bus* – mệnh lệnh gửi ra ngoài, tới bộ nhớ và các mô-đun vào–ra. Ví dụ tín hiệu MEMR (đọc bộ nhớ), MEMW (ghi bộ nhớ), IOR (đọc cổng vào–ra), IOW (ghi cổng vào–ra).

2.3.3 Vi thao tác

Định nghĩa 2.3: Vi thao tác

Vi thao tác (micro-operation) là việc nhỏ nhất mà CPU làm được trong một nhịp đồng hồ, thường là chuyển nội dung giữa hai thanh ghi hoặc một phép toán đơn của ALU.

Ta đã biết chu trình lệnh có sáu công đoạn. Nhưng mỗi công đoạn vẫn còn to: nó không xong trong một nhịp. CU phải chẻ nhỏ tiếp thành các vi thao tác, mỗi nhịp làm một cái.

Ví dụ, công đoạn nhận lệnh được chẻ thành ba vi thao tác:

$$\begin{aligned} t_1 : \quad MAR &\leftarrow PC && \text{(đưa địa chỉ lệnh ra thanh ghi địa chỉ)} \\ t_2 : \quad MBR &\leftarrow M[MAR]; \quad PC &\leftarrow PC + \Delta && \text{(đọc bộ nhớ, đồng thời tăng PC)} \\ t_3 : \quad IR &\leftarrow MBR && \text{(chuyển lệnh sang thanh ghi lệnh)} \end{aligned}$$

Trong đó $M[MAR]$ đọc là “nội dung ô nhớ có địa chỉ nằm trong MAR”, còn Δ là độ dài của một lệnh.

Bạn thấy ở nhịp t_2 có *hai* việc được ghép chung. Có được như vậy là vì hai việc đó dùng hai đường dây khác nhau nên không giẫm chân nhau: một bên là đường dữ liệu từ bộ nhớ về, một bên là mạch cộng riêng của PC.

Ghép được càng nhiều vi thao tác vào một nhịp thì lệnh chạy càng nhanh. Đó là bài toán trung tâm khi thiết kế CPU, và ta sẽ gặp lại ở mục 2.5.4.

2.3.4 Hai cách chế tạo khối điều khiển

Ta đã biết CU phải làm gì. Nhưng chế tạo nó ra sao? Có hai trường phái.

Cách 1 – Điều khiển bằng mạch cứng

Ở cách này (hardwired control), CU chỉ là một mạch logic thuần túy, xây bằng các cổng AND, OR, NOT như ở phần chuẩn bị.

Cách nghĩ rất trực tiếp: mỗi tín hiệu điều khiển là một bóng đèn. Ta viết ra biểu thức logic cho biết bóng đèn ấy sáng khi nào – phụ thuộc mã lệnh, các cờ, và đang ở nhịp thứ mấy. Sau đó rút gọn biểu thức và nối cổng cho đúng.

Ưu điểm: rất nhanh. Tín hiệu ra chỉ chậm hơn tín hiệu vào vài tầng cổng.

Nhược điểm: mạch phức tạp kinh khủng và gần như không sửa được. Muốn thêm một lệnh mới vào tập lệnh thì phải thiết kế lại gần như toàn bộ mạch.

Vì thế cách này hợp với các CPU có tập lệnh nhỏ và đều đặn, tức các bộ xử lý RISC (Reduced Instruction Set Computer – máy tính tập lệnh rút gọn). Trường phái ngược lại, có nhiều lệnh và lệnh phức tạp, gọi là CISC (Complex Instruction Set Computer).

Cách 2 – Điều khiển bằng vi chương trình

Ở cách này (microprogrammed control), người ta nghĩ hoàn toàn khác: thay vì nối dây, ta *viết chương trình* cho CU.

Cụ thể như sau. Giả sử CPU có 42 tín hiệu điều khiển. Ta xếp 42 tín hiệu đó thành một hàng 42 bit, gọi là một **từ điều khiển**: bit nào bằng 1 thì tín hiệu ấy được bật ở nhịp này.

Mỗi lệnh máy cần một dãy từ điều khiển nối tiếp nhau, mỗi từ cho một nhịp. Dãy đó gọi là một **vi chương trình**. Toàn bộ vi chương trình của mọi lệnh được nạp sẵn vào một bộ nhớ chỉ đọc tốc độ cao nằm ngay trong CU, gọi là **bộ nhớ điều khiển**.

Mỗi từ điều khiển như vậy còn được gọi là một **vi lệnh** (microinstruction) – tên gọi này sẽ gặp lại ở ví dụ ngay dưới và ở bài tập 9.

Khi đó, “giải mã lệnh” chỉ còn là việc tra xem vi chương trình của lệnh này bắt đầu ở địa chỉ nào, rồi đọc lần lượt từng vi lệnh ra và phát đi.

Ví dụ 2.5: Tính dung lượng bộ nhớ điều khiển

Một CPU dùng điều khiển vi chương trình có 42 tín hiệu điều khiển. Mỗi vi lệnh, ngoài phần tín hiệu điều khiển, còn có 6 bit ghi địa chỉ vi lệnh kế tiếp và 2 bit chọn điều kiện rẽ nhánh. Bộ nhớ điều khiển chứa được 1024 vi lệnh. Hãy tính độ rộng một vi lệnh và dung lượng bộ nhớ điều khiển.

Giải. Một vi lệnh gồm ba phần, cộng lại là xong:

$$42 + 6 + 2 = 50 \text{ bit.}$$

Bộ nhớ điều khiển có 1024 vi lệnh, mỗi vi lệnh 50 bit:

$$1024 \times 50 = 51\,200 \text{ bit.}$$

Đổi sang byte (chia 8) rồi sang KiB (chia tiếp 1024):

$$51\,200 \div 8 = 6400 \text{ byte} = 6400 \div 1024 = 6,25 \text{ KiB.}$$

Con số này tuy nhỏ theo chuẩn ngày nay, nhưng vào thập niên 1960 thì một bộ nhớ ROM tốc độ cao cỡ đó là rất đắt. Đó là lý do phương pháp vi chương trình chỉ trở nên phổ biến khi công nghệ ROM đủ rẻ.

So sánh hai cách

Bảng 2.3. So sánh điều khiển mạch cứng và điều khiển vi chương trình

Tiêu chí	Điều khiển mạch cứng	Điều khiển vi chương trình
Cách chế tạo	Nối các cổng logic thành mạch	Nạp vi chương trình vào bộ nhớ ROM
Tốc độ	Nhanh	Chậm hơn, vì mỗi nhịp phải đọc bộ nhớ điều khiển
Thiết kế	Rất khó khi tập lệnh lớn	Dễ hơn nhiều, làm có hệ thống
Sửa đổi	Phải thiết kế lại mạch	Chỉ cần ghi lại nội dung ROM
Diện tích chip	Nhỏ nếu tập lệnh đơn giản	Lớn hơn, do tốn chỗ cho ROM
Hợp với	Tập lệnh RISC (đơn giản, đều)	Tập lệnh CISC (nhiều, phức tạp)
Ví dụ	MIPS, ARM, RISC-V	VAX-11, Intel x86 đời đầu

Bộ xử lý x86 ngày nay dùng *cả hai* cách cùng lúc. Những lệnh đơn giản và hay gặp thì giải mã bằng mạch cứng cho nhanh. Những lệnh phức tạp và ít gặp thì đẩy sang bộ giải mã vi chương trình. Nhờ vậy chip vừa giữ được tương thích với tập lệnh CISC đồ sộ tích lũy từ năm 1978, vừa chạy nhanh với phần lớn chương trình thực tế.

Cách làm này còn một lợi ích bất ngờ: khi phát hiện lỗi phần cứng, nhà sản xuất có thể sửa bằng một bản cập nhật vi mã (microcode update) nạp lúc khởi động máy, thay vì phải thu hồi chip.

Ghi nhớ

CU là nhạc trưởng: nó không tính toán, chỉ phát tín hiệu điều khiển. Nó nhận vào bốn thứ (xung nhịp, mã lệnh, các cờ, yêu cầu từ bus) và phát ra hai nhóm tín hiệu (vào trong CPU, và ra bus hệ thống). Mỗi công đoạn của chu trình lệnh được CU chia nhỏ thành các vi thao

tác, mỗi nhịp một cái. CU chế tạo được theo hai cách: mạch cứng (nhanh, khó sửa) hoặc vi chương trình (chậm hơn, dễ sửa).

2.4 Các thanh ghi

2.4.1 Thanh ghi là gì và vì sao cần nó

Thanh ghi là những ô nhớ rất nhỏ nằm *ngay bên trong* CPU, dùng để giữ tạm những gì CPU đang cần lúc này.

Hãy quay lại hình ảnh nhà bếp. Kệ nguyên liệu (bộ nhớ) chứa được rất nhiều thứ, nhưng mỗi lần cần gì lại phải đi tới kệ mà lấy. Còn mặt bàn ngay trước mặt bạn (thanh ghi) thì bé xíu, chỉ để được vài món, nhưng với tay là tới.

Vì nằm cùng chip với ALU nên thanh ghi là loại bộ nhớ *nhanh nhất* – và cũng *đắt nhất* – trong máy tính. Nó được xem là mức đầu tiên trong phân cấp bộ nhớ mà bạn sẽ học ở chương 4.

Chia theo việc người lập trình có chạm tới được hay không, ta có hai loại:

- **Thanh ghi lập trình được:** có tên hẳn hoi trong tập lệnh, người viết hợp ngữ (assembly – ngôn ngữ lập trình gần sát lệnh máy nhất) đọc và ghi được. Ví dụ AX, BX của x86, hay \$t0–\$t9 của MIPS.
- **Thanh ghi không lập trình được:** chỉ CPU dùng nội bộ, tập lệnh không có cách nào chạm tới. Ví dụ MAR, MBR, IR mà ta đã gặp.

Chia theo công dụng thì có năm nhóm: thanh ghi địa chỉ, thanh ghi dữ liệu, thanh ghi đa năng, thanh ghi điều khiển/trạng thái và thanh ghi lệnh. Ta xem lần lượt.

2.4.2 Thanh ghi địa chỉ

Nhóm này chuyên giữ *địa chỉ*, tức là “chỗ nào” chứ không phải “cái gì”. Nhóm này gồm bộ đếm chương trình PC, con trỏ dữ liệu DP, thanh ghi cơ sở, thanh ghi chỉ số, và con trỏ ngăn xếp SP. Riêng SP gắn liền với ngăn xếp nên được nói ở mục sau.

Bộ đếm chương trình PC

Bộ đếm chương trình PC (Program Counter), còn gọi là con trỏ lệnh IP (Instruction Pointer), giữ địa chỉ của lệnh sắp được lấy vào. Đây là cái “ngón tay dò dòng” của CPU trên quyển sổ công thức.

Sau khi một lệnh được lấy vào, nội dung PC tự động tăng để trỏ sang lệnh kế tiếp. Độ rộng của PC quyết định chương trình dài tối đa bao nhiêu: PC 32 bit thì đánh địa chỉ được 2^{32} vị trí.

Con trỏ dữ liệu DP

Con trỏ dữ liệu DP (Data Pointer) giữ địa chỉ của ô nhớ *dữ liệu* mà CPU muốn truy nhập. Nếu PC luôn trỏ vào vùng lệnh thì DP trỏ vào vùng dữ liệu.

Thanh ghi cơ sở và thanh ghi chỉ số

Hai thanh ghi này đi thành cặp và sinh ra để làm việc với *mảng*.

Hình dung một dãy phòng trọ đánh số liên tiếp. Bạn cần tìm phòng thứ i trong dãy. Thay vì nhớ địa chỉ của cả 100 phòng, bạn chỉ cần nhớ hai thứ: địa chỉ phòng đầu tiên, và bạn đang cần phòng thứ mấy.

Trong CPU:

- *Thanh ghi cơ sở* (Base Register) giữ địa chỉ đầu của vùng dữ liệu, tức “địa chỉ phòng đầu tiên”;
- *Thanh ghi chỉ số* (Index Register) giữ khoảng cách từ chỗ cần tới so với chỗ đầu, tức “cần phòng thứ mấy”.

Địa chỉ thật cần truy nhập, gọi là địa chỉ vật lý, được tính bằng phép cộng đơn giản:

$$\text{địa chỉ vật lý} = \text{địa chỉ cơ sở} + \text{chỉ số}.$$

Nhờ cặp thanh ghi này, một vòng lặp duyệt mảng chỉ cần tăng thanh ghi chỉ số lên đúng kích thước một phần tử sau mỗi lần lặp là xong, không phải tính lại địa chỉ từ đầu.

2.4.3 Ngăn xếp và con trỏ ngăn xếp

Định nghĩa 2.4: Ngăn xếp

Ngăn xếp (stack) là một vùng nhớ hoạt động theo quy tắc LIFO – Last In, First Out: thứ cất vào sau cùng sẽ là thứ lấy ra đầu tiên.

Hình ảnh dễ nhớ nhất là *chồng đĩa trong quán ăn*. Rửa xong đĩa nào thì đặt lên trên cùng. Khi cần lấy, bạn cũng lấy chiếc trên cùng. Không ai rút đĩa ở giữa chồng ra cả.

Đặc điểm của ngăn xếp:

- thường dùng để phục vụ chương trình con: cất địa chỉ trở về, truyền tham số, chứa biến cục bộ;
- *đáy ngăn xếp* là một ô nhớ cố định, không đổi suốt quá trình chạy;
- *đỉnh ngăn xếp* là chỗ trên cùng, thay đổi liên tục mỗi lần cất vào hay lấy ra.

Con trỏ ngăn xếp SP (Stack Pointer) là thanh ghi giữ địa chỉ của đỉnh ngăn xếp – nó luôn

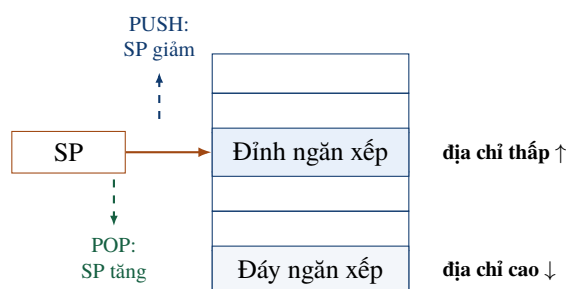
chỉ vào chiếc đĩa trên cùng. Hai thao tác cơ bản:

- **PUSH – cất một thông tin vào ngăn xếp:** nội dung của SP tự động *giảm*, rồi thông tin được ghi vào ô nhớ mà SP đang trỏ tới;
- **POP – lấy một thông tin ra khỏi ngăn xếp:** thông tin được đọc từ ô nhớ mà SP đang trỏ tới, rồi nội dung của SP tự động *tăng*;
- khi ngăn xếp rỗng, SP trở vào đáy.

Chỗ này dễ nhầm, nên đọc kỹ: trên hầu hết các bộ xử lý thông dụng, ngăn xếp *lớn dần về phía địa chỉ thấp*. Nghĩa là PUSH thì SP giảm, POP thì SP tăng – ngược với trực giác.

Vì sao lại làm ngược đời như vậy? Vì trong bộ nhớ còn một vùng nữa cũng lớn dần theo nhu cầu, là vùng dữ liệu động. Đặt ngăn xếp ở một đầu và vùng dữ liệu ở đầu kia, cho hai bên tiến về phía nhau, thì phần trống ở giữa được cả hai dùng chung. Cách nào cũng có lúc hết chỗ, nhưng cách này hết chỗ muộn nhất.

SP tăng giảm bao nhiêu mỗi lần? Bằng kích thước một mục ngăn xếp: 2 byte với x86 chế độ thực, 4 byte với kiến trúc 32 bit, 8 byte với kiến trúc 64 bit.



Hình 2.6. Ngăn xếp và con trỏ ngăn xếp SP

Ngăn xếp phục vụ chương trình con thế nào

Hai lệnh `CALL` (gọi chương trình con) và `RET` (trở về) thực chất chỉ là hai thao tác ngăn xếp đội lốt:

- `CALL` nhận làm hai việc: cất nội dung PC hiện tại vào ngăn xếp (SP giảm), rồi nạp địa chỉ nhận vào PC. Từ nhịp sau, CPU chạy ở chỗ mới.
- `RET` làm ngược lại: lấy giá trị trên đỉnh ngăn xếp nạp vào PC (SP tăng). Thế là CPU quay về đúng chỗ cũ.

Chính nhờ quy tắc LIFO mà chương trình con gọi được lồng nhau nhiều tầng, thậm chí gọi đệ quy (tức tự gọi lại chính nó), mà các địa chỉ trở về vẫn được lấy ra đúng thứ tự ngược lại. Chồng đĩa xếp vào thế nào thì dỡ ra thế ấy.

Ví dụ 2.6: Theo dõi con trỏ ngăn xếp qua các lệnh CALL/RET

Một bộ xử lý 16 bit, mỗi mục ngăn xếp chiếm 2 byte, ngăn xếp lớn dần về phía địa chỉ thấp. Ban đầu $SP = (2000)_{16}$. Chương trình thực hiện lần lượt: `PUSH AX`, `CALL SUB1`, và bên trong `SUB1` lại có `PUSH BX` rồi `CALL SUB2`. Hãy xác định giá trị `SP` sau mỗi lệnh, và giá trị `SP` sau khi `SUB2` thực hiện `RET`.

Giải. Quy tắc rất đơn giản: cất vào thì `SP` giảm 2, lấy ra thì `SP` tăng 2. Lưu ý `CALL` cũng là một lần cất (cất địa chỉ trở về).

Lệnh	Thao tác ngăn xếp	SP sau lệnh
(ban đầu)	–	$(2000)_{16}$
<code>PUSH AX</code>	cất <code>AX</code>	$(1FFE)_{16}$
<code>CALL SUB1</code>	cất địa chỉ trở về	$(1FFC)_{16}$
<code>PUSH BX</code>	cất <code>BX</code>	$(1FFA)_{16}$
<code>CALL SUB2</code>	cất địa chỉ trở về	$(1FF8)_{16}$
<code>RET</code>	lấy địa chỉ trở về nạp vào <code>PC</code>	$(1FFA)_{16}$

Nhận xét. Sau `RET` của `SUB2`, `SP` trở về đúng giá trị nó có ngay trước lệnh `CALL SUB2`. Đó là dấu hiệu mọi thứ đang đúng.

Nhưng thử tưởng tượng bên trong `SUB2` có một lệnh `PUSH` mà lập trình viên quên `POP` lại. Khi gặp `RET`, đỉnh ngăn xếp không còn là địa chỉ trở về nữa mà là dữ liệu đã push nhầm. CPU sẽ nạp dữ liệu đó vào `PC` và nhảy tới một chỗ vô nghĩa – chương trình chạy loạn ngay lập tức.

Lỗi này gọi là *mất cân bằng ngăn xếp*, và là một trong những lỗi khó tìm nhất khi lập trình hợp ngữ. Quy tắc vàng: trong một chương trình con, số lệnh `PUSH` phải bằng đúng số lệnh `POP` trước khi `RET`.

2.4.4 Thanh ghi dữ liệu và thanh ghi đa năng

Thanh ghi dữ liệu giữ dữ liệu tạm và kết quả trung gian. CPU cần có nhiều thanh ghi dữ liệu, vì mỗi giá trị giữ được trong thanh ghi là một lần không phải chạy ra bộ nhớ. Các thanh ghi dữ liệu thường rộng 8, 16, 32 hoặc 64 bit tùy kiến trúc.

Thanh ghi đa năng (general purpose register) là thanh ghi dùng được cho cả địa chỉ lẫn dữ liệu, tùy lệnh nào đang dùng nó.

Xu hướng thiết kế hiện đại là tăng số thanh ghi đa năng và giảm số thanh ghi chuyên dụng. Lý do: thanh ghi chuyên dụng chỉ dùng được vào một việc, nên hay ngồi không; thanh ghi đa năng thì trình biên dịch phân bổ linh hoạt hơn nhiều.

2.4.5 Thanh ghi lệnh

Thanh ghi lệnh `IR` (Instruction Register) giữ mã của lệnh đang được thực hiện. Nó được nạp ở cuối công đoạn nhận lệnh và giữ nguyên giá trị suốt các công đoạn còn lại, để CPU lúc nào

cũng có mã lệnh mà tra.

IR thuộc loại thanh ghi không lập trình được: không lệnh nào đọc hay ghi trực tiếp vào nó được.

2.4.6 Thanh ghi trạng thái

Thanh ghi trạng thái, còn gọi là thanh ghi cờ (Flag Register), là nơi chứa các cờ mà ta đã gặp ở mục 2.2.7 cùng vài bit trạng thái khác. Nội dung của nó chia làm hai nhóm.

Nhóm 1 – các cờ phép toán, báo kết quả phép toán vừa rồi ra sao: cờ Zero (ZF), cờ Sign (SF), cờ Carry (CF) và cờ Overflow (OF). Xem lại bảng 2.2.

Nhóm 2 – các cờ điều khiển, không nói về kết quả mà nói về *trạng thái làm việc* của chính CPU:

- **Cờ Interrupt (IF)** – cờ cho phép ngắt. Nếu $IF = 1$ thì CPU chấp nhận tín hiệu ngắt từ bên ngoài. Nếu $IF = 0$ thì CPU bỏ ngoài tai. Cờ này được tắt tạm khi CPU đang làm việc gì đó không được phép gián đoạn.
- **Cờ Direction (DF)** – cờ hướng, quy định thanh ghi chỉ số tăng hay giảm trong các lệnh xử lý chuỗi ký tự.
- **Cờ Trap (TF)** – cờ bẫy. Khi bật, CPU sinh một ngắt sau *mỗi* lệnh. Đây chính là cơ chế cho phép trình gỡ rối chạy chương trình từng bước một.

Định nghĩa 2.5: Từ trạng thái chương trình

Từ trạng thái chương trình (PSW – Program Status Word) là tập hợp thanh ghi cờ cùng các thông tin trạng thái khác mà CPU phải cất đi và khôi phục lại mỗi khi chuyển sang làm việc khác, chẳng hạn khi vào và ra khỏi chương trình con phục vụ ngắt.

Vì sao phải cất PSW? Hãy hình dung tình huống này. Chương trình chính vừa làm một phép trừ, kết quả bằng 0 nên $ZF = 1$. Lệnh kế tiếp là “nếu $ZF = 1$ thì nhảy”. Nhưng ngay giữa hai lệnh đó, một ngắt xảy ra. Chương trình xử lý ngắt cũng làm vài phép toán, và các phép toán đó *ghi đè* lên cờ ZF.

Khi quay về, lệnh nhảy sẽ nhìn vào cờ ZF của chương trình ngắt chứ không phải của chính nó – và quyết định sai.

Vì vậy PSW luôn nằm trong danh sách những thứ phải cất đầu tiên khi vào ngắt và khôi phục cuối cùng khi ra.

2.4.7 Tập thanh ghi của một số bộ xử lý thật

Đến đây bạn đã biết các loại thanh ghi trên lý thuyết. Bảng 2.4 cho thấy chúng xuất hiện thế nào trên ba kiến trúc thật mà bạn sẽ gặp trong nghề.

Bảng 2.4. Tập thanh ghi của ba kiến trúc tiêu biểu

Tiêu chí	x86-64	MIPS 32 bit	ARM Cortex-M
Số thanh ghi đa năng	16	32	13
Độ rộng	64 bit	32 bit	32 bit
Bộ đếm chương trình	RIP	PC	R15 (PC)
Con trỏ ngăn xếp	RSP	\$sp (\$29)	R13 (SP)
Thanh ghi cờ	RFLAGS	không có	APSR
Đặc điểm riêng	Nhiều thanh ghi chuyên dụng; tương thích ngược tới tận 8086	\$zero luôn bằng 0; so sánh bằng lệnh riêng	R14 là thanh ghi liên kết, chứa địa chỉ trở về

Cột giữa có một điều lạ đáng dừng lại: **MIPS không có thanh ghi cờ**.

Vậy nó so sánh kiểu gì? Thay vì “trừ hai số rồi xem cờ”, MIPS dùng lệnh so sánh trực tiếp như `beq` (nhảy nếu bằng) hay `slt` (đặt bằng 1 nếu nhỏ hơn). Mọi thông tin mà lệnh cần đều nằm ngay trong bản thân lệnh đó.

Lợi ích: hai lệnh liên tiếp không còn phụ thuộc ngằm vào nhau qua thanh ghi cờ. Mà đã không phụ thuộc thì mới xếp chúng chạy chồng lên nhau được – đó chính là điều kiện để làm ống lệnh (xem Phụ lục A) và chạy song song. Đây là một ví dụ rất cụ thể cho triết lý thiết kế RISC: bỏ bớt tiện nghi để đổi lấy tốc độ.

Ghi nhớ

Thanh ghi là bộ nhớ nhỏ, nhanh nhất, nằm trong CPU. Năm nhóm: địa chỉ (PC, DP, cơ sở, chỉ số, SP), dữ liệu, đa năng, lệnh (IR), và trạng thái (các cờ). Ngăn xếp là vùng nhớ LIFO do SP trỏ vào: PUSH thì SP giảm, POP thì SP tăng. Cặp `CALL/RET` chính là PUSH/POP địa chỉ trở về.

2.5 Hệ thống bus trong CPU

Cho tới giờ ta đã nói “đưa địa chỉ ra bus”, “nhận dữ liệu qua bus” khá nhiều lần mà chưa định nghĩa bus là gì. Bây giờ ta làm rõ.

Định nghĩa 2.6: Bus

Bus là một bó dây dẫn dùng chung, nối nhiều thành phần của máy tính lại với nhau để chúng trao đổi thông tin.

Hình dung một hành lang chung mà mọi phòng đều mở cửa ra đó. Ai muốn gửi gì cho ai

thì đặt ra hành lang, kèm theo địa chỉ người nhận. Ưu điểm là tiết kiệm: không phải kéo dây riêng giữa mọi cặp phòng. Nhược điểm là tại một thời điểm chỉ một người được nói, ai khác phải chờ.

Xét theo loại thông tin mang trên đó, bus chia làm ba loại.

2.5.1 Bus dữ liệu

Bus dữ liệu (data bus) chở dữ liệu và lệnh giữa CPU với bộ nhớ và với các mô-đun vào–ra. Đây là bus *hai chiều*: dữ liệu chạy được cả hai hướng.

Số đường dây của bus dữ liệu gọi là *độ rộng*, thường là 8, 16, 32 hoặc 64. Độ rộng cho biết CPU chuyển được bao nhiêu bit trong một lần trao đổi.

Nó ảnh hưởng trực tiếp tới tốc độ: bus 64 bit chở được 8 byte mỗi lần, gấp bốn lần bus 16 bit ở cùng tần số. Giống như so một con đường bốn làn với một con đường một làn.

2.5.2 Bus địa chỉ

Bus địa chỉ (address bus) chở địa chỉ, tức là “chỗ cần đọc” hoặc “chỗ cần ghi”. Đây là bus *một chiều*: chỉ CPU phát ra, bộ nhớ chỉ nghe. (Ngoại lệ duy nhất là khi bộ điều khiển DMA tạm nắm quyền làm chủ bus, sẽ học ở chương 6.)

Độ rộng bus địa chỉ quyết định máy tính *nhớ được bao nhiêu*. Với n đường địa chỉ, CPU tạo ra được 2^n tổ hợp khác nhau, tức đánh địa chỉ được 2^n ô nhớ.

Ví dụ 2.7: Độ rộng bus địa chỉ và không gian nhớ

Bộ vi xử lý 8088 dùng 16 đường địa chỉ để đánh địa chỉ cổng vào–ra. Hãy xác định không gian địa chỉ vào–ra của nó. So sánh với trường hợp bus địa chỉ 20 đường và 32 đường.

Giải. Công thức chỉ có một: n đường thì 2^n vị trí.

Với 16 đường: $2^{16} = 65\,536$ vị trí, tức 64 K vị trí.

Với 20 đường: $2^{20} = 1\,048\,576$ vị trí, tức 1 MB. Đây chính là toàn bộ không gian nhớ của 8086/8088 – con số huyền thoại “640 KB” của máy tính đời đầu bắt nguồn từ đây.

Với 32 đường: $2^{32} = 4\,294\,967\,296$ vị trí, tức 4 GB (xem thêm ví dụ 1.1 ở chương 1).

Con số 4 GB giải thích một chuyện bạn có thể đã nghe: máy chạy hệ điều hành 32 bit thì có cắm thêm RAM cũng gần như vô ích, vì địa chỉ không đủ để đánh số quá 4 GB (có vài cơ chế mở rộng đặc biệt, nhưng đó là ngoại lệ). Đó là một trong những lý do cả ngành chuyển sang kiến trúc 64 bit.

Một điểm nữa đáng lưu ý: bộ xử lý có *hai* không gian địa chỉ tách biệt, một cho bộ nhớ và một cho cổng vào–ra. Với 8088, không gian nhớ dùng 20 đường còn không gian vào–ra chỉ dùng 16 đường thấp, nên hai không gian có kích thước khác hẳn nhau. Cùng một địa chỉ trên bus, việc nó chỉ vào bộ nhớ hay chỉ vào cổng vào–ra là do *tín hiệu điều khiển* MEMR/MEMW hay IOR/IOW quyết định.

2.5.3 Bus điều khiển

Bus điều khiển (control bus) chở các tín hiệu điều khiển và tín hiệu định thời.

Bus này khác hai bus trên ở một điểm quan trọng: nó *không phải* một con số nhiều bit. Nó là một tập hợp các đường dây riêng lẻ, mỗi đường có một ý nghĩa riêng – giống như một bảng công tắc, mỗi công tắc một chức năng.

Ba nhóm tín hiệu chính:

- tín hiệu đọc/ghi bộ nhớ và đọc/ghi cổng vào–ra;
- tín hiệu điều khiển ngắt;
- tín hiệu điều khiển bus, ví dụ “xin nhường bus” và “đồng ý nhường bus”.

Hai tín hiệu gặp nhiều nhất trong môn học này:

- **MEMR** (Memory Read): ra lệnh cho bộ nhớ đọc dữ liệu ở địa chỉ đang có trên bus địa chỉ và đưa lên bus dữ liệu;
- **MEMW** (Memory Write): ra lệnh cho bộ nhớ lấy dữ liệu đang có trên bus dữ liệu ghi vào địa chỉ đang có trên bus địa chỉ.

Tương ứng với cổng vào–ra là cặp tín hiệu **IOR** và **IOW**.

2.5.4 Đường dữ liệu bên trong CPU

Bên trong CPU cũng có bus, gọi là bus trong, nối các thanh ghi với ALU. Cách tổ chức bus trong nghe có vẻ là chi tiết vụn vặt, nhưng nó quyết định một điều rất lớn: *một lệnh tốn bao nhiêu nhịp*.

Lý do đơn giản. Một sợi bus tại một thời điểm chỉ chở được một giá trị. Mà phép cộng cần *ba* giá trị đi qua: toán hạng thứ nhất vào, toán hạng thứ hai vào, kết quả ra. Nếu chỉ có một bus thì phải làm ba nhịp.

Bảng 2.5. So sánh ba phương án tổ chức bus trong CPU

Phương án	Cách hoạt động	Ưu điểm	Nhược điểm
Một bus	Mỗi nhịp chỉ chuyển được một giá trị; toán hạng thứ hai phải giữ tạm ở thanh ghi đệm	Mạch đơn giản, ít dây, tốn ít diện tích	Một lệnh cần nhiều nhịp
Hai bus	Hai toán hạng vào ALU đi trên hai bus khác nhau	Bớt được một nhịp so với phương án một bus	Nhiều dây hơn, điều khiển phức tạp hơn
Ba bus	Hai bus cho hai toán hạng vào, một bus cho kết quả ra	Một phép toán ALU xong trong đúng một nhịp	Tốn dây và diện tích nhất

Ví dụ 2.8: Thực hiện lệnh cộng trên đường dữ liệu một bus

Cho CPU tổ chức theo phương án một bus. ALU có một thanh ghi đệm Y ở đầu vào và một thanh ghi đệm Z ở đầu ra. Hãy viết dãy vi thao tác thực hiện lệnh $\text{ADD } R1, R2$ (nghĩa là $R1 \leftarrow R1 + R2$) và cho biết lệnh chiếm bao nhiêu nhịp, chưa kể công đoạn nhận lệnh.
Giải. Vì mỗi nhịp bus chỉ chở được một giá trị nên phải tách làm ba nhịp:

$$\begin{aligned} t_1 : & \quad Y \leftarrow R2 && \text{(đưa toán hạng thứ hai vào đệm)} \\ t_2 : & \quad Z \leftarrow R1 + Y && \text{(ALU cộng, kết quả giữ ở đệm ra)} \\ t_3 : & \quad R1 \leftarrow Z && \text{(ghi kết quả trở lại thanh ghi đích)} \end{aligned}$$

Vậy công đoạn thực hiện chiếm 3 nhịp.

So sánh: nếu tổ chức theo phương án ba bus, cả ba việc trên gộp lại thành một nhịp duy nhất $R1 \leftarrow R1 + R2$ – tiết kiệm 2 nhịp cho *mỗi* lệnh số học. Với một chương trình có hàng tỉ lệnh, khoản tiết kiệm đó là khổng lồ. Đó là lý do bộ xử lý hiệu năng cao chấp nhận tổn diện tích để làm nhiều bus trong.

Ghi nhớ

Ba loại bus: bus dữ liệu (hai chiều, độ rộng quyết định tốc độ), bus địa chỉ (một chiều, n đường thì nhớ được 2^n ô), bus điều khiển (tập các đường tín hiệu riêng lẻ như MEMR, MEMW, IOR, IOW). Bên trong CPU, càng nhiều bus thì lệnh càng tốn ít nhịp.

2.6 Hiệu năng CPU

Năm mục vừa rồi trả lời câu hỏi “CPU làm việc thế nào”. Mục cuối này trả lời một câu hỏi khác hẳn: *làm sao đo được CPU nhanh hay chậm*.

Đây là phần duy nhất trong chương phải tính toán nhiều, và cũng là phần chiếm gần hết số bài tập. Bù lại, toàn bộ mục này chỉ xoay quanh *một* công thức, các công thức khác đều suy ra từ đó. Hãy đọc chậm và làm lại mọi ví dụ.

2.6.1 Hiệu năng nghĩa là gì

Định nghĩa 2.7: Hiệu năng

Hiệu năng P (Performance) của một máy tính đối với một chương trình cho trước là nghịch đảo của thời gian thực hiện chương trình đó:

$$P = \frac{1}{t}.$$

Định nghĩa này chỉ nói lại điều ai cũng thấy: chạy càng nhanh (thời gian càng nhỏ) thì hiệu năng càng cao. Đưa nó về công thức để so sánh được bằng số.

Muốn so hai máy, ta lập tỉ số. Nói “máy A nhanh hơn máy B k lần” nghĩa là:

$$\frac{P_A}{P_B} = \frac{1/t_A}{1/t_B} = \frac{t_B}{t_A} = k.$$

Để ý kỹ chỗ này: tỉ số hiệu năng bằng tỉ số thời gian *đảo ngược*. Máy A nhanh hơn thì t_A nhỏ hơn, nên t_B phải nằm ở tử số. Đây là chỗ hay nhầm nhất.

Ví dụ 2.9: So sánh hai máy theo thời gian chạy

Một chương trình chạy hết 10 giây trên máy A và 15 giây trên máy B. Máy nào nhanh hơn và nhanh hơn bao nhiêu lần?

Giải. Máy A chạy hết ít thời gian hơn nên nhanh hơn. Bao nhiêu lần thì lập tỉ số, nhớ đảo ngược:

$$\frac{P_A}{P_B} = \frac{t_B}{t_A} = \frac{15 \text{ s}}{10 \text{ s}} = 1,5.$$

Vậy máy A nhanh hơn máy B 1,5 lần.

Hiệu năng luôn gắn với *một chương trình cụ thể*. Cùng hai máy A và B, đổi sang chương trình khác thì tỉ số có thể đảo chiều – ta sẽ thấy điều đó xảy ra thật ở bài tập cuối chương. Vì thế không thể nói máy nào “nhanh hơn” một cách tuyệt đối. Mọi phép đo hiệu năng nghiêm túc đều phải công bố kèm bộ chương trình đã dùng để đo. Khi đọc quảng cáo “CPU này nhanh hơn 30%”, câu hỏi đầu tiên phải là: nhanh hơn khi chạy cái gì?

2.6.2 Tốc độ xung nhịp CPU

CPU không làm việc tùy tiện lúc nào nó muốn. Nó làm theo nhịp, giống như đội chèo thuyền chỉ khua mái theo tiếng trống. Tiếng trống ấy gọi là **xung nhịp** (clock), do một mạch tạo dao động phát ra đều đặn.

Hai đại lượng cần phân biệt:

- **Chu kỳ xung nhịp** T_0 (clock period): thời gian của *một* nhịp. Đơn vị là giây, thực tế thường là ns hoặc ps.
- **Tần số xung nhịp** f_0 (clock rate): số nhịp trong *một* giây. Đơn vị là Hz, thực tế thường là GHz.

Hai đại lượng này là nghịch đảo của nhau:

$$f_0 = \frac{1}{T_0}, \quad \text{hay} \quad T_0 = \frac{1}{f_0}.$$

Ví dụ 2.10: Tính chu kỳ xung nhịp

Bộ xử lý có $f_0 = 4 \text{ GHz}$. Tính chu kỳ xung nhịp.

Giải. Trước hết đổi GHz sang Hz: $4 \text{ GHz} = 4 \times 10^9 \text{ Hz}$.

Sau đó lấy nghịch đảo:

$$T_0 = \frac{1}{4 \times 10^9} = 0,25 \times 10^{-9} \text{ s}.$$

Cuối cùng đổi sang đơn vị dễ đọc. Vì $10^{-9} \text{ s} = 1 \text{ ns}$ nên $T_0 = 0,25 \text{ ns}$.

Nói cách khác, mỗi nhịp làm việc của bộ xử lý này kéo dài một phần tư phần tỉ giây.

2.6.3 Thời gian chạy tính theo số chu kỳ

Bây giờ tới công thức đầu tiên thật sự có ích.

Một chương trình cần bao nhiêu nhịp thì gọi số nhịp đó là n . Mỗi nhịp dài T_0 . Vậy tổng thời gian là:

$$t_{CPU} = n \times T_0 = \frac{n}{f_0}. \quad (2.1)$$

Công thức này giống hệt bài toán “đi bộ”: số bước nhân thời gian mỗi bước bằng tổng thời gian.

Từ đó suy ra ngay hai cách – và chỉ hai cách – để làm CPU chạy nhanh hơn:

- **giảm số nhịp** n : làm cùng công việc mà cần ít nhịp hơn;
- **tăng tần số** f_0 : mỗi nhịp ngắn lại.

Ví dụ 2.11: Xác định tốc độ xung nhịp cần thiết

Hai máy tính A và B cùng chạy một chương trình. Máy A có tốc độ xung nhịp $f_A = 2 \text{ GHz}$ và chạy hết $t_A = 10 \text{ s}$. Máy B chạy hết $t_B = 6 \text{ s}$, nhưng số nhịp n_B mà nó cần nhiều hơn n_A 1,2 lần. Hãy xác định tốc độ xung nhịp cần thiết cho máy B.

Giải. Bài này có ba bước, làm lần lượt là ra.

Bước 1 – tìm số nhịp của máy A. Từ (2.1) ta có $n = t \times f$:

$$n_A = t_A \times f_A = 10 \text{ s} \times 2 \times 10^9 \text{ Hz} = 20 \times 10^9 \text{ nhịp}.$$

Bước 2 – tìm số nhịp của máy B. Đề cho n_B gấp 1,2 lần n_A :

$$n_B = 1,2 \times 20 \times 10^9 = 24 \times 10^9 \text{ nhịp}.$$

Bước 3 – tìm tần số của máy B. Máy B phải chạy hết ngàn ấy nhịp trong 6 s:

$$f_B = \frac{n_B}{t_B} = \frac{24 \times 10^9}{6} = 4 \times 10^9 \text{ Hz} = 4 \text{ GHz}.$$

Nhận xét. Máy B phải chạy ở tần số gấp đôi máy A, mà chỉ nhanh hơn được $10/6 \approx 1,67$ lần. Phần chênh lệch bị “ăn” bởi việc kiến trúc của máy B tốn nhiều nhịp hơn cho cùng một công việc. Tần số cao chưa chắc đã là máy tốt.

2.6.4 Thời gian chạy tính theo số lệnh

Công thức (2.1) có một chỗ bất tiện: ta hiếm khi biết trực tiếp số nhịp n . Cái ta biết là chương trình có bao nhiêu *lệnh*.

May thay, hai thứ đó liên hệ với nhau rất đơn giản:

$$n = IC \times CPI, \quad (2.2)$$

trong đó:

- IC (Instruction Count) là *số lệnh* của chương trình;
- CPI (Cycles Per Instruction) là *số nhịp trung bình cho mỗi lệnh*.

Ví dụ chương trình có 100 lệnh, mỗi lệnh trung bình tốn 2 nhịp, thì tổng cộng 200 nhịp. Chỉ có vậy.

Thay (2.2) vào (2.1) ta được công thức trung tâm của cả mục này:

$$t_{CPU} = IC \times CPI \times T_0 = \frac{IC \times CPI}{f_0} \quad (2.3)$$

Hãy đọc công thức (2.3) thành lời: *thời gian chạy bằng số lệnh, nhân số nhịp mỗi lệnh, nhân thời gian mỗi nhịp*. Nếu nhớ được câu này thì bạn không cần học thuộc công thức.

Điều đẹp nhất của công thức này là nó tách thời gian chạy thành ba thừa số độc lập nhau, mỗi thừa số thuộc về một tầng khác nhau của hệ thống:

- IC phụ thuộc vào thuật giải, ngôn ngữ lập trình, trình biên dịch và kiến trúc tập lệnh;
- CPI phụ thuộc vào kiến trúc tập lệnh và cách tổ chức phần cứng;
- T_0 phụ thuộc vào công nghệ chế tạo chip và cách tổ chức phần cứng.

Nhờ vậy, khi muốn tăng tốc một chương trình, bạn biết ngay có ba hướng để tấn công.

Ví dụ 2.12: So sánh hai máy cùng kiến trúc tập lệnh

Hai máy tính A và B có cùng kiến trúc tập lệnh. Máy A có chu kỳ xung nhịp $T_A = 250$ ps và $CPI_A = 2,0$. Máy B có $T_B = 500$ ps và $CPI_B = 1,2$. Máy nào nhanh hơn?

Giải. Đề không cho số lệnh IC . Không sao: hai máy cùng kiến trúc tập lệnh, nên cùng một chương trình sẽ dịch ra cùng số lệnh:

$$IC_A = IC_B = IC.$$

Ta cứ để IC nguyên như một chữ, cuối cùng nó sẽ tự triệt tiêu.

Áp dụng (2.3) cho từng máy:

$$t_A = IC \times CPI_A \times T_A = IC \times 2,0 \times 250 \text{ ps} = IC \times 500 \text{ ps},$$

$$t_B = IC \times CPI_B \times T_B = IC \times 1,2 \times 500 \text{ ps} = IC \times 600 \text{ ps}.$$

Lập tỉ số, IC triệt tiêu:

$$\frac{P_A}{P_B} = \frac{t_B}{t_A} = \frac{IC \times 600}{IC \times 500} = \frac{600}{500} = 1,2.$$

Kết luận: máy A nhanh hơn máy B 1,2 lần.

Điều đáng chú ý: máy A có CPI lớn hơn (2,0 so với 1,2) mà vẫn thắng, vì chu kỳ xung nhịp của nó ngắn hơn một nửa. Không được nhìn một thừa số mà kết luận.

Khi các lệnh không giống nhau: CPI trung bình

Từ đây trở đi sẽ gặp nhiều lần cụm *kiến trúc tập lệnh*. Nó chỉ có nghĩa là: bộ các lệnh máy mà một dòng CPU hiểu được, cùng khuôn dạng của chúng. Hai máy cùng kiến trúc tập lệnh thì chạy được cùng một file chương trình, và cùng một chương trình nguồn dịch ra sẽ cho cùng số lệnh trên cả hai máy. Đó chính là điều kiện cho phép ta triệt tiêu IC khi so sánh, như ở ví dụ ngay dưới đây.

Cho tới giờ ta coi mọi lệnh đều tốn như nhau. Thực tế không phải vậy: lệnh cộng hai thanh ghi tốn một nhịp, lệnh đọc bộ nhớ tốn vài nhịp, lệnh chia tốn hàng chục nhịp.

Khi đó CPI trong công thức phải hiểu là *CPI trung bình*, và ta phải tính nó ra.

Cách tính giống hệt cách tính điểm trung bình có hệ số. Giả sử chương trình có K loại lệnh; loại thứ i xuất hiện IC_i lần và mỗi lần tốn CPI_i nhịp. Tổng số nhịp là:

$$n = \sum_{i=1}^K (CPI_i \times IC_i), \quad (2.4)$$

và CPI trung bình là tổng số nhịp chia tổng số lệnh:

$$CPI_{TB} = \frac{n}{IC} = \frac{1}{IC} \sum_{i=1}^K (CPI_i \times IC_i). \quad (2.5)$$

Ký hiệu $\sum$ (đọc là “xích ma”) chỉ có nghĩa là “cộng tất cả lại”. Dòng $\sum_{i=1}^K (CPI_i \times IC_i)$ đọc là: với mỗi loại lệnh, lấy số nhịp nhân số lần xuất hiện, rồi cộng hết các kết quả đó lại. Trong bài tập, số loại lệnh thường chỉ là 3 hay 4 nên viết ra hết cũng được, không cần dùng ký hiệu này.

Ví dụ 2.13: Tính CPI trung bình của hai dãy lệnh

Cho bảng chỉ ra hai dãy lệnh sử dụng các lệnh thuộc ba loại A, B, C. Hãy tính CPI trung bình của mỗi dãy.

Loại lệnh	A	B	C
CPI theo loại lệnh	1	2	3
IC trong dãy lệnh 1	20	10	20
IC trong dãy lệnh 2	40	10	10

Giải. Với mỗi dãy, làm ba bước: đếm tổng số lệnh, tính tổng số nhịp, rồi chia.

Dãy lệnh 1.

- Tổng số lệnh: $IC = 20 + 10 + 20 = 50$.
- Tổng số nhịp: $n = \underbrace{1 \times 20}_{\text{loại A}} + \underbrace{2 \times 10}_{\text{loại B}} + \underbrace{3 \times 20}_{\text{loại C}} = 20 + 20 + 60 = 100$.
- CPI trung bình: $CPI_{TB} = 100/50 = 2,0$.

Dãy lệnh 2.

- Tổng số lệnh: $IC = 40 + 10 + 10 = 60$.
- Tổng số nhịp: $n = 1 \times 40 + 2 \times 10 + 3 \times 10 = 40 + 20 + 30 = 90$.
- CPI trung bình: $CPI_{TB} = 90/60 = 1,5$.

Nhận xét quan trọng. Dãy lệnh 2 có nhiều lệnh hơn (60 so với 50) nhưng lại cần ít nhịp hơn (90 so với 100), nên chạy nhanh hơn trên cùng một máy.

Ví dụ này cho thấy: **số lệnh không phải là thước đo hiệu năng**. Một chương trình dài chưa chắc đã chậm hơn chương trình ngắn.

2.6.5 MIPS – và vì sao phải cẩn thận với nó

Ngoài thời gian chạy, người ta còn hay dùng một con số khác: **MIPS** (Millions of Instructions Per Second – số triệu lệnh thực hiện trong một giây).

Cách xây dựng nó rất tự nhiên: lấy số lệnh chia thời gian, rồi chia tiếp cho 10^6 để ra đơn vị “triệu”:

$$MIPS = \frac{IC}{t_{CPU} \times 10^6}. \quad (2.6)$$

Thay t_{CPU} từ công thức (2.3) vào rồi rút gọn (IC triệt tiêu):

$$MIPS = \frac{IC}{\frac{IC \times CPI}{f_0} \times 10^6} = \frac{f_0}{CPI \times 10^6}. \quad (2.7)$$

Kết quả gọn bất ngờ: MIPS chỉ phụ thuộc tần số và CPI, không phụ thuộc chương trình dài hay ngắn. Từ đó suy ra công thức ngược, dùng khi đề cho MIPS và hỏi CPI:

$$CPI = \frac{f_0}{MIPS \times 10^6}. \quad (2.8)$$

Ví dụ 2.14: Tính MIPS và tính CPI

a) Tính MIPS của bộ xử lý có $f_0 = 2 \text{ GHz}$ và $CPI = 4$.

b) Tính CPI của bộ xử lý có $f_0 = 1 \text{ GHz}$ và đạt 400 MIPS.

Giải. Ta làm theo lối suy luận từng bước, rồi kiểm lại bằng công thức.

Câu a).

- Chu kỳ xung nhịp: $T_0 = 1/(2 \times 10^9) = 0,5 \text{ ns}$.
- Mỗi lệnh tốn 4 nhịp, nên một lệnh mất $4 \times 0,5 = 2 \text{ ns}$.
- Trong 1 giây (bằng 10^9 ns) làm được: $10^9 \div 2 = 5 \times 10^8$ lệnh.
- Đổi ra triệu: $5 \times 10^8 \div 10^6 = 500$.

Vậy bộ xử lý đạt **500 MIPS**. Kiểm lại bằng (2.7): $MIPS = 2 \times 10^9 / (4 \times 10^6) = 500$. Khớp.

Câu b).

- Chu kỳ xung nhịp: $T_0 = 1/10^9 = 1 \text{ ns}$.
- 400 MIPS nghĩa là $400 \times 10^6 = 4 \times 10^8$ lệnh mỗi giây.
- Thời gian một lệnh: $1 \div (4 \times 10^8) = 2,5 \times 10^{-9} \text{ s} = 2,5 \text{ ns}$.
- Mỗi nhịp dài 1 ns, nên một lệnh chiếm $2,5 \div 1 = 2,5$ nhịp.

Vậy $CPI = 2,5$. Kiểm lại bằng (2.8): $CPI = 10^9 / (400 \times 10^6) = 2,5$. Khớp.

MIPS nghe rất kêu nhưng lại là thước đo dễ đánh lừa, vì nó *không hề tính đến việc một lệnh làm được bao nhiêu việc*.

Hình dung hai người thợ xây. Người thứ nhất khiêng từng viên gạch, một phút được 60 viên. Người thứ hai khiêng cả xô 10 viên, một phút được 20 xô. Đếm “số lần khiêng” thì

người thứ nhất thắng đậm (60 so với 20), nhưng người thứ hai mới là người xây xong nhà trước.

Với CPU cũng vậy: một máy CISC cần 100 lệnh cho công việc mà máy RISC cần 150 lệnh. Máy RISC có thể đạt MIPS cao hơn trong khi thực tế chạy chậm hơn.

Kết luận: MIPS chỉ có ý nghĩa khi so sánh hai máy *cùng kiến trúc tập lệnh* và chạy *cùng một chương trình*. Ngoài trường hợp đó, hãy quay về so sánh thời gian chạy thật.

2.6.6 Những yếu tố ảnh hưởng đến hiệu năng

Tổng kết lại, hiệu năng của một hệ thống phụ thuộc năm yếu tố, xếp từ mức trừu tượng cao xuống mức thấp:

- (1) **Thuật giải** – quyết định phải làm bao nhiêu phép, tức ảnh hưởng tới IC . Đây là yếu tố có biên độ lớn nhất, và cũng là thứ nằm trong tầm tay bạn nhiều nhất. Đổi từ một thuật toán chậm sang một thuật toán nhanh hơn về mặt độ phức tạp (chẳng hạn từ $O(n^2)$ sang $O(n \log n)$ – ký hiệu cho biết khối lượng tính toán tăng thế nào khi dữ liệu lớn dần) có thể rút thời gian xuống hàng nghìn lần – điều mà không một cải tiến phần cứng nào làm nổi.
- (2) **Ngôn ngữ lập trình** – ảnh hưởng tới IC và CPI .
- (3) **Chương trình dịch** – quyết định dãy lệnh máy được sinh ra, ảnh hưởng tới IC và CPI .
- (4) **Kiến trúc tập lệnh** – ảnh hưởng tới cả ba thừa số IC , CPI , T_0 .
- (5) **Phần cứng** – công nghệ chế tạo và cách tổ chức, ảnh hưởng tới CPI và T_0 .

Bảng 2.6. Yếu tố ảnh hưởng tới từng thừa số trong công thức hiệu năng

Yếu tố	IC	CPI	T_0
Thuật giải	có	có	không
Ngôn ngữ lập trình	có	có	không
Chương trình dịch	có	có	không
Kiến trúc tập lệnh	có	có	có
Tổ chức phần cứng	không	có	có
Công nghệ chế tạo	không	không	có

Ghi nhớ

Chỉ cần nhớ một công thức: $t_{CPU} = IC \times CPI \times T_0 = \frac{IC \times CPI}{f_0}$. Mọi thứ khác suy ra từ đó: $T_0 = 1/f_0$; CPI trung bình bằng tổng số nhịp chia tổng số lệnh; $MIPS = f_0/(CPI \times 10^6)$. So sánh hai máy thì lập tỉ số thời gian và *nhớ đảo ngược*. MIPS chỉ dùng được khi hai máy cùng kiến trúc tập lệnh.

Phụ lục A: Kỹ thuật ống dẫn lệnh

Suốt chương này ta hình dung CPU làm việc tuần tự: lệnh trước xong hết sáu công đoạn thì lệnh sau mới bắt đầu. Cách hiểu đó đúng, nhưng nó bỏ phí rất nhiều.

Hãy nghĩ tới việc giặt quần áo. Bạn có ba mẻ đồ, mỗi mẻ phải qua ba công đoạn: máy giặt, máy sấy, rồi gấp. Nếu làm tuần tự – giặt xong mẻ 1, sấy mẻ 1, gấp mẻ 1, rồi mới bắt đầu mẻ 2 – thì trong lúc bạn ngồi gấp mẻ 1, cả máy giặt lẫn máy sấy đều đứng không.

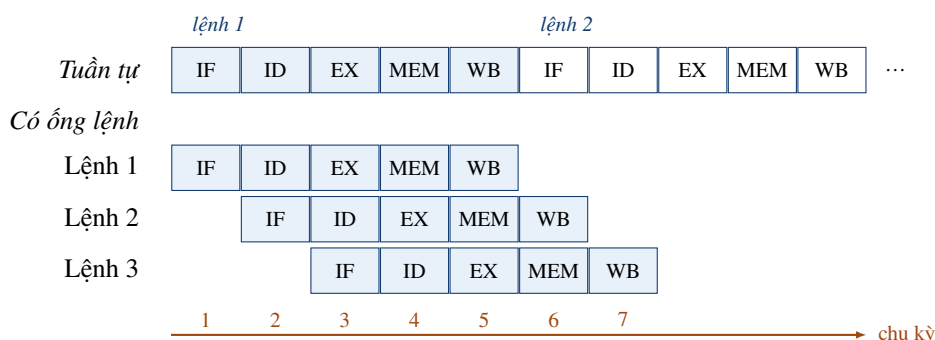
Cách làm khôn hơn: mẻ 1 vừa sang máy sấy thì tổng ngay mẻ 2 vào máy giặt. Ba máy chạy cùng lúc, mỗi máy làm một mẻ khác nhau.

CPU cũng làm đúng như vậy, và kỹ thuật đó gọi là ống dẫn lệnh.

Định nghĩa 2.8: Ống dẫn lệnh

Ống dẫn lệnh (pipeline) là kỹ thuật chia chu trình lệnh thành nhiều tầng độc lập và cho các lệnh liên tiếp nhau cùng chạy trong ống, mỗi lệnh ở một tầng khác nhau, giống như dây chuyền lắp ráp trong nhà máy.

Mô hình MIPS kinh điển chia chu trình lệnh thành năm tầng: IF (nhận lệnh), ID (giải mã và đọc thanh ghi), EX (thực hiện ở ALU), MEM (truy nhập bộ nhớ) và WB (ghi kết quả trở lại thanh ghi). Hình 2.7 so sánh cách thực hiện tuần tự với cách thực hiện có ống dẫn lệnh.



Hình 2.7. Thực hiện tuần tự và thực hiện có ống dẫn lệnh năm tầng

Có một điều dễ hiểu nhầm ở đây, nên nói rõ: **ống lệnh không làm một lệnh riêng lẻ chạy nhanh hơn**. Một lệnh vẫn phải đi qua đủ năm tầng, mất đúng năm nhịp như trước. Mẻ quần áo của bạn vẫn mất từng ấy thời gian để giặt, sấy và gấp.

Cái nó cải thiện là *thông lượng* – số việc hoàn tất trong một đơn vị thời gian. Khi ống đã đầy, cứ mỗi nhịp lại có *một* lệnh xong. Nhìn từ bên ngoài, CPU như thể làm xong một lệnh mỗi nhịp.

Về lý thuyết, ống k tầng cho tốc độ gấp k lần so với làm tuần tự. Thực tế không bao giờ đạt được con số đó, vì ba loại trục trặc:

- **Xung đột cấu trúc:** hai lệnh trong ống cùng cần một tài nguyên phần cứng tại cùng một chu kỳ.

- **Xung đột dữ liệu:** lệnh sau cần kết quả mà lệnh trước chưa kịp ghi ra.
- **Xung đột điều khiển:** gặp lệnh rẽ nhánh, CPU chưa biết lệnh kế tiếp nằm ở đâu nên phải đợi hoặc phải dự đoán.

Ba loại xung đột này, cùng cách xử lý chúng, sẽ được nói kỹ ở chương 3 và ở các môn chuyên sâu hơn. Ở mức chương này, bạn chỉ cần ghi nhận một điều – và điều đó lại rất quan trọng cho phần hiệu năng vừa học:

- Không có ống lệnh, mỗi lệnh tốn nhiều nhịp, nên CPI lớn hơn 1 khá nhiều.
- Có ống lệnh, CPI tiến dần về 1.
- Bộ xử lý *siêu vô hướng* (superscalar) còn đi xa hơn: nó có nhiều ống lệnh chạy song song, nên mỗi nhịp hoàn tất được vài lệnh. Khi đó CPI nhỏ hơn 1.

Khi $CPI < 1$, người ta hay lật ngược nó lại cho dễ đọc và gọi là **IPC** (Instructions Per Cycle – số lệnh mỗi nhịp): $IPC = 1/CPI$. Nói “ $IPC = 2$ ” nghe xuôi tai hơn “ $CPI = 0,5$ ”, tuy hai câu là một. Bạn sẽ gặp lại IPC ở bài tập 13.

Phụ lục B: Nhập môn hợp ngữ

Cả chương này nói về việc CPU thực hiện lệnh. Nhưng bạn chưa hề tự viết một lệnh nào. Phụ lục này lấp chỗ trống đó.

Vì sao phải học hợp ngữ

Bên trong bộ nhớ, lệnh chỉ là những dãy bit. Lệnh “cộng thanh ghi B vào thanh ghi A” trên máy giả lập của ta là ba byte 0A 00 01. Không ai viết chương trình theo kiểu đó được.

Định nghĩa 2.9: Hợp ngữ

Hợp ngữ (assembly language) là cách viết lệnh máy bằng từ gợi nhớ thay cho dãy bit. Mỗi dòng hợp ngữ ứng với đúng một lệnh máy. Chương trình dịch hợp ngữ sang mã máy gọi là *trình hợp dịch* (assembler).

Ba byte khó nhớ ở trên, viết bằng hợp ngữ chỉ là:

```
ADD A, B
```

Ngày nay hầu như không ai viết cả chương trình bằng hợp ngữ nữa. Nhưng học nó vẫn rất đáng, vì ba lý do:

- Hợp ngữ là *ngôn ngữ duy nhất* cho bạn nhìn thấy trực tiếp thanh ghi, cờ, ngăn xếp và chu trình lệnh – tức toàn bộ những gì chương này dạy.
- Khi lập trình nhúng hoặc gỡ rối ở mức thấp, bạn sẽ phải đọc hợp ngữ.

- Hiểu hợp ngữ thì mới hiểu vì sao một dòng lệnh C lại nhanh hay chậm.

Cỗ máy dùng trong phụ lục này

Ta dùng một bộ xử lý 8 bit giả lập, cố ý làm thật nhỏ để học cho nhanh. Nó có đúng những thứ đã học ở chương này, không hơn:

- bốn thanh ghi đa năng A, B, C, D, mỗi cái 8 bit, chứa được số từ 0 đến 255;
- con trỏ ngăn xếp SP, khởi động ở 231, ngăn xếp lớn về phía địa chỉ thấp;
- bộ đếm chương trình PC – trên trang mô phỏng nó mang tên IP;
- thanh ghi lệnh IR, thanh ghi địa chỉ MAR và thanh ghi đệm dữ liệu MBR: ba thanh ghi này làm việc hoàn toàn bên trong CPU nên trang mô phỏng không hiện ra, nhưng chúng vẫn ở đó và vẫn hoạt động đúng như mục 2.3.3 mô tả;
- hai cờ: z (kết quả bằng 0) và c (có nhớ hoặc phải mượn);
- bộ nhớ 256 byte, đánh địa chỉ từ 0 đến 255.

Máy này không phải máy tưởng tượng: nó chính là máy của *Simple 8-bit Assembler Simulator*, một trang web miễn phí chạy thẳng trong trình duyệt. Mọi đoạn mã in trong phụ lục này gõ vào đó là chạy được ngay. Địa chỉ trang và hướng dẫn dùng nằm ở Phụ lục C.

Cú pháp cũng được chọn cho *giống 8086* – bộ xử lý mà bạn sẽ học ở chương 3. Nhờ vậy khi lên chương 3, bạn chỉ phải học thêm chứ không phải học lại. Cuối phụ lục có bảng đối chiếu từng lệnh sang 8086.

Một dòng hợp ngữ gồm những gì

Mỗi dòng có tối đa bốn phần, phần nào cũng có thể vắng:

nhãn: LỆNH toán hạng đích, toán hạng nguồn ; ghi chú

- **Nhãn** là tên đặt cho địa chỉ của dòng này, kết thúc bằng dấu hai chấm. Có nhãn thì mới nhảy tới đây được. Không cần đếm địa chỉ bằng tay bao giờ.
- **Lệnh** là từ gọi nhớ, ví dụ MOV, ADD. Viết hoa hay thường đều được.
- **Toán hạng**: phần lớn lệnh có hai toán hạng, ngăn cách bằng dấu phẩy. Quy ước quan trọng: *cái đứng trước là đích, cái đứng sau là nguồn*. Kết quả luôn được ghi vào toán hạng đích.
- **Ghi chú** bắt đầu bằng dấu chấm phẩy. Mọi thứ sau nó bị bỏ qua.

Năm cách chỉ ra một toán hạng

Cách chỉ ra toán hạng gọi là *chế độ địa chỉ*. Mục 2.1.4 đã nói tới hai chế độ hay gặp nhất khi toán hạng nằm trong bộ nhớ; bảng dưới liệt kê đủ năm cách mà máy giả lập chấp nhận, kể cả khi toán hạng không nằm trong bộ nhớ.

Bảng 2.7. Năm cách viết toán hạng và ý nghĩa của chúng

Cách viết	Tên gọi	Nghĩa
A	thanh ghi	Lấy giá trị đang nằm trong thanh ghi A.
7	tức thì	Chính con số 7, nằm ngay trong lệnh nên không phải đi lấy. Viết được ở hệ mười (7), hệ mười sáu (0x07) hay hệ hai (0b111).
[200]	trực tiếp	Nội dung của ô nhớ số 200.
[B]	gián tiếp qua thanh ghi	Nội dung của ô nhớ mà <i>giá trị trong B</i> chỉ tới.
[B+3]	gián tiếp có độ dời	Nội dung của ô nhớ số (<i>giá trị trong B</i>) + 3. Độ dời viết được từ -16 tới +15.

Chú ý một điểm dễ lẫn. Mục 2.1.4 gọi “gián tiếp” là trường hợp phải đọc bộ nhớ *hai lần*: lần đầu lấy địa chỉ, lần sau mới lấy dữ liệu. Còn [B] ở đây là *gián tiếp qua thanh ghi*: địa chỉ đã nằm sẵn trong thanh ghi rồi, chỉ phải đọc bộ nhớ một lần. Đây là biến thể rẻ hơn, và cũng là biến thể mà bộ xử lý thật dùng nhiều nhất.

Ba dòng cuối của bảng là chỗ dễ nhầm nhất, nên phân biệt thật rõ bằng một ví dụ. Giả sử $B = 200$ và ô nhớ 200 đang chứa số 42. *Bốn dòng dưới đây chỉ để đọc*, vì muốn chạy thật thì phải đặt trước hai giá trị đó.

```
MOV A, B      ; A nhận 200  -- chính giá trị của B
MOV A, [B]    ; A nhận 42   -- nội dung ô nhớ mà B trỏ tới
MOV A, [200]  ; A nhận 42   -- nội dung ô nhớ 200
MOV A, 200    ; A nhận 200  -- chính con số 200
```

Cặp ngoặc vuông [] đọc là “nội dung của ô nhớ”. Có ngoặc thì phải đi ra bộ nhớ lấy, tốn thêm một chu kỳ bus; không ngoặc thì lấy ngay tại chỗ. Đây đúng là sự khác nhau giữa hai dòng đầu và ba dòng cuối của bảng 2.7, và cũng là lý do vì sao chương trình dùng nhiều thanh ghi thì chạy nhanh hơn.

Nhóm lệnh thứ nhất: chuyển dữ liệu

Chỉ có một lệnh, nhưng là lệnh dùng nhiều nhất: MOV đích, nguồn chép giá trị từ nguồn sang đích. Bản thân nguồn không bị thay đổi – tên “MOV” (move, di chuyển) hơi gây hiểu nhầm, thực chất nó là *sao chép*.

```
MOV A, 25      ; đặt 25 vào A
MOV B, A       ; chép A sang B, A vẫn còn 25
MOV [100], A   ; ghi A ra ô nhớ 100
MOV A, [100]   ; đọc ngược lại từ ô nhớ 100
```

Nhóm lệnh thứ hai: số học và logic

Nhóm này gọi ALU làm việc, và *luôn luôn cập nhật các cờ*. Đó là điều phải nhớ.

```
ADD A, B       ; A = A + B
SUB A, 10      ; A = A - 10
INC A          ; A = A + 1
DEC C          ; C = C - 1
AND A, 0b1111  ; giữ lại 4 bit thấp của A
OR A, 0b1000   ; bật bit trọng số 8 lên 1
XOR A, A       ; cách nhanh nhất để đặt A về 0
NOT A          ; đảo toàn bộ 8 bit
SHL A, 1       ; dịch trái 1 bit = nhân 2
SHR A, 2       ; dịch phải 2 bit = chia 4
```

Dòng `XOR A, A` đáng dừng lại một chút. Vì $x \oplus x = 0$ với mọi bit, nên XOR một thanh ghi với chính nó luôn cho 0. Người viết hợp ngữ hay dùng mẹo này thay cho `MOV A, 0` vì nó ngắn hơn và trên nhiều bộ xử lý thì nhanh hơn.

Nhóm lệnh thứ ba: so sánh và rẽ nhánh

Đến đây mới là chỗ máy tính biết “quyết định”. Cơ chế gồm đúng hai bước:

1. `CMP a, b` – ALU tính thử $a - b$ nhưng *vứt kết quả đi*, chỉ giữ lại các cờ. Nếu $a = b$ thì $Z = 1$; nếu $a < b$ thì phải mượn nên $C = 1$.
2. Lệnh nhảy nhìn vào cờ đó mà quyết định có đổi PC hay không.

Bảng 2.8. Các lệnh nhảy và cờ mà chúng nhìn vào

Lệnh	Điều kiện	Sau <code>CMP a, b</code> thì nhảy khi
JMP nhãn	không có điều kiện	luôn luôn
JZ / JE	$Z = 1$	$a = b$
JNZ / JNE	$Z = 0$	$a \neq b$
JC / JB	$C = 1$	$a < b$
JNC / JAE	$C = 0$	$a \geq b$
JA	$C = 0$ và $Z = 0$	$a > b$
JBE	$C = 1$ hoặc $Z = 1$	$a \leq b$

Bảng trên chỉ đúng cho *số không dấu*. Với số có dấu phải dùng cờ khác, và bộ xử lý thật có riêng một bộ lệnh nhảy cho trường hợp đó – đúng như đã nói ở mục 2.2.7. Máy giả lập của ta chỉ làm việc với số không dấu cho gọn.

Mẹo nhớ tên lệnh: A là *above* (lớn hơn), B là *below* (nhỏ hơn), E là *equal* (bằng), N là *not* (không).

Ghép hai thứ lại, ta được cấu trúc *nếu...thì* quen thuộc:

```
CMP A, 100      ; A có nhỏ hơn 100 không?
JNC lon         ; C = 0 nghĩa là A >= 100 -> nhảy
MOV D, 1        ; tới đây tức là A < 100
JMP tiep
lon:
MOV D, 2        ; tới đây tức là A >= 100
tiep:
HLT
```

Vòng lặp

Vòng lặp chỉ là một lệnh nhảy *lùi* về phía sau. Khuôn mẫu hay dùng nhất là đếm lùi về 0, vì khi đó chỉ cần nhìn cờ Z:

```
MOV C, 5        ; lặp 5 lần
lap:
; ... phần thân vòng lặp ...
DEC C          ; bớt một lượt, cờ Z tự cập nhật
JNZ lap        ; C chưa về 0 thì quay lại
```

Chú ý: không cần lệnh CMP nào cả. Bản thân DEC đã đặt cờ Z rồi – đây là một minh họa rất cụ thể cho việc ALU luôn cập nhật cờ sau mỗi phép toán.

Nhóm lệnh thứ tư: ngăn xếp và chương trình con

Bốn lệnh, đúng như đã học ở mục 2.4.3:

- PUSH x – SP giảm, rồi cất x vào đỉnh ngăn xếp;
- POP r – lấy đỉnh ngăn xếp vào r, rồi SP tăng;
- CALL nhãn – cất địa chỉ trở về vào ngăn xếp, rồi nhảy tới nhãn;
- RET – lấy địa chỉ trở về khỏi ngăn xếp, nạp vào PC.

```

MOV A, 6
CALL nhandoi    ; A thành 12
MOV [200], A
HLT

nhandoi:        ; chương trình con: A = A * 2
    PUSH B      ; giữ hộ B cho người gọi
    MOV B, A
    ADD A, B
    POP B       ; trả B lại nguyên vẹn
    RET
    
```

Hai dòng PUSH B và POP B là *quy tắc lịch sự* giữa chương trình con và người gọi: chương trình con mượn thanh ghi nào thì phải trả lại nguyên như cũ. Nếu quên POP B, chẳng những B bị hỏng mà lệnh RET còn lấy nhầm giá trị B làm địa chỉ trở về và chương trình chạy loạn – đúng lỗi mất cân bằng ngăn xếp đã cảnh báo ở ví dụ 2.6.

Đặt sẵn dữ liệu vào bộ nhớ

Lệnh giả DB (define byte) không sinh ra lệnh máy nào; nó chỉ đặt sẵn giá trị vào bộ nhớ lúc nạp chương trình. Dùng nó để tạo mảng:

```

day: DB 3
      DB 9
      DB 12
    
```

Nhân day bây giờ mang *địa chỉ* của phần tử đầu tiên. Ghép với chế độ địa chỉ gián tiếp, ta duyệt được cả mảng:

```

MOV B, day      ; B trở vào phần tử đầu
MOV A, [B]      ; đọc phần tử B đang trỏ tới
INC B          ; sang phần tử kế tiếp
    
```

Đây chính là cách con trỏ hoạt động trong ngôn ngữ C, chỉ khác là C giấu nó đi.

Khối DB phải đặt *sau* lệnh HLT. Lý do rất đơn giản: CPU không phân biệt được lệnh với dữ liệu – cả hai đều là byte. Nếu để khối dữ liệu lên trước, CPU chạy tới đó sẽ coi dữ liệu là mã lệnh và làm những việc vô nghĩa. Đây chính là hệ quả của nguyên lý von Neumann đã nói ở mục 2.1.1: lệnh và dữ liệu nằm chung một bộ nhớ.

Chương trình phục vụ ngắt trông như thế nào

Mục 2.1.4 nói CPU kiểm tra ngắt ở cuối mỗi chu trình lệnh, và nếu có yêu cầu thì cất PC vào ngăn xếp rồi nhảy tới chương trình phục vụ ngắt. Nhìn từ góc hợp ngữ, việc đó giống hệt một lệnh CALL – chỉ khác ở chỗ *phần cứng tự gọi*, chương trình chính không hề biết.

Vì vậy một chương trình phục vụ ngắt viết ra không khác chương trình con là mấy:

```
phucvu:          ; phần cứng tự nhảy vào đây khi có ngắt
    PUSH A       ; giữ hộ thanh ghi cho chương trình chính
    MOV A, 255
    MOV [250], A ; việc cần làm khi có ngắt
    POP A        ; trả lại nguyên vẹn
    RET          ; quay về đúng chỗ đã dừng
```

Điểm khác biệt duy nhất, và cũng là điểm quan trọng nhất: chương trình phục vụ ngắt *bắt buộc* phải trả lại mọi thanh ghi và mọi cờ nguyên như cũ. Chương trình con thường thì người gọi biết mình đang gọi nên còn liệu trước được; còn ngắt thì cắt ngang giữa chừng ở một chỗ không ai đoán được. Đây chính là lý do phải cất PSW như đã nói ở mục 2.4.7.

Trang mô phỏng ở Phụ lục C *không có* cơ chế ngắt – nó không có thiết bị ngoài nào để xin ngắt cả. Muốn xem đoạn mã trên chạy thì gọi tay bằng CALL phucvu, rồi tự hình dung rằng ở máy thật, lệnh gọi đó do phần cứng phát ra chứ không do bạn viết.

Tập lệnh đầy đủ

Không phải lệnh nào cũng nhận đủ bốn cách viết toán hạng. Máy giả lập giới hạn như sau, và trình dịch sẽ báo lỗi nếu viết sai:

- toán hạng *đích* của mọi lệnh số học và logic phải là **thanh ghi**;
- INC, DEC, NOT, POP chỉ nhận đúng một thanh ghi;
- CALL chỉ nhận một địa chỉ hoặc một nhãn;
- MOV không chép thẳng từ ô nhớ này sang ô nhớ khác được – phải đi vòng qua một thanh ghi.

Một số lệnh còn có tên gọi khác, viết cách nào cũng được: JNA = JBE, SAL = SHL, SAR = SHR. Lệnh DB chỉ nhận *một* giá trị mỗi dòng: muốn đặt ba byte thì phải viết ba dòng DB. Riêng chuỗi ký tự thì viết gọn được thành DB "Hi", trình dịch tự trải ra thành từng mã ký tự.

Bảng 2.9. Toàn bộ tập lệnh của máy giả lập

Lệnh	Cờ đổi	Việc làm
MOV đ, ng	không	đích $\leftarrow$ nguồn
ADD đ, ng	Z, C	đích $\leftarrow$ đích + nguồn
SUB đ, ng	Z, C	đích $\leftarrow$ đích – nguồn
INC r	Z	$r \leftarrow r + 1$
DEC r	Z	$r \leftarrow r - 1$
MUL ng	Z, C	$A \leftarrow A \times$ nguồn; chỉ giữ 8 bit thấp, $C = 1$ nếu tràn
DIV ng	Z, C	$A \leftarrow A \div$ nguồn (chia nguyên); chia cho 0 thì máy dừng và bật cờ F
AND đ, ng	Z, C	phép AND theo từng bit
OR đ, ng	Z, C	phép OR theo từng bit
XOR đ, ng	Z, C	phép XOR theo từng bit
NOT r	Z, C	đảo toàn bộ bit của r
SHL r, n	Z, C	dịch trái n bit ($\times 2^n$)
SHR r, n	Z, C	dịch phải n bit ($\div 2^n$)
CMP a, b	Z, C	tính thử $a - b$, chỉ giữ cờ
JMP nhãn	không	$PC \leftarrow$ nhãn
JZ JNZ JC JNC JA JBE	không	nhảy nếu điều kiện thoả (bảng 2.8)
PUSH x	không	SP giảm, cất x vào đỉnh ngăn xếp
POP r	không	lấy đỉnh ngăn xếp vào r, SP tăng
CALL nhãn	không	cất địa chỉ trở về rồi nhảy
RET	không	lấy địa chỉ trở về, nạp vào PC
HLT	không	dừng hẳn bộ xử lý
DB giá trị	–	đặt sẵn dữ liệu vào bộ nhớ (không phải lệnh máy)

Hai ví dụ tổng hợp

Ví dụ 2.15: Dò tay một chương trình

Cho chương trình dưới đây. Hãy dò từng lệnh và cho biết giá trị cuối cùng của A, c và ô nhớ 200.

```

MOV A, 0      ; tổng
MOV C, 4      ; đếm lùi
lap:
ADD A, C
DEC C
JNZ lap
MOV [200], A
HLT
    
```

Giải. Lập bảng theo dõi từng vòng lặp. Cột cuối là cờ Z sau lệnh DEC c, vì chính nó quyết định có lặp tiếp hay không.

Vòng	A trước	C	A sau ADD	Z sau DEC
1	0	4	4	0 (C còn 3)
2	4	3	7	0 (C còn 2)
3	7	2	9	0 (C còn 1)
4	9	1	10	1 (C còn 0)

Ở vòng thứ tư, DEC c làm C về 0 nên $Z = 1$, lệnh JNZ không nhảy nữa và chương trình đi tiếp.

Đáp số: $A = 10$, $C = 0$, ô nhớ 200 = 10. Chương trình này tính $4 + 3 + 2 + 1 = 10$.

Ví dụ 2.16: Đếm số phần tử bằng 0 trong một mảng

Viết chương trình đếm xem trong mảng 6 phần tử dưới đây có bao nhiêu phần tử bằng 0, cất kết quả vào ô nhớ 200.

Giải. Cần ba thanh ghi: một làm con trỏ, một đếm số phần tử còn lại, một đếm kết quả.

```

MOV B, day      ; B: con trỏ chạy trên mảng
MOV C, 6        ; C: còn 6 phần tử phải xét
MOV D, 0        ; D: kết quả đếm được

lap:
MOV A, [B]      ; lấy phần tử ra
CMP A, 0        ; nó có bằng 0 không?
JNZ khác        ; khác 0 thì bỏ qua
INC D           ; bằng 0 thì đếm thêm một

khac:
INC B           ; sang phần tử kế tiếp
DEC C           ; bớt một phần tử phải xét
JNZ lap         ; còn thì lặp
MOV [200], D
HLT

day: DB 7
      DB 0
      DB 4
      DB 0
      DB 0
      DB 9
    
```

Mảng có ba số 0 nên ô nhớ 200 sẽ chứa 3.

Hai chỗ đáng chú ý. Thứ nhất, nhãn *khac* nằm ở giữa vòng lặp, đóng vai “bỏ qua phần thân” – đây là cách hợp ngữ diễn đạt câu *nếu...thì* không có nhánh *ngược lại*. Thứ hai, thứ tự DEC C rồi mới JNZ lap là bắt buộc: nếu đảo lại, cờ Z sẽ là cờ do INC B đặt chứ không phải do DEC C, và vòng lặp sẽ sai.

Bắc cầu sang 8086

Chương 3 sẽ học hợp ngữ 8086 thật. Bảng dưới cho thấy bạn đã đi được bao xa: phần lớn lệnh giữ nguyên tên, chỉ đổi tên thanh ghi.

Ghi nhớ

Một dòng hợp ngữ bằng một lệnh máy. Cấu trúc: nhãn: LỆNH đích, nguồn ; ghi chú – đích luôn đứng trước. Năm cách viết toán hạng ứng với năm chế độ địa chỉ: A (thanh ghi), 7 (tức thì), [200] (trực tiếp), [B] (gián tiếp), [B+3] (gián tiếp có độ dời). Muốn máy *quyết định* thì dùng CMP đặt cờ rồi J... nhìn cờ. Muốn *lặp* thì nhảy lùi. Muốn gọi chương trình con thì CALL và RET, nhớ trả lại thanh ghi đã mượn.

Bảng 2.10. Đối chiếu máy giả lập với bộ xử lý 8086

Máy giả lập	8086	Khác nhau ở chỗ nào
A B C D (8 bit)	AX BX CX DX (16 bit)	8086 tách được mỗi thanh ghi thành hai nửa, ví dụ AH và AL
SP	SP	giống hệt, chỉ khác là mỗi mục ngăn xếp chiếm 2 byte
PC	IP	cùng vai trò, chỉ khác tên
Cờ Z, C	ZF, CF và thêm SF, OF	8086 có đủ bốn cờ như bảng 2.2
MOV ADD SUB INC DEC	giống hệt	không phải học lại
CMP và các lệnh J...	giống hệt	8086 có thêm bộ lệnh nhảy cho số có dấu: JG, JL
PUSH POP CALL RET	giống hệt	không phải học lại
HLT	INT 21h để kết thúc	8086 kết thúc bằng lời gọi hệ điều hành
—	.MODEL, .DATA, .CODE	8086 cần khai báo cấu trúc chương trình trước khi viết lệnh

Phụ lục C: Thực hành trên web

Đọc về CPU là một chuyện, nhìn nó chạy lại là chuyện khác. Ba công cụ dưới đây đều *miễn phí*, chạy thẳng trong trình duyệt, *không phải cài đặt gì*, không phải đăng ký tài khoản, và dùng được cả trên điện thoại. Hãy mở ít nhất một cái trong lúc đọc lại chương này.

1. Simple 8-bit Assembler Simulator – công cụ chính

<https://schweigi.github.io/assembler-simulator/>

Đây là công cụ dùng cho toàn bộ Phụ lục B. Nó chạy *đúng* tập lệnh và *đúng* cú pháp đã học ở đó, nên mọi đoạn mã in trong tài liệu này gõ vào đây là chạy được ngay, không phải sửa một chữ nào.

Máy mà nó mô phỏng cũng chính là máy đã mô tả ở đầu Phụ lục B: bốn thanh ghi A, B, C, D, con trỏ lệnh IP, con trỏ ngăn xếp SP, hai cờ Z và C, bộ nhớ 256 byte.

Năm bước để bắt đầu:

1. Xóa chương trình mẫu có sẵn trong ô soạn thảo, gõ vào đoạn mã của bạn. Bắt đầu bằng bài đơn giản nhất ở Phụ lục B.
2. Bấm **Assemble** (hợp dịch). Nếu có lỗi cú pháp, công cụ sẽ báo ngay dưới ô soạn thảo. Nếu không lỗi, nhìn sang bảng **RAM**: chương trình vừa biến thành các byte mã lệnh nằm ở đầu bộ nhớ.
3. Bấm **Step** (từng bước) liên tục. Mỗi lần bấm là CPU thực hiện trọn *một lệnh*. Nhìn hàng

thanh ghi phía trên đổi giá trị, nhìn IP nhảy sang lệnh kế tiếp, và nhìn ô nhớ tương ứng trong bảng RAM sáng lên.

4. Khi đã quen, bấm **Run** để chạy cả chương trình, **Stop** để dừng giữa chừng, **Reset** để đưa mọi thứ về trạng thái ban đầu.
5. Bảng RAM có nút chuyển giữa cách hiển thị *hex* và *decimal*. Lúc mới học nên để *decimal* cho dễ đối chiếu với số trong đề bài, khi đã quen thì chuyển sang *hex* vì gọn hơn.

Nhìn ở đâu để thấy cái gì:

Bảng 2.11. Đối chiếu màn hình mô phỏng với các mục của chương

Chỗ trên màn hình	Ứng với mục nào của chương
Hàng A B C D	mục 2.4.4 – thanh ghi dữ liệu và đa năng
Ô IP	mục 2.4.2 – bộ đếm chương trình (chương này gọi là PC)
Ô SP	mục 2.4.3 – con trỏ ngăn xếp
Hai cờ Z và C	mục 2.2.7 – cờ trạng thái
Bảng RAM 256 ô	mục 2.5.2 – không gian địa chỉ
Vùng RAM đầu bảng	mã lệnh – xem được lệnh đã thành byte thế nào
Bảng Labels	địa chỉ mà mỗi nhãn ứng với
Ô đếm Instructions	mục 2.6.4 – số lệnh <i>IC</i> (bật ở nút hiện/ẩn)
Ô Output	vào–ra ánh xạ bộ nhớ (xem ghi chú dưới)

Công cụ này có một vùng màn hình ảo rất đáng thử: ghi mã ký tự vào các ô nhớ từ 232 đến 255 thì chữ hiện ra ở ô **Output**. Ví dụ `MOV [232], 'A'` sẽ in ra chữ A.

Cách làm đó gọi là *vào–ra ánh xạ bộ nhớ*: thiết bị được gán sẵn cho một dải ô nhớ, nên điều khiển thiết bị chỉ đơn giản là ghi vào ô nhớ, không cần lệnh vào–ra riêng. Đây là một trong hai cách tổ chức vào–ra mà bạn sẽ học kỹ ở chương 6.

Cũng vì vùng này chiếm từ ô 232 trở lên nên ngăn xếp bắt đầu ngay bên dưới nó và lớn dần xuống phía địa chỉ thấp.

Hai khác biệt nhỏ so với Phụ lục B, không ảnh hưởng gì tới bài học nhưng nên biết để khỏi bỡ ngỡ:

- Bộ đếm chương trình ở đây tên là IP chứ không phải PC. Hai tên cùng chỉ một thứ, như đã nói ở mục 2.4.2.
- Ngoài Z và C còn có thêm một cờ F (fault), bật lên khi chương trình làm điều gì đó sai, ví dụ chia cho 0. Chương 2 không dùng tới cờ này.
- Thứ tự bên trong lệnh `PUSH` hơi khác: công cụ ghi giá trị vào *chính ô mà SP đang trỏ* rồi mới giảm SP, chứ không giảm trước. Giá trị của SP sau lệnh vẫn đúng như mô tả ở

mục 2.4.3, chỉ có điều muốn tìm giá trị vừa cất trong bảng RAM thì phải nhìn ô SP+1 chứ không phải ô SP.

- Lệnh SAR ở đây chỉ là *tên khác của SHR* – công cụ không có phép dịch phải số học thật. Các bài về SAR ở mục 2.2 phải làm trên giấy.

2. Little Man Computer

<https://peterhigginson.co.uk/lmc/>

Mô hình “người tí hon trong máy tính” kinh điển. Điểm mạnh riêng của nó là **hoạt hình**: bấm chạy từng bước, bạn thấy đúng vòng nhận lệnh – giải mã – thực hiện diễn ra trước mắt, với mũi tên chạy từ bộ nhớ sang thanh ghi lệnh, rồi sang bộ cộng.

Tập lệnh chỉ vỏn vẹn 12 từ gợi nhớ, đơn giản hơn hẳn máy ở Phụ lục B, và cú pháp cũng khác (chỉ có một thanh ghi tích lũy, không có A B C D). Vì vậy đừng dùng nó để làm bài tập – hãy dùng nó cho đúng một việc: *nhìn thấy chu trình lệnh*.

Nếu sau khi đọc mục 2.1.4 mà bạn vẫn thấy chu trình lệnh mơ hồ thì đây là công cụ nên mở trước tiên. Nó bỏ hết chi tiết rườm rà và chỉ để lại đúng cái vòng lặp cốt lõi.

3. Venus – trình mô phỏng RISC-V

<https://venus.kvakil.me/>

Đây là bước tiếp theo, dành cho khi bạn đã vững hai công cụ trên. RISC-V là kiến trúc tập lệnh mở đang được dùng rộng rãi trong nghiên cứu và trong công nghiệp; Venus cho chạy hợp ngữ RISC-V thẳng trên trình duyệt. Giao diện có hai thẻ **Editor** và **Simulator**, các nút **Run**, **Step**, **Prev**, **Reset**, bảng 32 thanh ghi và bảng bộ nhớ. Đáng chú ý nhất là ba thẻ *Machine Code*, *Basic Code* và *Original Code*: chúng cho thấy cùng một chương trình ở ba mức, từ hợp ngữ bạn viết cho tới mã máy nhị phân thật.

Kiến trúc này có 32 thanh ghi và *không có thanh ghi cờ* – cùng triết lý RISC với MIPS trong bảng 2.4, và vì lý do đã bàn ở đó.

Nên dùng công cụ nào

Ghi nhớ

Lộ trình gợi ý: mở *Simple 8-bit Assembler Simulator* trong lúc học Phụ lục B, gõ lại từng khung mã và bấm **Step** cho tới khi đoán trước được thanh ghi nào sắp đổi. Sau đó làm nhóm bài tập E. Ai thấy chu trình lệnh còn mơ hồ thì mở thêm Little Man Computer cho dễ hình dung; ai muốn đi xa hơn thì thử Venus để chuẩn bị cho chương 3 và các môn sau.

Câu hỏi ôn tập

Bảng 2.12. So sánh ba công cụ

Công cụ	Cú pháp	Độ khó	Mạnh nhất ở
Simple Assembler	8-bit Đúng Phụ lục B	Dễ	thanh ghi, cờ, ngăn xếp, mã máy thật, vào-ra ánh xạ bộ nhớ
Little Man Computer	Riêng, rất gọn	Rất dễ	hoạt hình chu trình lệnh, chỉ 12 lệnh
Venus (RISC-V)	RISC-V	Khó hơn	kiến trúc thật, dùng được cho các môn sau

Các câu dưới đây đều trả lời được bằng lời, không cần tính toán. Mười ba câu đầu bám sát đề cương của bộ môn, nên hãy làm chắc phần đó trước.

Cách dùng hiệu quả nhất: che tài liệu lại, tự trả lời bằng lời của mình, rồi mới mở ra đối chiếu. Nếu trả lời được mà không cần nhìn sách thì phần đó bạn đã thuộc.

1. Liệt kê các thành phần và nhiệm vụ của CPU.
2. Mô tả tổ chức một máy tính đơn giản và hoạt động của các bộ phận chính trong CPU.
3. Mô tả hoạt động của CPU diễn ra trong một chu kỳ lệnh.
4. Mô tả chức năng của ALU.
5. Mô tả chức năng của CU.
6. Nêu vai trò tổng quát của các thanh ghi trong bộ vi xử lý.
7. Bộ thanh ghi gồm những loại nào?
8. Thanh ghi trạng thái chương trình (Program Status Word) là gì?
9. Thế nào là ngắt? Nêu các giai đoạn thực hiện ngắt của CPU.
10. Định nghĩa ống dẫn lệnh.
11. Vẽ hình mô tả ống dẫn lệnh. Tại sao kỹ thuật ống dẫn lệnh làm tăng tốc độ CPU?
12. Định nghĩa bộ nhớ ngăn xếp, mô tả hoạt động khi cất và khi lấy dữ liệu ra khỏi ngăn xếp.
13. Nêu chức năng và phân loại hệ thống bus.
14. Vì sao nói CPU có hai chức năng chứ không phải một? Cho ví dụ minh họa chức năng điều khiển.
15. Vì sao nội dung PC được tăng ngay trong pha nhận lệnh chứ không phải sau khi thực hiện xong lệnh?

16. Phân biệt thanh ghi lập trình được và thanh ghi không lập trình được. Cho ba ví dụ mỗi loại.
17. So sánh chế độ nhận toán hạng trực tiếp với chế độ gián tiếp về số chu kỳ bus và về tính linh hoạt.
18. Nêu vai trò của các thanh ghi MAR, MBR, IR trong pha nhận lệnh.
19. Vì sao bộ cộng bán phần không dùng được cho các bit từ bit 1 trở lên?
20. Viết biểu thức logic của bit tổng và bit nhớ ra trong bộ cộng toàn phần. Phát biểu bằng lời hai biểu thức đó.
21. Nhược điểm của bộ cộng nối tiếp là gì? Bộ cộng nhìn trước bit nhớ khắc phục nhược điểm đó bằng cách nào?
22. Giải thích vì sao ALU không cần mạch trừ riêng.
23. Phân biệt dịch phải logic và dịch phải số học. Dùng nhầm thì hậu quả thế nào?
24. Phân biệt cờ CF và cờ OF. Có thể xảy ra trường hợp cả hai cờ cùng bằng 1 không? Cho ví dụ.
25. Nêu bốn nhóm tín hiệu vào và hai nhóm tín hiệu ra của khối điều khiển.
26. Vi thao tác là gì? Viết dãy vi thao tác của pha nhận lệnh.
27. So sánh điều khiển bằng mạch cứng với điều khiển bằng vi chương trình theo bốn tiêu chí: tốc độ, độ phức tạp thiết kế, khả năng sửa đổi và loại tập lệnh phù hợp.
28. Vì sao ngăn xếp lại phù hợp với việc phục vụ chương trình con lồng nhau?
29. Trình bày diễn biến của SP và của ngăn xếp khi thực hiện cặp lệnh `CALL – RET`.
30. Vì sao phải cất PSW khi vào chương trình con phục vụ ngắt?
31. Vì sao kiến trúc MIPS không có thanh ghi cờ? Lợi ích của lựa chọn thiết kế đó?
32. Độ rộng bus địa chỉ và độ rộng bus dữ liệu ảnh hưởng tới thông số nào của hệ thống?
33. So sánh ba phương án tổ chức bus trong CPU: một bus, hai bus và ba bus.
34. Viết công thức tính thời gian thực hiện chương trình theo IC , CPI và T_0 . Mỗi thừa số chịu ảnh hưởng của những yếu tố nào?
35. Vì sao không thể dùng MIPS để so sánh hai bộ xử lý có kiến trúc tập lệnh khác nhau?

Bài tập

Mười sáu bài dưới đây chia làm bốn nhóm theo bốn mục của chương. Đáp án đầy đủ nằm ngay sau phần này – nhưng hãy tự làm hết một lượt rồi mới xem, nếu không thì việc làm bài chẳng còn tác dụng gì.

Nếu bí, đừng mở đáp án ngay. Hãy quay lại đọc ví dụ mẫu trong mục tương ứng: gần như bài nào cũng có một ví dụ cùng dạng ở phần lý thuyết.

A. Chu trình lệnh và các thanh ghi

Gợi ý chung cho nhóm A. Ba điều dùng đi dùng lại: (1) bus địa chỉ n đường thì đánh địa chỉ được 2^n ô; (2) PC tăng bằng đúng độ dài một lệnh, và tăng ngay ở công đoạn nhận lệnh; (3) PUSH thì SP giảm, POP thì SP tăng, mức tăng giảm bằng kích thước một mục ngăn xếp.

1. Một bộ xử lý có bus địa chỉ 32 đường, bus dữ liệu 64 đường, lệnh dài cố định 4 byte, bộ nhớ đánh địa chỉ theo byte. Hãy xác định: a) dung lượng bộ nhớ tối đa quản lý được; b) độ rộng tối thiểu của các thanh ghi PC, MAR và MBR; c) giá trị PC sau khi nhận xong lệnh nằm tại địa chỉ $(0040\ 1000)_{16}$.
2. Bộ nhớ đánh địa chỉ theo byte, lệnh dài 4 byte. Tại thời điểm đang xét $PC = (0200)_{16}$, ngăn nhớ $(0200)_{16}$ chứa lệnh `SUB R3, [0310]`, $R3 = (0050)_{16}$ và ngăn nhớ $(0310)_{16}$ chứa $(0012)_{16}$. Hãy lập bảng theo dõi năm công đoạn của chu trình lệnh, ghi rõ nội dung MAR, MBR, IR, PC và R3 sau mỗi công đoạn.
3. Một bộ xử lý dùng chế độ địa chỉ gián tiếp. Lệnh `LOAD R1, @[1000]` nghĩa là: đọc nội dung ngăn nhớ $(1000)_{16}$, coi giá trị đó là địa chỉ, rồi đọc nội dung ở địa chỉ ấy vào R1. Biết $M((1000)_{16}) = (2000)_{16}$ và $M((2000)_{16}) = (00AB)_{16}$. Hãy nêu dãy thao tác trên bus và cho biết lệnh này tốn bao nhiêu chu kỳ bus, so với chế độ trực tiếp thì tốn thêm bao nhiêu.
4. Một bộ xử lý 16 bit, mỗi mục ngăn xếp 2 byte, ngăn xếp lớn dần về phía địa chỉ thấp, ban đầu $SP = (3000)_{16}$. Chương trình thực hiện lần lượt: `PUSH AX`, `PUSH BX`, `CALL SUB`, và trong `SUB` có `PUSH CX` rồi `POP CX` rồi `RET`. Hãy lập bảng giá trị SP sau mỗi lệnh. Nếu lập trình viên quên lệnh `POP CX` thì điều gì xảy ra khi gặp `RET`?

B. Đơn vị số học và logic

Gợi ý chung cho nhóm B. Với bài tính cờ, hãy làm theo đúng bốn bước của ví dụ 2.2: đổi sang nhị phân → lấy bù hai nếu là phép trừ → cộng → đọc kết quả và xét từng cờ. Nhớ CF dành cho cách hiểu *không dấu*, còn OF dành cho cách hiểu *có dấu*; hai cờ này xét độc lập với nhau.

5. Lập bảng chân lý của bộ cộng toàn phần và viết biểu thức logic của S và C_{out} . Chỉ ra

rằng bộ cộng toàn phần có thể ghép từ hai bộ cộng bán phần cùng một cổng OR.

6. Thực hiện các phép tính sau trên số bù hai 8 bit. Với mỗi phép, ghi kết quả nhị phân, giá trị thập phân và bốn cờ ZF, SF, CF, OF: a) $45 - 78$; b) $(-60) + (-70)$; c) $96 + 32$; d) $100 - 100$.
7. Cho $A = 10010110$. Hãy xác định kết quả của các phép sau và cho biết giá trị thập phân của kết quả khi hiểu A là số bù hai: a) $\text{SHL } A, 1$; b) $\text{SHR } A, 1$; c) $\text{SAR } A, 1$; d) $\text{ROR } A, 2$.
8. Một vi điều khiển 8 bit không có lệnh nhân. Hãy viết cách tính các tích sau chỉ bằng phép dịch, phép cộng và phép trừ: a) $A \times 12$; b) $A \times 20$; c) $A \times 15$. Kiểm tra kết quả câu c) với $A = 9$.

C. Khối điều khiển và hệ thống bus

Gợi ý chung cho nhóm C. Độ rộng một vi lệnh bằng tổng số bit của các phần hợp thành. Tốc độ trao đổi dữ liệu bằng số byte mỗi chu kỳ bus chia cho thời gian một chu kỳ bus. Và nhớ: dung lượng nhớ tối đa chỉ phụ thuộc *bus địa chỉ*, không liên quan gì tới bus dữ liệu.

9. Một CPU dùng điều khiển vi chương trình có 56 tín hiệu điều khiển; mỗi vi lệnh có thêm 7 bit địa chỉ vi lệnh kế tiếp và 3 bit chọn điều kiện rẽ nhánh. Bộ nhớ điều khiển chứa 2048 vi lệnh. Tính độ rộng một vi lệnh và dung lượng bộ nhớ điều khiển theo bit và theo KiB.
10. CPU tổ chức theo phương án một bus, có thanh ghi đệm Y ở đầu vào ALU và thanh ghi đệm Z ở đầu ra. Hãy viết dãy vi thao tác cho lệnh $\text{SUB } R1, [1000]$ (nghĩa là $R1 \leftarrow R1 - M[(1000)_{16}]$), kể cả pha nhận lệnh, và cho biết lệnh chiếm tất cả bao nhiêu nhịp.
11. Một hệ thống có bus địa chỉ 24 đường và bus dữ liệu 32 đường, chu kỳ bus 10 ns. Hãy tính: a) dung lượng bộ nhớ tối đa; b) tốc độ trao đổi dữ liệu tối đa theo MB/s; c) nếu tăng bus dữ liệu lên 64 đường thì tốc độ đó thay đổi thế nào và dung lượng nhớ tối đa có đổi không?

D. Hiệu năng CPU

Bốn bài đầu của nhóm này lấy nguyên văn từ *đề bài tập Chương 2* của bộ môn, ở đó chúng được đánh số 14, 15, 16 và 17.

Gợi ý chung cho nhóm D. Cả nhóm chỉ dùng một công thức: $t_{CPU} = IC \times CPI \times T_0 = \frac{IC \times CPI}{f_0}$. Ba việc nên làm với mọi bài:

1. Viết ra ba đại lượng IC , CPI , f_0 mà đề đã cho, và đánh dấu cái nào đang phải tìm.

2. Nếu đề không cho IC mà hai máy cùng kiến trúc tập lệnh, cứ để IC nguyên như một chữ – lập tỉ số thì nó tự triệt tiêu.
3. Đổi hết về đơn vị cơ bản (Hz và giây) trước khi thay số, rồi mới đổi kết quả về ns hay MHz cho dễ đọc.

Nhắc lại chỗ hay nhầm nhất: khi so hai máy, tỉ số hiệu năng bằng tỉ số thời gian *đảo ngược*, $P_A/P_B = t_B/t_A$.

12. Cho ba bộ xử lý P1, P2 và P3 cùng chạy một tập lệnh, với tốc độ xung nhịp và CPI cho trong bảng sau.

Bộ xử lý	Clock rate	CPI
P1	2 GHz	1,5
P2	1,5 GHz	1,0
P3	3 GHz	2,5

- a) Bộ xử lý nào có hiệu năng cao nhất theo tiêu chí số lệnh thực thi trong một giây (IPS) và số triệu lệnh thực thi trong một giây (MIPS)?
- b) Nếu mỗi bộ xử lý chạy một chương trình nào đó hết 10 giây, tìm tổng số chu kỳ và tổng số lệnh tương ứng.
- c) Nếu muốn giảm 30% thời gian thực thi, mà việc đó dẫn tới tăng 20% CPI, thì tốc độ xung nhịp mới của từng bộ xử lý phải là bao nhiêu?

13. Cho bảng thông số của ba bộ xử lý sau.

Bộ xử lý	Clock rate	Số lệnh	Thời gian
P1	2 GHz	20×10^9	7 s
P2	1,5 GHz	30×10^9	10 s
P3	3 GHz	90×10^9	9 s

- a) Tìm IPC – số lệnh thực hiện trong một chu kỳ – của mỗi bộ xử lý.
- b) Tìm tốc độ xung nhịp mới cho P2 để P2 giảm thời gian thực thi bằng P1.
- c) Tìm số lệnh của P2 để thời gian thực thi của nó bằng của P3.

14. Hai bộ xử lý có cùng tập lệnh. Tập lệnh được chia thành bốn lớp theo CPI (A, B, C và D) như bảng sau.

Bộ xử lý	Clock rate	CPI lớp A	CPI lớp B	CPI lớp C	CPI lớp D
P1	1,5 GHz	1	2	3	4
P2	2 GHz	2	2	2	3

- Cho chương trình có 10^6 lệnh, trong đó 10% lệnh lớp A, 20% lớp B, 50% lớp C và 20% lớp D. Bộ xử lý nào thực hiện chương trình này nhanh hơn?
- Tính CPI trung bình của mỗi bộ xử lý với chương trình ở câu a).
- Tìm số chu kỳ cần thiết cho mỗi bộ xử lý.
- Giả sử lệnh toán học (Arith) cần 1 chu kỳ; lệnh đọc dữ liệu từ bộ nhớ (Load) và ghi dữ liệu vào bộ nhớ (Store) cần 5 chu kỳ; lệnh rẽ nhánh (Branch) cần 2 chu kỳ. Tìm thời gian thực thi của chương trình dưới đây trên bộ xử lý 2 GHz.

Arith	Store	Load	Branch	Tổng
500	50	100	50	700

- Tìm CPI của chương trình ở câu d).
- Nếu số lệnh Load giảm một nửa thì chương trình tăng tốc bao nhiêu lần (speedup) và CPI mới là bao nhiêu?

15. Xét hai bộ xử lý P1 và P2 có cùng một tập lệnh, gồm năm lớp lệnh A, B, C, D và E. Tốc độ xung nhịp và CPI của mỗi lớp cho trong bảng sau.

Bộ xử lý	Clock rate	CPI lớp A	CPI lớp B	CPI lớp C	CPI lớp D	CPI lớp E
P1	1,0 GHz	1	2	3	4	3
P2	1,5 GHz	2	2	2	4	4

- Một máy tính được xem là đạt hiệu suất đỉnh điểm (peak performance) khi nó thực thi chuỗi lệnh cho tốc độ nhanh nhất. Tính số lệnh thực thi trong một giây khi P1 và P2 đạt hiệu suất đỉnh điểm.
- Nếu số lệnh của một chương trình được chia đều cho các lớp lệnh, ngoại trừ lớp A có số lệnh gấp đôi các lớp khác, thì máy nào chạy nhanh hơn và nhanh hơn bao nhiêu lần?
- Nếu số lệnh được chia đều cho các lớp, ngoại trừ lớp E có số lệnh gấp đôi các lớp khác, thì máy nào chạy nhanh hơn và nhanh hơn bao nhiêu lần?
- Bảng dưới cho biết sự phân chia số lượng lệnh theo nhóm lệnh của hai chương trình khác nhau.

	Compute	Load	Store	Branch	Tổng
Chương trình 1	1000	400	100	50	1550
Chương trình 2	1500	300	100	100	2000

Lưu ý: cột “Tổng” ở đây đã được tính lại bằng tổng bốn cột trước. Bảng trong đề gốc ghi 15 500 và 1750, không khớp với các số hạng; hãy dùng tổng đúng khi tính CPI.

Giả sử lệnh tính toán (Compute) mất 1 chu kỳ, lệnh đọc dữ liệu từ bộ nhớ (Load) và ghi dữ liệu vào bộ nhớ (Store) mất 10 chu kỳ, lệnh rẽ nhánh (Branch) mất 3 chu kỳ. Hãy tính thời gian thực thi trên một bộ xử lý MIPS 3 GHz.

e) Giả sử lệnh Compute mất 1 chu kỳ, lệnh Load và Store mất 2 chu kỳ, lệnh Branch mất 3 chu kỳ. Hãy tính lại thời gian thực thi trên bộ xử lý MIPS 3 GHz.

16. Một chương trình chạy trên bộ xử lý 2,5 GHz gồm 8×10^8 lệnh, trong đó 45% lệnh số học (1 chu kỳ), 25% lệnh truy nhập bộ nhớ (4 chu kỳ), 20% lệnh rẽ nhánh (2 chu kỳ) và 10% lệnh nhân/chia (12 chu kỳ). Hãy tính: a) CPI trung bình; b) thời gian thực hiện chương trình; c) MIPS đạt được; d) nếu cải tiến phần cứng để lệnh nhân/chia chỉ còn 6 chu kỳ thì chương trình tăng tốc bao nhiêu lần?

E. Lập trình hợp ngữ

Gợi ý chung cho nhóm E. Cả nhóm dùng tập lệnh ở Phụ lục B. Nên mở *Simple 8-bit Assembler Simulator* (Phụ lục C) để gõ vào và chạy thử: sai ở đâu thấy ngay ở đó. Ba khuôn mẫu dùng đi dùng lại: *nếu...thì* là CMP rồi J...; *lặp* là đếm lùi bằng DEC rồi JNZ; *duyet mảng* là một thanh ghi làm con trỏ với [B] và INC B.

17. Dò tay chương trình dưới đây. Lập bảng theo dõi A, B và cho biết giá trị cuối cùng của A, B, D, cờ Z, và nội dung các ô nhớ 200, 201, 202.

```
MOV A, 3
MOV B, 200
MOV [B], A
SHL A, 1
INC B
MOV [B], A
SHL A, 1
INC B
MOV [B], A
CMP A, 12
JZ ket
```

```
MOV D, 99
ket:
HLT
```

18. Ô nhớ 100 và 101 chứa hai số không dấu. Viết chương trình tính *hiệu của số lớn trừ số nhỏ* và ghi vào ô nhớ 102, sao cho kết quả luôn dương dù hai số vào theo thứ tự nào. Kiểm tra với 37 và 92 – nhớ dùng hai lệnh `MOV` để nạp sẵn hai số đó vào ô nhớ trước khi tính, vì trình mô phỏng không cho sửa bộ nhớ bằng chuột.
19. Cho mảng sáu phần tử đặt sẵn bằng `DB: 4, 25, 10, 11, 3, 200`. Viết chương trình đếm xem có bao nhiêu phần tử *lớn hơn 10*, ghi kết quả vào ô nhớ 200. (Chú ý: “lớn hơn 10” khác “lớn hơn hoặc bằng 10”.)
20. Viết chương trình con tên `nhân3` nhân ba giá trị trong thanh ghi `A`, chỉ dùng phép dịch và phép cộng, không dùng `MUL`. Chương trình con phải trả lại mọi thanh ghi khác nguyên như cũ. Gọi nó hai lần, với $A = 5$ rồi $A = 20$, cất hai kết quả vào ô 200 và 201.
21. *Thực hành trên trang mô phỏng*. Gõ chương trình ở bài 17 vào *Simple 8-bit Assembler Simulator* (Phụ lục C) rồi bấm **Assemble**.
 - a) Chương trình chiếm bao nhiêu byte trong bộ nhớ? Tính bằng tay trước (mỗi lệnh tốn 1 byte mã lệnh cộng 1 byte cho mỗi toán hạng), rồi kiểm lại trên màn hình: bảng **Labels** cho biết nhãn `ket` nằm ở địa chỉ nào. Vì sao số byte lại khác số dòng lệnh?
 - b) Bấm **Step** liên tục cho tới khi chương trình dừng, đếm số lần bấm. Chương trình đã thực hiện bao nhiêu lệnh? Con số đó ứng với ký hiệu nào trong công thức hiệu năng ở mục 2.6.4? Vì sao nó khác số dòng lệnh trong mã nguồn?
 - c) Giả sử đem chương trình này chạy trên một bộ xử lý thật có bảng CPI sau: lệnh chỉ làm việc giữa thanh ghi và hằng số (`MOV A, 3`, `SHL`, `INC`, `CMP`) tốn 2 chu kỳ; lệnh ghi ra bộ nhớ (`MOV [B], A`) tốn 4 chu kỳ; lệnh nhảy tốn 3 chu kỳ; `HLT` tốn 1 chu kỳ. Hãy tính tổng số chu kỳ và CPI trung bình.
 - d) Bộ xử lý chạy ở 1 GHz. Tính thời gian thực hiện chương trình.
 - e) Vì sao lệnh ghi ra bộ nhớ lại tốn nhiều chu kỳ hơn lệnh chỉ làm việc giữa các thanh ghi?

Đáp án và hướng dẫn giải

Phần này trình bày lời giải đầy đủ, không chỉ đáp số. Nếu kết quả của bạn khác, hãy dò từng bước xem lệch từ chỗ nào – thường là ở khâu đổi đơn vị hoặc ở chỗ lập tỉ số bị ngược.

A. Chu trình lệnh và các thanh ghi

Bài 1. a) $2^{32} = 4\,294\,967\,296$ byte = 4 GB. b) PC và MAR phải chứa được địa chỉ nên rộng tối thiểu 32 bit; MBR nối với bus dữ liệu nên rộng 64 bit. c) Lệnh dài 4 byte, bộ nhớ đánh địa chỉ theo byte, nên $PC \leftarrow (0040\,1000)_{16} + 4 = (0040\,1004)_{16}$.

Bài 2. Lệnh trừ nội dung ngăn nhớ $(0310)_{16}$ khỏi R3. Kết quả $(0050)_{16} - (0012)_{16} = (003E)_{16}$.

Công đoạn	Vi thao tác chính	MAR	MBR	PC	R3
(ban đầu)	–	–	–	$(0200)_{16}$	$(0050)_{16}$
Nhận lệnh	$MAR \leftarrow PC$; đọc; $IR \leftarrow MBR$; $PC + 4$	$(0200)_{16}$	mã lệnh	$(0204)_{16}$	$(0050)_{16}$
Giải mã	CU nhận dạng lệnh SUB	$(0200)_{16}$	mã lệnh	$(0204)_{16}$	$(0050)_{16}$
Nhận toán hạng	$MAR \leftarrow (0310)_{16}$; đọc	$(0310)_{16}$	$(0012)_{16}$	$(0204)_{16}$	$(0050)_{16}$
Thực hiện	$R3 \leftarrow R3 - MBR$	$(0310)_{16}$	$(0012)_{16}$	$(0204)_{16}$	$(003E)_{16}$
Cất toán hạng	không có	$(0310)_{16}$	$(0012)_{16}$	$(0204)_{16}$	$(003E)_{16}$

Các cờ sau lệnh: kết quả $(003E)_{16}$ khác 0 và dương nên $ZF = 0$, $SF = 0$; không mượn nên $CF = 0$; không tràn nên $OF = 0$.

Bài 3. Dãy thao tác trên bus:

1. phát địa chỉ $(1000)_{16}$, phát MEMR, đọc về $(2000)_{16}$ – chu kỳ bus 1;
2. phát địa chỉ $(2000)_{16}$, phát MEMR, đọc về $(00AB)_{16}$ – chu kỳ bus 2;
3. $R1 \leftarrow (00AB)_{16}$.

Kể cả chu kỳ bus dùng để nhận lệnh thì lệnh này tốn 3 chu kỳ bus; riêng phần nhận toán hạng tốn 2 chu kỳ, nhiều hơn chế độ trực tiếp đúng 1 chu kỳ bus.

Bài 4. Mỗi mục 2 byte: PUSH thì SP giảm 2, POP thì SP tăng 2.

Lệnh	Thao tác	SP sau lệnh
(ban đầu)	–	$(3000)_{16}$
PUSH AX	cất 2 byte	$(2FFE)_{16}$
PUSH BX	cất 2 byte	$(2FFC)_{16}$
CALL SUB	cất địa chỉ trở về	$(2FFA)_{16}$
PUSH CX	cất 2 byte	$(2FF8)_{16}$
POP CX	lấy ra 2 byte	$(2FFA)_{16}$
RET	lấy địa chỉ trở về nạp vào PC	$(2FFC)_{16}$

Nếu quên POP CX, tại thời điểm gặp RET thì $SP = (2FF8)_{16}$ và đỉnh ngăn xếp đang chứa giá trị của CX chứ không phải địa chỉ trở về. Lệnh RET sẽ nạp giá trị CX vào PC, tức là CPU nhảy

tới một địa chỉ vô nghĩa và chương trình chạy loạn. Đây là lỗi mất cân bằng ngăn xếp: mỗi PUSH trong chương trình con phải có đúng một POP tương ứng trước khi RET.

B. Đơn vị số học và logic

Bài 5. Bảng chân lý xem bảng 2.1. Biểu thức: $S = A \oplus B \oplus C_{in}$ và $C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$.

Ghép từ hai bộ cộng bán phần: bộ thứ nhất cộng A và B , cho $S_1 = A \oplus B$ và $C_1 = A \cdot B$; bộ thứ hai cộng S_1 với C_{in} , cho $S = S_1 \oplus C_{in}$ và $C_2 = S_1 \cdot C_{in}$. Cuối cùng $C_{out} = C_1 + C_2$, tức chỉ cần thêm một cổng OR.

Bài 6.

	Phép tính	Kết quả 8 bit	Giá trị	ZF	SF	CF	OF
a)	$45 - 78$	1101 1111	-33	0	1	1	0
b)	$(-60) + (-70)$	0111 1110	+126	0	0	1	1
c)	$96 + 32$	1000 0000	-128	0	1	0	1
d)	$100 - 100$	0000 0000	0	1	0	0	0

Hướng dẫn: a) $45 = 0010\ 1101$, $-78 = 1011\ 0010$, tổng là $1101\ 1111 = -33$; khác dấu nên không tràn, phép trừ có mượn nên $CF = 1$. b) $-60 = 1100\ 0100$, $-70 = 1011\ 1010$; tổng thật là -130 , vượt dưới -128 nên tràn có dấu, $OF = 1$; kết quả 8 bit là $0111\ 1110 = +126$. c) $96 + 32 = 128 > 127$: hai số dương cho kết quả âm nên $OF = 1$; hiểu là số không dấu thì 128 vẫn biểu diễn được nên $CF = 0$. d) Kết quả 0 nên $ZF = 1$; trừ hai số bằng nhau, không mượn, $CF = 0$.

Bài 7. $A = 1001\ 0110$, hiểu là số bù hai thì $A = -106$.

	Phép	Kết quả	Giá trị (bù hai)
a)	SHL A , 1	0010 1100	+44 – sai so với -212 vì đã tràn
b)	SHR A , 1	0100 1011	+75 – sai, vì SHR dành cho số không dấu
c)	SAR A , 1	1100 1011	-53 – đúng bằng phần nguyên của $-106/2$
d)	ROR A , 2	1010 0101	-91

Câu b) và c) chính là minh họa điển hình cho việc dùng nhầm lệnh dịch: cùng một dãy bit, SHR cho +75 còn SAR cho -53; chỉ SAR mới đúng với số có dấu.

Bài 8. a) $12 = 8 + 4$, nên $A \times 12 = (A \ll 3) + (A \ll 2)$. b) $20 = 16 + 4$, nên $A \times 20 = (A \ll 4) + (A \ll 2)$. c) $15 = 16 - 1$, nên $A \times 15 = (A \ll 4) - A$ – chỉ một phép dịch và một phép trừ, rẻ hơn cách $8 + 4 + 2 + 1$ tốn ba phép cộng.

Kiểm tra c) với $A = 9$: $A \ll 4 = 144$, $144 - 9 = 135 = 9 \times 15$. Đúng.

C. Khối điều khiển và hệ thống bus

Bài 9. Độ rộng một vi lệnh = $56 + 7 + 3 = 66$ bit. Dung lượng bộ nhớ điều khiển:

$$2048 \times 66 = 135\,168 \text{ bit} = 16\,896 \text{ byte} = 16,5 \text{ KiB}.$$

Bài 10. Dãy vi thao tác (mỗi dòng là một nhịp):

$$\begin{aligned} t_1 : & \quad MAR \leftarrow PC \\ t_2 : & \quad MBR \leftarrow M[MAR]; \quad PC \leftarrow PC + \Delta \\ t_3 : & \quad IR \leftarrow MBR \\ t_4 : & \quad MAR \leftarrow IR(\text{địa chỉ}) \\ t_5 : & \quad MBR \leftarrow M[MAR] \\ t_6 : & \quad Y \leftarrow MBR \\ t_7 : & \quad Z \leftarrow R1 - Y \\ t_8 : & \quad R1 \leftarrow Z \end{aligned}$$

Tổng cộng 8 nhịp, trong đó 3 nhịp cho pha nhận lệnh, 2 nhịp cho pha nhận toán hạng và 3 nhịp cho pha thực hiện. Với tổ chức ba bus, ba nhịp t_6-t_8 gộp lại còn một nhịp, lệnh chỉ còn 6 nhịp.

Bài 11. a) $2^{24} = 16\,777\,216 \text{ byte} = 16 \text{ MB}$. b) Mỗi chu kỳ bus chuyển 32 bit = 4 byte trong 10 ns, nên tốc độ là

$$\frac{4 \text{ byte}}{10 \times 10^{-9} \text{ s}} = 4 \times 10^8 \text{ byte/s} = 400 \text{ MB/s},$$

trong đó tốc độ truyền quy ước lấy $1 \text{ MB} = 10^6 \text{ byte}$. c) Bus dữ liệu 64 đường chuyển 8 byte mỗi chu kỳ, tốc độ tăng gấp đôi thành 800 MB/s. Dung lượng nhớ tối đa *không đổi*, vì nó chỉ phụ thuộc số đường địa chỉ.

D. Hiệu năng CPU

Bài 12.

a) Số lệnh trong một giây $IPS = f_0/CPI$:

$$\begin{aligned} IPS_{P1} &= \frac{2 \times 10^9}{1,5} \approx 1,33 \times 10^9, \\ IPS_{P2} &= \frac{1,5 \times 10^9}{1,0} = 1,50 \times 10^9, \\ IPS_{P3} &= \frac{3 \times 10^9}{2,5} = 1,20 \times 10^9. \end{aligned}$$

Tương ứng 1333,3 MIPS, 1500 MIPS và 1200 MIPS. Vậy **P2 có hiệu năng cao nhất**, dù tốc độ xung nhịp của nó thấp nhất.

b) Trong 10 giây, số chu kỳ $n = f_0 \times t$ và số lệnh $IC = n/CPI$:

	Số chu kỳ	Số lệnh
P1	$2,0 \times 10^{10}$	$1,33 \times 10^{10}$
P2	$1,5 \times 10^{10}$	$1,50 \times 10^{10}$
P3	$3,0 \times 10^{10}$	$1,20 \times 10^{10}$

c) Số lệnh không đổi. Từ $t = IC \times CPI/f_0$, muốn $t' = 0,7t$ với $CPI' = 1,2 CPI$ thì

$$f'_0 = \frac{1,2}{0,7} f_0 \approx 1,714 f_0.$$

Do đó $f'_{P1} \approx 3,43 \text{ GHz}$; $f'_{P2} \approx 2,57 \text{ GHz}$; $f'_{P3} \approx 5,14 \text{ GHz}$.

Bài 13.

a) $IPC = IC/n$ với $n = f_0 \times t$:

$$IPC_{P1} = \frac{20 \times 10^9}{2 \times 10^9 \times 7} = \frac{20}{14} \approx 1,43,$$

$$IPC_{P2} = \frac{30 \times 10^9}{1,5 \times 10^9 \times 10} = \frac{30}{15} = 2,00,$$

$$IPC_{P3} = \frac{90 \times 10^9}{3 \times 10^9 \times 9} = \frac{90}{27} \approx 3,33.$$

Cả ba đều lớn hơn 1, tức đây là các bộ xử lý siêu vô hướng.

b) Giữ nguyên IC và IPC của P2 thì số chu kỳ cần là $n = 30 \times 10^9 / 2,00 = 15 \times 10^9$. Muốn chạy hết trong 7 s:

$$f'_0 = \frac{15 \times 10^9}{7} \approx 2,14 \text{ GHz}.$$

c) Trong 9 s, P2 ở 1,5 GHz cho $n = 1,5 \times 10^9 \times 9 = 13,5 \times 10^9$ chu kỳ. Với $IPC = 2,00$:

$$IC = 13,5 \times 10^9 \times 2,00 = 27 \times 10^9 \text{ lệnh}.$$

Bài 14.

a) và b) Hai bộ xử lý cùng số lệnh, nên so sánh qua số chu kỳ và tần số. Trước hết tính CPI trung bình, $CPI_{TB} = \sum(CPI_i \times \text{tỉ lệ}_i)$:

$$CPI_{P1} = 0,1 \times 1 + 0,2 \times 2 + 0,5 \times 3 + 0,2 \times 4 = 2,8,$$

$$CPI_{P2} = 0,1 \times 2 + 0,2 \times 2 + 0,5 \times 2 + 0,2 \times 3 = 2,2.$$

c) Số chu kỳ $n = IC \times CPI_{TB}$: $n_{P1} = 2,8 \times 10^6$ và $n_{P2} = 2,2 \times 10^6$.

Thời gian thực hiện:

$$t_{P1} = \frac{2,8 \times 10^6}{1,5 \times 10^9} \approx 1,867 \text{ ms}, \quad t_{P2} = \frac{2,2 \times 10^6}{2 \times 10^9} = 1,100 \text{ ms}.$$

Vậy **P2 thực hiện chương trình này nhanh hơn** – đáp số của câu a) – và nhanh hơn $1,867/1,100 \approx 1,70$ lần.

d) Số chu kỳ của chương trình:

$$n = 500 \times 1 + 50 \times 5 + 100 \times 5 + 50 \times 2 = 500 + 250 + 500 + 100 = 1350.$$

Trên bộ xử lý 2 GHz:

$$t = \frac{1350}{2 \times 10^9} = 675 \text{ ns}.$$

e) $CPI = 1350/700 \approx 1,93$.

f) Load giảm còn 50 lệnh: $IC' = 650$ và

$$n' = 500 + 250 + 50 \times 5 + 100 = 1100, \quad t' = \frac{1100}{2 \times 10^9} = 550 \text{ ns}.$$

Tăng tốc $= t/t' = 1350/1100 \approx 1,23$ lần; $CPI' = 1100/650 \approx 1,69$.

Bài 15.

a) Hiệu suất đỉnh điểm đạt được khi chuỗi lệnh chỉ gồm lớp lệnh có CPI nhỏ nhất. Với P1, lớp A có $CPI = 1$:

$$IPS_{P1} = \frac{1,0 \times 10^9}{1} = 1,0 \times 10^9 \text{ lệnh/s} = 1000 \text{ MIPS}.$$

Với P2, CPI nhỏ nhất là 2 (các lớp A, B, C):

$$IPS_{P2} = \frac{1,5 \times 10^9}{2} = 0,75 \times 10^9 \text{ lệnh/s} = 750 \text{ MIPS}.$$

b) Tỷ lệ các lớp là $A : B : C : D : E = 2 : 1 : 1 : 1 : 1$, tổng 6 phần.

$$CPI_{P1} = \frac{2 \cdot 1 + 1 \cdot 2 + 1 \cdot 3 + 1 \cdot 4 + 1 \cdot 3}{6} = \frac{14}{6} \approx 2,333,$$

$$CPI_{P2} = \frac{2 \cdot 2 + 1 \cdot 2 + 1 \cdot 2 + 1 \cdot 4 + 1 \cdot 4}{6} = \frac{16}{6} \approx 2,667.$$

Thời gian cho mỗi lệnh:

$$\frac{t_{P1}}{IC} = \frac{2,333}{10^9} = 2,333 \text{ ns}, \quad \frac{t_{P2}}{IC} = \frac{2,667}{1,5 \times 10^9} = 1,778 \text{ ns}.$$

Vậy **P2 nhanh hơn**, và nhanh hơn $2,333/1,778 = 1,3125$ lần.

c) Tỷ lệ $A : B : C : D : E = 1 : 1 : 1 : 1 : 2$, tổng 6 phần.

$$CPI_{P1} = \frac{1 + 2 + 3 + 4 + 2 \cdot 3}{6} = \frac{16}{6} \approx 2,667,$$

$$CPI_{P2} = \frac{2 + 2 + 2 + 4 + 2 \cdot 4}{6} = \frac{18}{6} = 3,000.$$

$$\frac{t_{P1}}{IC} = 2,667 \text{ ns}, \quad \frac{t_{P2}}{IC} = \frac{3,000}{1,5} = 2,000 \text{ ns}.$$

Vậy **P2 nhanh hơn** $2,667/2,000 \approx 1,33$ lần.

d) Với Compute = 1, Load = Store = 10, Branch = 3 chu kỳ:

$$n_1 = 1000 \cdot 1 + 400 \cdot 10 + 100 \cdot 10 + 50 \cdot 3 = 1000 + 4000 + 1000 + 150 = 6150,$$

$$n_2 = 1500 \cdot 1 + 300 \cdot 10 + 100 \cdot 10 + 100 \cdot 3 = 1500 + 3000 + 1000 + 300 = 5800.$$

Trên bộ xử lý 3 GHz:

$$t_1 = \frac{6150}{3 \times 10^9} = 2,05 \mu s, \quad t_2 = \frac{5800}{3 \times 10^9} \approx 1,93 \mu s.$$

e) Với Load = Store = 2 chu kỳ:

$$n_1 = 1000 + 800 + 200 + 150 = 2150, \quad n_2 = 1500 + 600 + 200 + 300 = 2600,$$

$$t_1 = \frac{2150}{3 \times 10^9} \approx 717 \text{ ns}, \quad t_2 = \frac{2600}{3 \times 10^9} \approx 867 \text{ ns}.$$

Ở câu d) và e), thứ tự nhanh–chậm giữa hai chương trình *đảo ngược*: khi Load/Store tốn 10 chu kỳ thì chương trình 2 nhanh hơn, còn khi chỉ tốn 2 chu kỳ thì chương trình 1 nhanh hơn. Lý do là chương trình 1 có tỷ lệ lệnh truy nhập bộ nhớ cao hơn, nên nó được lợi nhiều hơn khi bộ nhớ nhanh lên. Đây là minh họa rõ ràng cho nhận định ở mục 2.6.1: hiệu năng luôn gắn với một chương trình cụ thể.

Bài 16.

$$a) CPI_{TB} = 0,45 \times 1 + 0,25 \times 4 + 0,20 \times 2 + 0,10 \times 12 = 0,45 + 1,00 + 0,40 + 1,20 = 3,05.$$

$$b) \text{Số chu kỳ } n = 8 \times 10^8 \times 3,05 = 2,44 \times 10^9, \text{ do đó}$$

$$t = \frac{2,44 \times 10^9}{2,5 \times 10^9} = 0,976 \text{ s}.$$

$$c) MIPS = \frac{f_0}{CPI \times 10^6} = \frac{2,5 \times 10^9}{3,05 \times 10^6} \approx 819,7 \text{ MIPS}.$$

$$d) CPI' = 0,45 + 1,00 + 0,40 + 0,10 \times 6 = 2,45, \text{ nên}$$

$$\text{tăng tốc} = \frac{3,05}{2,45} \approx 1,24 \text{ lần.}$$

Nhận xét: lệnh nhân/chia chỉ chiếm 10% số lệnh nhưng chiếm tới $1,20/3,05 \approx 39\%$ tổng số chu kỳ, nên cải tiến chúng đem lại hiệu quả lớn. Đây chính là nội dung của định luật Amdahl: nên tối ưu phần chiếm nhiều thời gian nhất.

E. Lập trình hợp ngữ

Bài 17. Bảng theo dõi:

#	Lệnh	A	B	Z	Bộ nhớ vừa ghi
1	MOV A, 3	3	–	–	
2	MOV B, 200	3	200	–	
3	MOV [B], A	3	200	–	ô 200 ← 3
4	SHL A, 1	6	200	0	
5	INC B	6	201	0	
6	MOV [B], A	6	201	0	ô 201 ← 6
7	SHL A, 1	12	201	0	
8	INC B	12	202	0	
9	MOV [B], A	12	202	0	ô 202 ← 12
10	CMP A, 12	12	202	1	12 – 12 = 0 nên Z = 1
11	JZ ket	12	202	1	Z = 1 nên nhảy, bỏ qua dòng 12
12	HLT	12	202	1	

Đáp số: $A = 12, B = 202, D = 0, Z = 1; \text{ô } 200 = 3, \text{ô } 201 = 6, \text{ô } 202 = 12.$

Điểm mấu chốt: lệnh MOV D, 99 *không bao giờ chạy*, vì JZ đã nhảy qua nó. Chương trình này ghi ra dãy 3, 6, 12 – mỗi lần dịch trái một bit là nhân đôi.

Bài 18. Ý tưởng: so sánh trước, nếu số bé đứng trước thì đổi chỗ hai số, sau đó mới trừ. Dùng C làm chỗ để tạm khi đổi chỗ.

```
MOV [100], 37 ; nạp sẵn dữ liệu để chạy thử
MOV [101], 92
MOV A, [100]
MOV B, [101]
CMP A, B
JNC tru ; C = 0 tức A >= B, trừ được ngay
MOV C, A ; đổi chỗ A và B
MOV A, B
MOV B, C
```

```

MOV A, B
MOV B, C
tru:
SUB A, B
MOV [102], A
HLT

```

Kiểm tra với ô 100 = 37 và ô 101 = 92: CMP cho $37 < 92$ nên $C = 1$, không nhảy, hai số đổi chỗ thành $A = 92, B = 37$; kết quả $92 - 37 = 55$ ghi vào ô 102.

Nếu bỏ phần đổi chỗ mà trừ thẳng, ta được $37 - 92 = -55$; trên 8 bit không dấu con số đó hiện ra thành 201, và cờ $C = 1$ báo đã phải mượn. Đây đúng là tình huống ở ví dụ 2.2 – máy không sai, chỉ là ta đọc kết quả theo cách không phù hợp.

Bài 19.

```

MOV B, day      ; con trỏ chạy trên mảng
MOV C, 6        ; còn 6 phần tử
MOV D, 0        ; kết quả đếm
lap:
MOV A, [B]
CMP A, 10
JBE bo          ; A <= 10 thì không tính
INC D
bo:
INC B
DEC C
JNZ lap
MOV [200], D
HLT

day: DB 4
     DB 25
     DB 10
     DB 11
     DB 3
     DB 200

```

Các phần tử lớn hơn 10 là 25, 11 và 200, nên ô 200 nhận giá trị **3**.

Chỗ dễ sai nhất là chọn lệnh nhảy. Đề hỏi “lớn hơn 10”, nên phần tử *bằng* 10 phải bị loại; do đó điều kiện bỏ qua là “nhỏ hơn hoặc bằng”, tức JBE. Nếu dùng JC (chỉ nhỏ hơn) thì số 10 sẽ bị đếm nhầm và kết quả ra 4.

Bài 20. $A \times 3 = A \times 2 + A = (A \ll 1) + A$. Cần giữ lại giá trị A ban đầu, nên chép sang B trước khi dịch – và vì mượn B nên phải PUSH/POP để trả lại.

```

MOV A, 5
CALL nhan3
MOV [200], A    ; 15
MOV A, 20
CALL nhan3
MOV [201], A    ; 60
HLT

nhan3:
    PUSH B      ; mượn B thì phải giữ hộ
    MOV B, A    ; B giữ giá trị gốc
    SHL A, 1    ; A = 2A
    ADD A, B    ; A = 2A + A = 3A
    POP B      ; trả B nguyên vẹn
    RET
    
```

Kết quả: ô 200 = 15, ô 201 = 60, và SP trở về đúng 231 khi chương trình dừng – dấu hiệu ngăn xếp cân bằng.

Bài 21.

a) Chương trình chiếm **34 byte**, trong khi chỉ có **13 dòng lệnh**. Cộng theo độ dài từng lệnh:

Dạng lệnh	Số byte	Trong chương trình
Hai toán hạng (MOV A, 3, MOV [B], A, SHL, CMP)	3	9 lệnh → 27 byte
Một toán hạng (INC B, JZ ket)	2	3 lệnh → 6 byte
Không toán hạng (HLT)	1	1 lệnh → 1 byte
Tổng		34 byte

Kiểm lại trên màn hình: bảng **Labels** cho thấy nhãn ket nằm ở địa chỉ **33**, mà lệnh HLT ở đó chiếm 1 byte, nên chương trình dùng hết các ô nhớ từ 0 đến 33, tức 34 byte.

Số byte khác số dòng lệnh vì *các lệnh dài ngắn khác nhau* – điều này cũng giải thích vì sao PC không phải lúc nào cũng tăng 1, mà tăng đúng bằng độ dài lệnh vừa nhận, như đã nói ở mục 2.1.4.

b) Phải bấm **Step** đúng **12 lần** thì chương trình mới dừng. Đây chính là *IC* (Instruction Count) trong công thức $t_{CPU} = IC \times CPI \times T_0$.

Chú ý $IC = 12$ chứ không phải 13, vì lệnh MOV D, 99 *không bao giờ chạy* – lệnh JZ đã nhảy qua nó, đúng như đã thấy ở bài 17. Số dòng lệnh viết trong mã nguồn và số lệnh thực sự chạy là hai chuyện khác nhau; công thức hiệu năng dùng con số thứ hai.

c) Đếm số lệnh từng loại theo bảng dò tay ở bài 17:

Loại lệnh	Số lệnh	CPI	Số chu kỳ
Thanh ghi và hằng số (MOV A, 3, MOV B, 200, 2 lệnh SHL, 2 lệnh INC, CMP)	7	2	14
Ghi ra bộ nhớ (3 lệnh MOV [B], A)	3	4	12
Nhảy (JZ)	1	3	3
Dừng (HLT)	1	1	1
Tổng	12		30

Áp dụng công thức (2.5):

$$CPI_{TB} = \frac{30}{12} = 2,5.$$

d) Máy chạy ở 1 GHz nên $T_0 = 1/10^9 = 1 \text{ ns}$. Áp dụng công thức (2.3):

$$t_{CPU} = IC \times CPI \times T_0 = 12 \times 2,5 \times 1 \text{ ns} = 30 \text{ ns}.$$

Cũng ra đúng như vậy nếu lấy thẳng tổng số chu kỳ nhân T_0 : $30 \times 1 = 30 \text{ ns}$.

e) Vì lệnh ghi ra bộ nhớ phải đi ra ngoài CPU. Nó tốn thêm một chu kỳ bus: đưa địa chỉ ra bus địa chỉ, đưa dữ liệu ra bus dữ liệu, phát tín hiệu MEMW, rồi chờ bộ nhớ ghi xong. Còn lệnh làm việc giữa các thanh ghi thì mọi thứ diễn ra ngay bên trong CPU, chỉ đi qua bus trong – nhanh hơn hẳn, đúng như bảng so sánh tốc độ ở mục 2.1.1 đã cho thấy.

Đây chính là lý do các loại lệnh có CPI khác nhau, và vì sao phải tính CPI trung bình. Nó cũng cho thấy vì sao giữ được dữ liệu trong thanh ghi thì chương trình chạy nhanh hơn – một nhận xét sẽ còn gặp lại ở chương 4 khi học bộ nhớ đệm.

Bảng CPI trong đề là *giả định* cho dễ tính. Bộ xử lý thật dùng ống lệnh nên CPI của nó nhỏ hơn nhiều, thường quanh 1 (xem Phụ lục A). Điều đáng nhớ ở bài này không phải con số 2,5, mà là *cách* đi từ số lệnh và bảng CPI sang thời gian chạy thật.

Bảng thuật ngữ Việt – Anh

Tài liệu chuyên ngành và tài liệu tra cứu hầu hết viết bằng tiếng Anh, nên bạn cần quen với cả hai cách gọi. Bảng dưới đây gom lại những thuật ngữ đã xuất hiện trong chương.

Tiếng Việt	Tiếng Anh	Viết tắt / ghi chú
Bộ xử lý trung tâm	Central Processing Unit	CPU
Đơn vị điều khiển	Control Unit	CU

Tiếng Việt	Tiếng Anh	Viết tắt / ghi chú
Đơn vị số học và logic	Arithmetic and Logic Unit	ALU
Tập thanh ghi	Register File	RF
Đơn vị dấu phẩy động	Floating Point Unit	FPU
Chu trình lệnh	Instruction cycle	6 công đoạn
Nhận lệnh	Fetch instruction	công đoạn 1
Giải mã lệnh	Decode instruction	công đoạn 2
Nhận toán hạng	Fetch operand	công đoạn 3
Thực hiện lệnh	Execute instruction	công đoạn 4
Cất toán hạng	Store operand	công đoạn 5
Ngắt	Interrupt	công đoạn 6
Toán hạng	Operand	dữ liệu mà lệnh làm việc trên đó
Vi thao tác	Micro-operation	việc nhỏ nhất trong 1 nhịp
Hợp ngữ	Assembly language	mỗi dòng = một lệnh máy
Trình hợp dịch	Assembler	dịch hợp ngữ ra mã máy
Từ gợi nhớ	Mnemonic	MOV, ADD, JMP...
Toán hạng đích	Destination operand	viết trước dấu phẩy
Toán hạng nguồn	Source operand	viết sau dấu phẩy
Nhãn	Label	tên đặt cho một địa chỉ
Chế độ tức thì	Immediate	hằng số nằm trong lệnh
Chương trình con	Subroutine	gọi bằng CALL, về bằng RET
Từ điều khiển	Control word	1 hàng bit, 1 nhịp
Vi lệnh	Microinstruction	tên khác của từ điều khiển
Vi chương trình	Microprogram	dãy vi lệnh của một lệnh máy
Tập lệnh rút gọn	Reduced Instruction Set Computer	RISC
Tập lệnh phức tạp	Complex Instruction Set Computer	CISC
Điều khiển mạch cứng	Hardwired control	
Bộ nhớ điều khiển	Control memory	chứa vi chương trình
Bộ đếm chương trình	Program Counter	PC (còn gọi IP)
Thanh ghi lệnh	Instruction Register	IR
Thanh ghi địa chỉ bộ nhớ	Memory Address Register	MAR

Tiếng Việt	Tiếng Anh	Viết tắt / ghi chú
Thanh ghi đệm dữ liệu	Memory Buffer Register	MBR
Con trỏ dữ liệu	Data Pointer	DP
Con trỏ ngăn xếp	Stack Pointer	SP
Thanh ghi cơ sở	Base Register	
Thanh ghi chỉ số	Index Register	
Thanh ghi đa năng	General Purpose Register	
Từ trạng thái chương trình	Program Status Word	PSW
Chế độ địa chỉ	Addressing mode	trực tiếp, gián tiếp, ...
Chu kỳ bus	Bus cycle	một lần trao đổi trên bus
Thanh ghi trạng thái	Flag Register	còn gọi thanh ghi cờ
Ngăn xếp	Stack	cấu trúc LIFO
Cất vào ngăn xếp	Push	SP giảm
Lấy ra khỏi ngăn xếp	Pop	SP tăng
Cờ rỗng	Zero Flag	ZF
Cờ dấu	Sign Flag	SF
Cờ nhớ	Carry Flag	CF – tràn số không dấu
Cờ tràn	Overflow Flag	OF – tràn số có dấu
Cờ cho phép ngắt	Interrupt Flag	IF
Cờ hướng	Direction Flag	DF
Cờ bẫy	Trap Flag	TF
Mã bù một	One's complement	đảo toàn bộ bit
Mã bù hai	Two's complement	bù một cộng 1
Bộ cộng bán phần	Half adder	cộng 2 bit
Bộ cộng toàn phần	Full adder	cộng 3 bit
Bộ cộng nối tiếp	Ripple-carry adder	bit nhớ lan dần
Bộ cộng nhìn trước bit nhớ	Carry-lookahead adder	tính trước bit nhớ
Bit nhớ	Carry	C
Dịch trái logic	Logical shift left	SHL – nhân 2
Dịch phải logic	Logical shift right	SHR – số không dấu
Dịch phải số học	Arithmetic shift right	SAR – số có dấu
Quay trái, quay phải	Rotate left, rotate right	ROL, ROR
Bus dữ liệu	Data bus	hai chiều
Bus địa chỉ	Address bus	một chiều

Tiếng Việt	Tiếng Anh	Viết tắt / ghi chú
Bus điều khiển	Control bus	các đường tín hiệu riêng
Đọc bộ nhớ	Memory Read	MEMR
Ghi bộ nhớ	Memory Write	MEMW
Đọc cổng vào-ra	I/O Read	IOR
Ghi cổng vào-ra	I/O Write	IOW
Hiệu năng	Performance	$P = 1/t$
Xung nhịp	Clock	
Chu kỳ xung nhịp	Clock period	T_0
Tần số xung nhịp	Clock rate	f_0
Số lệnh của chương trình	Instruction Count	IC
Số nhịp trên một lệnh	Cycles Per Instruction	CPI
Số lệnh trên một nhịp	Instructions Per Cycle	$IPC = 1/CPI$
Số lệnh mỗi giây	Instructions Per Second	IPS
Số triệu lệnh mỗi giây	Millions of Instructions Per Second	MIPS
Hiệu suất đỉnh điểm	Peak performance	
Tăng tốc	Speedup	tỉ số thời gian cũ trên mới
Ống dẫn lệnh	Pipeline	5 tầng IF/ID/EX/MEM/WB
Thông lượng	Throughput	số việc xong mỗi đơn vị thời gian
Xung đột cấu trúc	Structural hazard	tranh tài nguyên
Xung đột dữ liệu	Data hazard	chờ kết quả lệnh trước
Xung đột điều khiển	Control hazard	do lệnh rẽ nhánh
Siêu vô hướng	Superscalar	nhiều ống lệnh song song

HẾT CHƯƠNG 2